UNIVERSITY OF Southampton

University of Southampton Research Repository

Copyright © and Moral Rights for this thesis and, where applicable, any accompanying data are retained by the author and/or other copyright owners. A copy can be downloaded for personal non-commercial research or study, without prior permission or charge. This thesis and the accompanying data cannot be reproduced or quoted extensively from without first obtaining permission in writing from the copyright holder/s. The content of the thesis and accompanying research data (where applicable) must not be changed in any way or sold commercially in any format or medium without the formal permission of the copyright holder/s.

When referring to this thesis and any accompanying data, full bibliographic details must be given, e.g.

Thesis: Author (Year of Submission) "Full thesis title", University of Southampton, name of the University Faculty or School or Department, PhD Thesis, pagination.

Data: Author (Year) Title. URI [dataset]

UNIVERSITY OF SOUTHAMPTON

Faculty of Engineering and Physical Sciences
School of Electronics and Computer Science

A Proof-of-Space Based Architecture for Decentralised DNS Recursive Resolvers

Volume 1 of 2

by

George Giamouridis

MSc, BSc

ORCiD: 0009-0009-8353-1862

A thesis for the degree of
Doctor of Philosophy

October 2026

University of Southampton

Abstract

Faculty of Engineering and Physical Sciences
School of Electronics and Computer Science

Doctor of Philosophy

A Proof-of-Space Based Architecture for Decentralised DNS Recursive Resolvers

by George Giamouridis

The Domain Name System (DNS) is a core component of the Internet, providing the fundamental mapping of human-readable names to network addresses. Despite its importance, the system remains highly centralised and therefore at risk of censorship, surveillance, and single points of failure. Much of the focus has been on securing and decentralising DNS as a whole; however, recursive resolvers continue to rely on central infrastructure. This research proposes a decentralised DNS resolver as an alternative to traditional recursive resolvers. The system uses the Chord protocol and Distributed Ledger Technology (DLT) to distribute DNS resolution across a network of nodes, without altering existing zone authority structures. Beyond DNS, the research addresses the broader challenge of sustainability in decentralised systems by introducing a practical Proof of Space (PoSpace) protocol. The protocol guarantees security while resisting amortisation and rationality attacks, and experimental evaluation against a baseline implementation shows a 90% reduction in plot generation time, from about 15 hours to 1.64 hours on identical hardware and software. Together, these contributions expand the design of decentralised infrastructures by addressing both the application layer, through a distributed DNS resolver, and the consensus layer, through a more efficient PoSpace scheme that reduces computational costs, energy consumption, bandwidth usage, and hardware requirements, making large-scale deployment more practical.

Contents

List of Figures **ix**

List of Tables **xi**

Listings **xiii**

Declaration of Authorship **xv**

Acknowledgements **xvii**

1 Introduction **1**

2 Background **11**
2.1 The Domain Name System (DNS) . . . 11
2.1.1 DNS Hierarchical Structure . . . 12
2.1.2 DNS Governance . . . 13
2.1.3 The DNS Protocol . . . 14
2.1.4 Domain Name Registration . . . 16
2.1.5 DNS Security Extensions . . . 17
2.2 Peer-to-Peer (P2P) Architecture . . . 18
2.2.1 Overview of P2P Networks . . . 18
2.2.2 Structured P2P Networks . . . 19
2.2.3 Chord Distributed Hash Table (DHT) . . . 20
2.2.4 Transition to Contribution . . . 22
2.3 Decentralised Consensus . . . 23
2.3.1 Consensus Overview . . . 23
2.3.2 Merkle Trees . . . 23
2.3.3 Proof-of-Space Consensus . . . 24
2.3.4 Transition to Contribution . . . 26

3 Related Work **27**
3.1 Blockchain-based DNS Solutions . . . 28
3.1.1 Namecoin . . . 28
3.1.2 Blockstack . . . 29
3.1.3 Ethereum Name Service (ENS) . . . 30
3.1.4 Handshake . . . 31
3.1.5 Unstoppable Domains . . . 32
3.1.6 B-DNS . . . 33

3.1.7 Blockzone . . . 34
3.1.8 Other BDNS Proposals . . . 36
3.2 Alternative Non-Blockchain DNS Solutions . . . 39
3.3 Proof-of-Space Solutions . . . 41
3.3.1 Foundational Proof-of-Space . . . 42
3.3.2 SpaceMint . . . 43
3.3.3 Simple Proofs of Sequential Work . . . 44
3.3.4 Proofs of Space-Time . . . 44
3.3.5 Chia Network . . . 45
3.3.6 AstroChain . . . 46
3.3.7 Etherspace . . . 47
3.3.8 Filecoin . . . 48
3.4 Linking Related Work to The Research . . . 50

4 Threat Model **51**
4.1 Adversary Assumptions . . . 51
4.2 Threat Model for the Decentralised Resolver . . . 52
4.2.1 Integrity Threats . . . 52
4.2.2 Availability Threats . . . 53
4.2.3 Privacy Threats . . . 53
4.2.4 Censorship Threats . . . 54
4.3 Threat Model for the Proof-of-Space Protocol . . . 55
4.3.1 Fake Storage Commitment (Amortisation Attack) . . . 55
4.3.2 On Demand Computation (Rationality Attack) . . . 56
4.3.3 Sybil Attack via Multiple Identities . . . 56
4.4 Out-of-Scope Threats . . . 57

5 Construction of the Decentralised Resolver **59**
5.1 Formal System Specification . . . 59
5.2 High-Level Overview . . . 60
5.3 Storage Model and Node Initialisation . . . 61
5.3.1 Key Assignment via Cryptographic Hashing . . . 62
5.3.2 Chunk Placement Using the Chord DHT . . . 63
5.3.3 Node Join Procedure and Topology Update . . . 64
5.3.4 Replication for Fault Tolerance . . . 65
5.4 Processing DNS Queries . . . 66
5.4.1 Distributed Lookup and Query Validation . . . 66
5.4.2 Fallback Resolution via Traditional DNS . . . 68
5.4.3 Mechanics of the Chunk-Level Permissioned Ledger . . . 68
Consortium Membership and Churn Management . . . 68
Consensus and Validation Role . . . 68
Storage and Synchronisation . . . 69
Incentives and Sybil Resistance . . . 69
5.5 Updating DNS Records . . . 70
5.5.1 Updates Based on TTL . . . 70
5.5.2 Updates Based on Traditional DNS Fallback . . . 71
5.6 Evaluation of The Decentralised Resolver . . . 72

5.6.1 Centralisation and Single Points of Failure . . . 72
5.6.2 Censorship and Tampering . . . 73
5.6.3 Security Considerations . . . 74
5.6.4 Lack of Transparency and Control . . . 75
5.7 Chapter Summary . . . 76

6 Practical Proof of Space Construction 79
6.1 Initialisation Phase . . . 80
6.1.1 Key Generation . . . 80
6.1.2 Data Generation (Plotting) . . . 81
6.1.3 Plotting Through Iterative Hashing . . . 81
6.1.4 Merkle Tree Construction . . . 82
6.1.5 Optimization . . . 82
6.1.6 Tree Signing . . . 82
6.2 Verification Phase . . . 83
6.2.1 Challenge Generation . . . 83
6.2.2 Challenge Verification . . . 84
6.2.3 Periodic Storage Verification . . . 85
6.3 Security Evaluation . . . 86
6.3.1 Fake Storage Commitment (Amortisation Attack) . . . 86
6.3.2 On Demand Computation (Rationality Attack) . . . 87
6.3.3 Sybil Attack via Multiple Identities . . . 88
6.4 Performance Evaluation . . . 89
6.4.1 Experimental Setup: Chia Network Implementation . . . 90
6.4.2 Baseline: Chia PoSpace Performance Evaluation . . . 92
6.4.3 Proposed PoSpace Protocol Performance Evaluation . . . 93
6.4.4 Comparative Security Analysis: Chia vs. Proposed PoSpace . . . 95
6.5 System-Wide Performance and Malicious Scenario Analysis . . . 96
6.5.1 Latency and Scale-up Simulation . . . 96
6.5.2 Throughput and Resource Efficiency . . . 96
6.5.3 Malicious Scenario Analysis (Threat Model Validation) . . . 97
6.6 Chapter Summary . . . 97

7 Future Work 99
7.1 Decentralised DNS Resolver . . . 99
7.1.1 Performance Evaluation . . . 99
7.1.2 Security and Robustness . . . 100
7.1.3 Compatibility with Existing DNS Infrastructure . . . 100
7.2 Proof of Space Initialisation . . . 101
7.2.1 Risk of Fast Regeneration . . . 101
7.2.2 Time-Based Proof Validation . . . 102
7.2.3 Fairness and Resource Asymmetry . . . 102

8 Conclusions 105

Appendix A Installing and Running the Proposed PoSpace 107
Appendix A.1 Requirements . . . 107
Appendix A.2 Installation . . . 107

Appendix A.3 Running the PoSpace 108
Appendix A.4 Verifying Installation 108

Glossary **109**

References **113**

List of Figures

2.1 The hierarchical structure of DNS. 12
2.2 DNS query lifecycle from stub resolver to ANS. 15
2.3 The domain name registration proccess and components. 16
2.4 Example of a simple Merkle tree with four leaves. 24

5.1 The structure and components of the decentralised resolver. 61

6.1 The PoSpace challenge–response interaction between the Prover and the Verifier. 80
6.2 Reconstruction of the Merkle root based on the Prover's challenge response. 84
6.3 Chia PoSpace initialisation phase timings across trials. 93
6.4 Proposed PoSpace initialisation resource breakdown across trials. 95
6.5 Average total initialisation time across trials (in hours) 98

List of Tables

2.1 DNS governance hierarchy and responsibilities. 14
2.2 Finger table structure for node n_i in Chord DHT. 21

3.1 Objectives, mechanisms, supported TLDs and limitations of BDNS. . . . 36
3.2 Objectives, mechanisms, and limitations of non-blockchain DNS solutions. 41
3.3 Objectives, mechanisms, and limitations of PoSpace and related protocols. 49

4.1 Threats in the Decentralised Resolver. 58
4.2 Threats in the Proof-of-Space protocol. 58

5.1 Comparison of Traditional DNS vs. Proposed Decentralised DNS 76

6.1 Chia and proposed PoSpace initialisation performance. 90
6.2 Chia PoSpace initialisation phase performance. 92
6.3 Proposed PoSpace initialisation phase performance. 94
6.4 Security and operational trade-offs between Chia and the proposed framework. 95

Listings

Appendix A.1 Cloning and installing dependencies 107
Appendix A.2 Running the proof-of-space 108

Declaration of Authorship

I declare that this thesis and the work presented in it is my own and has been generated by me as the result of my own original research.

I confirm that:

1. This work was done wholly or mainly while in candidature for a research degree at this University;
2. Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated;
3. Where I have consulted the published work of others, this is always clearly attributed;
4. Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work;
5. I have acknowledged all main sources of help;
6. Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself;
7. Parts of this work have been published as:
   (a) G. Giamouridis, B. Kang, and L. Aniello. *"Blockchain-Based DNS: Current Solutions and Challenges to Adoption", 6th Distributed Ledger Technology Workshop*, Turin, Italy, May 2024.
   (b) G. Giamouridis, B. Kang, and L. Aniello. *"Decentralized DNS Resolution Based on Distributed Ledger Technology", 7th Distributed Ledger Technology Workshop*, Pizzo, Calabria, Italy, June 2025, accepted (to appear).
   (c) G. Giamouridis, B. Kang, and L. Aniello. *"A Practical Proof of Space Protocol for Efficient and Secure Blockchain Consensus", IET Blockchain*, submitted.

Signed: George Giamouridis Date: 25/09/2025

Acknowledgements

This work was partially supported by the UK Research and Innovation (DTP Scholarship under grant EP/T517859/1); and the Academic Centre of Excellence in Cyber Security Research - University of Southampton (EP/R007268/1).

To my family, for their endless support.

Chapter 1

Introduction

Motivation

In a Transmission Control Protocol/Internet Protocol (TCP/IP) network, each host has an assigned unique IP address, or an identifier for interfaces on computers or routers, as well as routing information for IP packets. Users could only use IP addresses, but with their increasing number and the level of complexity, memorising them has become impractical. Instead, users often use symbolic names, which are a more convenient method of referring to devices and networks. To communicate with a particular device, users need to know the corresponding IP address. This method of using symbolic names instead of IP addresses is reasonable to use for a small network of devices, but this task is impossible in large-scale networks, such as the World Wide Web.

DNS was developed to address this issue by creating an automated, scalable way of connecting human-friendly names (i.e., example.com) to network addresses (IP addresses). DNS is a primary component of the internet, and it provides users with the advantage of being able to retrieve resources by name instead of being limited to numbers. DNS resolves queries in a distributed hierarchical system using layers of servers.

The Centralisation Problem

While the initial objective of the DNS design was to create a distributed protocol, much of its infrastructure relies on centralisation. Recursive resolvers are the most significant point of centralisation, as they generate the vast majority of DNS traffic between clients and authoritative servers. As NS1's 2023 Global DNS Traffic Report (1) outlines, public resolvers generate nearly 60% of all recursive DNS traffic, with

Google Public DNS generating more than 30% and AWS accounting for 16%. While nearly 9% of DNS queries continue to be resolved by internet service providers (ISPs), such as Xfinity (2) or T-Mobile (3), this still represents a form of centralisation. ISPs also tend to be grouped by region or jurisdiction, share policies, and are indirectly affected by different regulations. The remaining percentage of DNS traffic is handled mostly by private resolvers, enterprise environments, smaller DNS providers, or other caching resolvers.

The original design of the DNS, defined in the Request for Comments (RFC) 1034 (4) and RFC 1035 (5) in the 1980s, was envisioned as a truly distributed system, with no single entity controlling the resolution process. However, due to several technical factors, the real-world implementation of DNS has moved towards a more centralised system.

Economic Incentives: Operating high-performance DNS resolvers requires significant infrastructure investment, including servers, bandwidth, monitoring, and global distribution to reduce latency (6). Large technology companies are able to absorb those costs by taking advantage of economies of scale and their existing substantial data centre infrastructure. They often provide DNS services at no charge to end users to create a competitive advantage, while collecting useful data on users that can be monetised in the form of targeted advertising, sales, intelligent marketing, or by connecting task functionality to existing services (7). This model produces centralisation, since it is difficult for smaller independent operators to provide that level of resolver services in a timely, reliable, and consistent manner. For decentralised alternatives, the lack of direct monetisation strategies creates sustainability challenges, as volunteer nodes face expenses without clear benefits.

User Experience: Public resolvers usually perform better than those provided by ISPs, especially in areas with weaker telecommunications infrastructure (8; 9). They offer lower latency, faster updates, and higher uptime, all of which improve the user experience. Centralised providers also compete by adding features such as automatic malware blocking, phishing protection, and parental controls, making their services easier to use and more attractive. As more and more people use these services, strong network effects emerge, and providers face growing pressure to maintain and expand global infrastructure. In contrast, decentralised resolvers can suffer from extra delays due to the need for coordination or differences in cached data, which can make them less competitive.

Technical Complexity: Securing DNS infrastructure and keeping DNS services up to date involves specific knowledge and capabilities around protocol and cache configuration, security patching, and other areas of expertise. Small and medium organisations often lack the resources and staff needed to manage DNS tasks effectively. As a result, they outsource DNS resolution to managed services from major

providers, trading operational simplicity for increased dependency on centralised infrastructure (10; 11). The implications of central operators having asymmetrical power is that they have the potential for scaling that distributed expertise in secondary services, such as operational complexity, global cybersecurity operations and the cost that comes with providing continuous systems. With decentralised systems, existing complexity exchanges cost, skill, and control across many areas of decision-making, like consensus protocols, node reliability, and onboarding.

Network Effects: As more users seem to prefer using popular public resolvers, these services become more attractive to content providers and network operators, creating a self driving cycle of centralisation. Large providers can negotiate direct peering agreements, deploy infrastructure in multiple geographic regions, and integrate with Content Delivery Networks (CDNs) to further reduce latency (12). These capabilities have resulted in an expansion of providers' competitive advantages that are extremely difficult for smaller or decentralised alternatives to overcome. The concentration of users also brings with it stronger motivation for app developers and enterprise users to optimise their services for large resolvers, increasingly giving more power to a few players. In decentralised systems, the standardisation and network effects are much weaker because of fragmentation, uneven performance, and a lack of strong coordination incentives.

Security Vulnerabilities of DNS

Due to the fact that recursive resolvers are concentrated in the hands of a few major providers, this centralisation introduces several critical security vulnerabilities, including a single point of failure, exposure of user queries to surveillance, and susceptibility to resolver manipulation (13; 10).

In June 2021, a software bug from a system misconfiguration caused a global outage on Fastly's edge cloud services and affected many major organisations, including Amazon, Reddit, The New York Times, Twitch, and the UK government's GOV.UK (14). Another major incident occurred in June 2022, when Cloudflare experienced an outage because of a change in the configuration of its infrastructure network (15). The incident affected around 19 of its data centres, which operated key hubs carrying considerable traffic. Since Cloudflare operates as both a CDN and DNS provider, the availability consequences affected millions of websites and applications, including Discord, Shopify, and DoorDash, and demonstrated how traffic concentration through just a few large providers maximises the widespread impacts of individual failures.

Privacy and Surveillance Risks: Surveillance is another systemic risk. Recursive resolvers observe every domain queried by their clients, effectively exposing a complete record of users' online activity. Centralisation increases this risk because a

few entities can passively collect, or be compelled to disclose, vast amounts of query data. Prime examples such as the NSA's PRISM initiative (16) revealed how DNS traffic can be intercepted to build user profiles and monitor communications without consent.

The implications for privacy in the DNS extend beyond government tracking. Commercial DNS providers log query data for behavioural analytics, threat intelligence, and market research. DNS queries provide a rich picture of browsing patterns, interests, and approximate demographic location. Centralised providers are able to group data from millions of users and build high-resolution behavioural profiles, which can be valuable for targeted advertising, threat intelligence, or, in some cases, they may even resell the data. Unlike web traffic, DNS queries are unencrypted and originate prior to establishing HyperText Transfer Protocol Secure (HTTPS), making them easy to harvest.

Even when queries are encrypted through mechanisms such as DNS over HTTPS (DoH) (17) or DNS over TLS (DoT) (18), resolver centralisation concentrates sensitive metadata at a limited number of geographical points where governments, ISPs, or corporations can apply surveillance policies. While encryption protects the query content, the timing, frequency, and destination of queries remain observable, allowing sophisticated traffic analysis techniques to model user behaviour (19).

The manipulation of DNS resolvers presents another attack vector. Malicious or compromised DNS resolvers can easily inject false responses into DNS queries, redirect traffic, or censor content, affecting both the confidentiality and integrity of communications (13). Attackers can also compromise the resolver infrastructure to redirect user traffic to malicious websites, initiating phishing attacks, stealing account credentials, or distributing malware on a large scale (10). These attacks exploit the central position of resolvers, since a compromise can affect thousands, if not millions, of users simultaneously. In state-sponsored campaigns such as DNSpionage, the attackers hijacked DNS servers in the Middle East so that emails and Virtual Private Network (VPN) traffic could be redirected and, in some cases, valid Transport Layer Security (TLS) certificates could be obtained, making it easier to perform man-in-the-middle (MITM) interception (20; 21).

Although the Domain Name System Security Extensions (DNSSEC) provide cryptographic protection against cache poisoning, its global adoption is still limited. The main reasons are deployment complexity, performance overhead, and compatibility issues with older systems (22). Where DNSSEC validation is not used, attackers can take advantage of predictable transaction IDs, port numbers, or other protocol weaknesses to insert forged responses into resolver caches.

Censorship and Information Control

Beyond security, centralised DNS resolvers also give governments a direct way to engage in censorship. They make it simple for authorities to block access to websites, a practice many regimes rely on. China is a prime example, using DNS tampering, filtering, cache poisoning, and deep packet inspection to block thousands of sites, including Google, Facebook, and Twitter (23). The well-known system called the Great Firewall of China in the West does more than block or redirect DNS queries. It also monitors traffic to catch attempts to bypass restrictions, such as switching resolvers, using alternative DNS providers, or connecting through VPNs.

Similar cases have occurred in other countries too: in 2014, Turkey banned Twitter following the spread of government corruption images (24), the UK blocked The Pirate Bay to enforce copyright restrictions (25), and Russia banned Telegram in 2018 after the company refused to provide encryption keys (26). These examples make it clear that DNS centralisation enables effective, wide-reaching censorship, allowing governments or powerful entities to control information access across entire populations.

Global Censorship Patterns: Depending on the political, legal, and social context, countries use different types of DNS censorship. Authoritarian regimes implement comprehensive filtering based on categories, which could include everything from political objections to social media sites or independent news (27). Democratic nations typically apply targeted blocking of certain categories of illegal content, which may include domains associated with copyright violations, child-exploitation material, or terrorist sites operating across borders (28). Corporate censorship involves contractually or legally constrained private companies to filter access to content based on terms of service or other legal requirements (29).

Modern censorship systems involve increasingly sophisticated techniques that go beyond simple DNS manipulation, including behavioural analysis, to monitor query patterns and identify fraudulent attempts, dynamic blocking, that automatically detects and blocks popular fraud tools, and collateral censorship, that blocks entire services to prevent access to specific content.

Impact on Digital Rights: DNS censorship has serious consequences for fundamental rights and freedoms. By having unrestricted access to the system that helps users find online resources, without user consent, authorities or centralised providers gain excessive control over what information people can access. This directly affects freedom of expression by restricting access to websites and limiting individuals' ability to obtain information, especially for journalists, activists, and civil society organisations (30). Academic freedom is also violated when researchers, educators, and students face barriers to scholarly resources, scientific databases, or online

collaboration platforms. Censorship further affects economic development by limiting access to essential digital services, global e-commerce platforms, and tools that support entrepreneurship and innovation. It also extends the digital divide, as people with less technical knowledge or fewer resources are disproportionately affected, while more skilled users often find ways to bypass restrictions.

Research Gaps

The issues outlined above show that important research gaps still exist, despite decades of DNS development and extensive academic study. The core tension between centralisation and decentralisation in DNS architecture continues to raise unresolved challenges. Although the causes and impacts of DNS centralisation are well-documented, major gaps remain in creating practical alternatives that can deliver the same performance and reliability as today's centralised systems. By analysing these gaps, this research leads to the following research questions.

How can recursive resolution scale in a way that does not introduce new bottlenecks? All current decentralised protocols suggest practical trade-offs between distribution and performance, with respect to the efficiency of routing queries. Locating authoritative data in a distributed network adds significant communication costs. Load balancing becomes complicated, and query loads cannot be fairly distributed across nodes with different computational capabilities, network connections, and geographic locations. Cache compatibility is another issue, as keeping consistency across a distributed cache with minimal synchronisation overhead requires accurate coordination between the involved mechanisms, which may compromise performance benefits.

With queries travelling through third-party infrastructure, how can user privacy be ensured? Existing solutions do not achieve sufficient privacy due to fundamental limitations. Metadata exposure is challenging because encrypted queries reveal timing, frequency, and routing, which allows powerful traffic analysis. Trust distribution can also increase privacy risks when users switch from trusted centralised resolvers to groups of distributed nodes that may not be trusted. Privacy protocols may lead to performance trade-offs, as they often introduce significant latency and bandwidth usage, which can negatively impact user experience and reduce adoption.

How can we overcome manipulation and censorship in a system where a small number of operators maintain control? This question involves many technical and social aspects which can come together in complex ways. Detection resistance requires avoiding identification by increasingly sophisticated censorship systems that use machine learning algorithms and behavioural analysis to identify fraudulent attempts. The system's durability must be maintained against adaptive adversaries

who can update filtering rules in real-time and deploy new detection mechanisms as soon as cheating tools become popular.

What security approaches can decentralised DNS systems use when anyone is able to join the system and potentially act adversarially against it? Traditional security approaches are based on trusted operators, but decentralised DNS systems face some fundamental security issues while allowing anyone to join and act adversarially. First, to prevent Sybil attacks, it must be ensured that the adversary cannot create multiple fake identities to affect the behaviour in the system. In order to achieve that, the system has to rely on robust identity checking processes or a consensus protocol. Second, it is critical to determine how Byzantine behaviour is handled, which is necessary for dealing with nodes that are operating in a different way than the protocol defines, whether it is from adversarial goals, bugs in the software in use, or errors with the actual protocol implementation.

Contributions

We present the main contributions of this research, which address the limitations of existing decentralised DNS systems and open consensus protocols. First, the work proposes a decentralised DNS recursive resolver that uses the Chord Distributed Hash Table (DHT). This design focuses on the recursive resolver and offers fault-tolerant, censorship-resistant query resolution with full compatibility with the legacy DNS infrastructure. Second, we define a lightweight, efficient Proof-of-Space (PoSpace) protocol. This protocol is integrated into the network join process, providing strong Sybil resistance through a verifiable data storage participation scheme, as well as mitigation against rationality and amortisation attacks and resource overhead, compared with existing PoSpace protocols.

Together, these contributions enable a secure, scalable, and deployable system that combines decentralised DNS resolution with network membership validation. Below, we provide a detailed description of each contribution, including mechanisms and the ways they enable practical deployment in real-world distributed networks.

Decentralised DNS Resolution Based on Distributed Hash Tables

This research presents a decentralised DNS recursive resolver built on the Chord DHT architecture. Unlike earlier attempts that aimed to replace the entire DNS hierarchy, this work focuses on the recursive resolver layer, which remains the main point of centralisation and vulnerability in today's DNS infrastructure.

In the proposed design, recursive resolution is replicated across a network of cooperating nodes. Each node can independently resolve queries, cache responses for future requests, and forward requests to other nodes. The nodes work together through the Chord DHT to manage distributed data and protocols, allowing them to improve latency and cache performance in the network. By reducing reliance on centralised infrastructure, the system eliminates single points of failure while remaining fully compatible with the existing DNS standards.

The architecture provides fault-tolerant routing paths, distributed caching of responses, and cryptographic proof of replies. It also fully supports traditional DNS query–response formats and is usable alongside legacy recursive resolvers, ensuring integration without disrupting current services.

Practical Proof-of-Space for Secure Network Membership

The second contribution addresses the problem of securing open, decentralised networks through a novel, optimised PoSpace protocol that is part of the network membership process. This PoSpace requires nodes to commit verifiable physical storage resources to join the system, which creates economic costs to take over the network through a Sybil attack, while remaining lightweight and usable for legitimate users.

In contrast to traditional PoSpace schemes that introduce a high initialisation overhead and ongoing costs for verification and identity maintenance, the proposed PoSpace scheme provides an efficient solution for distributed systems like DNS resolvers. The protocol uses cryptographic challenges to periodically check the committed storage space to verify that resources are still being dedicated to network operation, while minimising the ongoing computational overhead.

The proposed model creates strong Sybil resistance by making multiple fake identities economically infeasible, while enabling legitimate nodes to participate in distributed caching, query routing, and network maintenance. Integration with the Chord DHT network ensures that the process of validating the membership runs smoothly during the join process, keeping the network secure without centralised control.

Integrated System Architecture

The two contributions mentioned above combine Chord DHT-based recursive resolution with PoSpace-secured membership, creating an alternative to traditional DNS that is operationally efficient. The system tackles scalability, security, and trust issues present in the current infrastructure but is lightweight enough to deploy in real-world cases.

The system guarantees fault-tolerant, censorship-resistant query handling, while remaining fully compatible with the legacy DNS infrastructure. This is accomplished by limiting the design to the recursive resolution layer rather than attempting to replace the entire DNS hierarchy, an approach taken by most existing solutions. Because of this focus, the system can be deployed incrementally alongside existing infrastructure (recursive resolvers, and legacy DNS), reducing barriers to adoption and enabling a gradual transition towards resilient DNS resolution.

Thesis Structure

The rest of the thesis is organised as follows. The Background chapter provides the necessary technical context by establishing the foundation of DNS, peer-to-peer (P2P) architectures, and decentralised consensus mechanisms relevant to this work. The Related Work chapter reviews existing research and proposals. It highlights previous efforts towards decentralised DNS systems and existing PoSpace solutions. It also highlights the limitations of the research for specific problems currently being targeted in this work. The Threat Model chapter establishes the assumptions from a security perspective and the assumptions about the adversarial capabilities in this work, addressing the decentralised resolver and the PoSpace protocol, respectively. The Methodology chapter consists of two parts that comprehensively describe the technical details of the proposed solution. The first part covers the decentralised resolver implemented on Chord DHT, while the second examines the PoSpace protocol for securing network membership. Both parts contain design details and evaluations with respect to specific experiments. The research wraps up with the Conclusions and Future Work chapters, where the main findings and contributions are summarised, while the future directions are highlighted.

Chapter 2

Background

The Background chapter introduces the fundamental technologies of this research. It begins with the backbone of the DNS protocol, which is the primary focus of this research, explaining its components, workflow (resolution process), and layered structure. The focus is on how DNS works, including the roles of recursive resolvers, Authoritative Name Servers (ANS), and caching mechanisms. The chapter then gives an overview of P2P architectures, specifically highlighting the Chord protocol. Key concepts such as DHTs, finger tables, and the processes for node joining, departure, and lookup are explained to establish the basis for the proposed decentralised resolver design. Finally, the chapter introduces decentralised consensus mechanisms. It outlines the principles of consensus in distributed systems and examines PoSpace as a consensus mechanism. These concepts provide the necessary background for understanding how secure participation is achieved in the proposed system.

2.1 The Domain Name System (DNS)

DNS was introduced by Paul Mockapetris in 1983 and referenced in RFC 882 and RFC 883 [1] (4; 5). It was designed to replace the problematic host table system with a scalable, distributed, and hierarchical naming architecture. DNS interprets human-readable domain names and converts them into numerical IP addresses, making the user experience less complex, while at the same time allowing reliable communications over the internet.

DNS is designed with a hierarchical approach (as explained in Section 2.1.1) and most of its core consists of centralised components. In this model, ANSs manage specific zones and assign subdomains. Records such as A (address), AAAA (IPv6 address), MX (mail exchanger), NS (name server), and CNAME (alias) are stored in zone files on

[1] The RFC 882 and RFC 883 updated and replaced later on by RFC 1034 and RFC 1035

ANSs and propagated through the system. Recursive resolvers act as intermediary components. Their job is to query multiple servers to resolve a domain and cache results locally, improving performance and reducing latency.

DNS is an application-layer protocol (31) that allows computers, routers, and name servers exchange query/response messages about domain names. It supports tasks like iterative and recursive lookups, zone transfers, and caching, while the initial idea of its architecture was to scale, remain reliable, and store data (DNS records) consistently worldwide. However, in terms of security aspects, it did not include strong mechanisms or privacy protection at first. This led to extensions such as DNSSEC (RFC 4033–4035 (32)) and DoH (17), which were added to fill those gaps.

2.1.1 DNS Hierarchical Structure

Figure 2.1 presents the hierarchical and distributed DNS design. The top-level is the root represented by a dot (”.”), which is maintained by 13 clusters of root servers (lettered A–M) and distributed globally in order to provide redundancy and resilience (33). Below the root are top-level domains (TLDs), which include generic TLDs (gTLDs) such as .com, .org, and .net, and country-code TLDs (ccTLDs) such as .us, .uk, and .de. TLDs are managed by registries in compliance with policies coordinated by the Internet Corporation for Assigned Names and Numbers (ICANN), as described in Section 2.1.2.

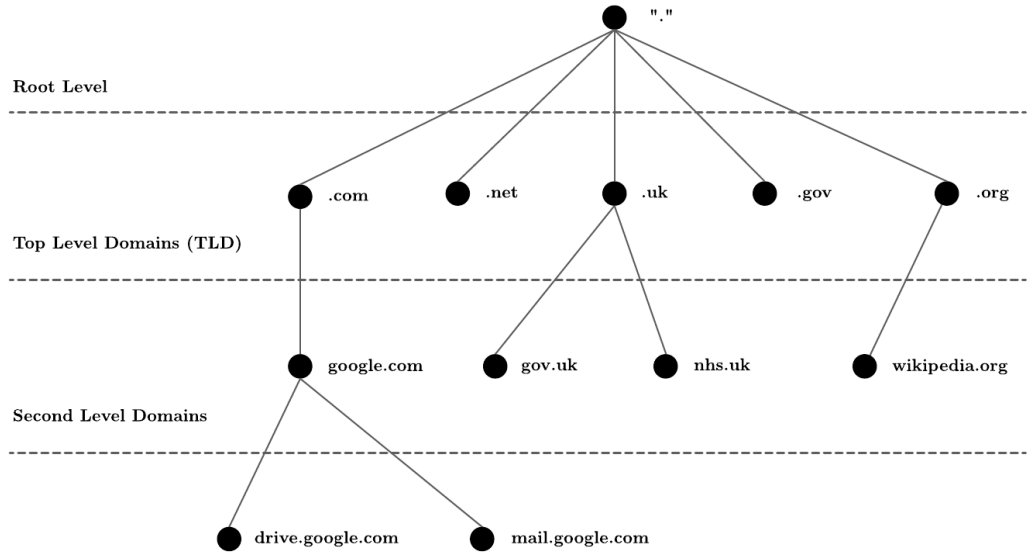


FIGURE 2.1: The hierarchical structure of DNS.

Second-level domains (SLDs) are placed directly under the TLDs, usually representing either organisations, institutions, or services. Subdomains add further detail,

representing a particular host, service, or department. DNS records are stored in zone files on name servers, which contain the information needed about the domains. Zone files also contain the most commonly used record types, such as A records, which associate domains with their IPv4 addresses; AAAA records, which define a domain's IPv6 address; MX records for mail servers; CNAME records for domain aliases; NS records, which refer to the ANSs; and TXT records, used for text storage and commonly for domain or policy verification of the domain or policy.

ANSs manage zone files and provide responses to queries initiated by recursive resolvers. Recursive resolvers are often configured and operated by an ISP or public service, such as Google Public DNS or Cloudflare (8), and initiate recursive queries by contacting the root, TLD, and ANSs. To improve performance and reduce network traffic, recursive resolvers cache responses locally for the duration configured in the time-to-live (TTL) value of each record. Changes are made through zone transfers. That can be done using an asynchronous full zone transfer (AXFR) or Incremental Zone Transfer (IXFR) (34) between primary and secondary servers to allow for consistent distribution of zone information across multiple servers. DNS security extensions are also an increasingly important component; for example, DNSSEC provides validation of responses to mitigate spoofing of responses to queries or cache poisoning of DNS records (discussed in more detail in Section 2.1.5).

2.1.2 DNS Governance

DNS governance defines the policies, protocols, and components responsibilities that ensure the system's security, stability, and global interoperability. ICANN serves as the main coordinating body for managing domain names, IP address space, and protocol parameters worldwide. It follows a multi-stakeholder model, which combines the input and involvement of technical experts, governments, industry members, civil society, and the wider internet community to support transparent and balanced decision-making (35).

Within ICANN, supporting organisations develop policies that apply to gTLDs and ccTLDs (such as the Generic Names Supporting Organisation (GNSO) and the Country Code Names Supporting Organisation (ccNSO)), while advisory committees provide recommendations with regard to regulatory, technical, or security matters (such as the Governmental Advisory Committee (GAC) and the Security and Stability Advisory Committee (SSAC)). Finally, the Internet Assigned Numbers Authority (IANA), which operates as a department under the oversight of ICANN, manages the root zone files, allocates TLDs, coordinates blocks of IP addresses, and maintains a record of protocol parameters, in the form of IANA-registered RFCs (36).

In practice, DNS governance verifies that the DNS participants (TLD registries, registrars, and ANSs) are following the correct processes for domain delegation, management of zones, and resolution of disputes. Security and resilience are developed through coordinated mechanisms such as DNSSEC, root zone key signing ceremonies, and monitoring of root server performance. The governance framework is continuously evolving to address development needs, new threats, and policy requirements, while maintaining the integrity, stability, and global availability of the DNS, and providing balance among technical, commercial, and public-interest factors. Table 2.1 summarises the DNS governance structure and its responsibilities.

Layer	Responsibilities/Entities
ICANN	Primary coordinating body. Oversees global allocation of domain names, IP addresses, and protocol parameters through a multi stakeholder model.
Supporting Organisations	GNSO and ccNSO develop policies for gTLDs and ccTLDs.
Advisory Committees	GAC and SSAC provide recommendations on regulatory, security, and stability issues.
IANA	Manages the root zone files, allocates TLDs, coordinates IP address blocks, and maintains protocol parameter registries.
Root Servers	13 logical root servers, operated by multiple organisations, ensure distribution and availability of the root zone.
TLD Registries and Registrars	Responsible for TLD operation, domain registration, delegation, and dispute resolution.
Authoritative Name Servers	Host DNS zone files and provide definitive answers to queries.
Registrants (Domain Holders)	End users, enterprises, or organisations that register and manage domain names.

TABLE 2.1: DNS governance hierarchy and responsibilities.

2.1.3 The DNS Protocol

DNS operates on a client-server model and uses a hierarchical and distributed approach to translate domain names into IP addresses. Figure 2.2 presents this recursive query process, showing each stage from the stub resolver to the ANS.

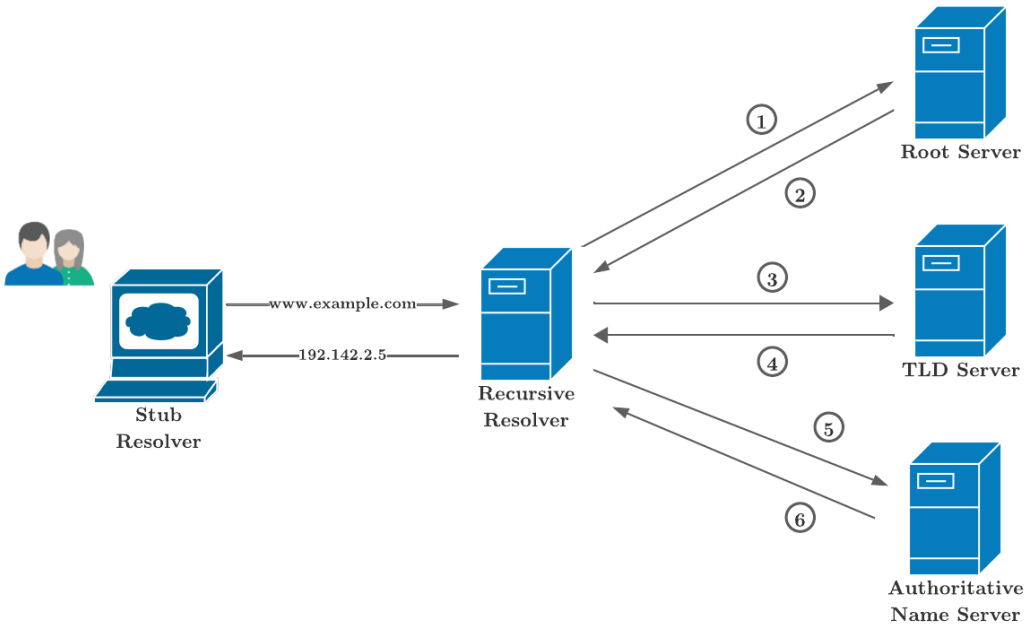


FIGURE 2.2: DNS query lifecycle from stub resolver to ANS.

A stub resolver, integrated into the user's device, will initiate a DNS query every time an application requests a domain name. The query is sent over User Datagram Protocol (UDP) (typically port 53) to a local recursive resolver, which is usually provided by an ISP. If the resolver already has a cached entry corresponding to the requested domain, it returns the mapped IP address back to the stub resolver in order to reduce latency and traffic across the network.

If the requested domain is not in the cache or the entry has expired, the recursive resolver will perform iterative queries; it will first contact a Root DNS Server, which will return a referral to the appropriate TLD server (such as ".com", ".org", etc.), and then the resolver will query the TLD server, which will respond with the details of an ANS for the domain. The resolver will then query the ANS for the requested DNS record. During this process, responses include a TTL value that specifies how long the resolver can cache the information.

Resolvers address potential network issues by using retry, timeout, and failover mechanisms. For large responses, or DNSSEC-signed data, resolvers use TCP instead of UDP. By using UDP they address reliability and integrity of data when retrieving responses from the ANS. Once the response is obtained, the recursive resolver caches it according to the TTL and returns the result to the stub resolver, completing the resolution process and enabling the client application to connect to the destination server.

2.1.4 Domain Name Registration

Domain name registration is the way individuals or organisations receive usage rights for a particular domain name in the DNS, as shown in Figure 2.3. Registries, overseen by the IANA and administered by ICANN, are responsible for TLDs like .com and .net (37; 38). Their primary function involves the maintenance of records about domain ownership, assigning individual domains to registrants, whether individuals or organisations (39). This hierarchical structure describes the division of responsibilities in the DNS to ensure orderly governance of internet resources.

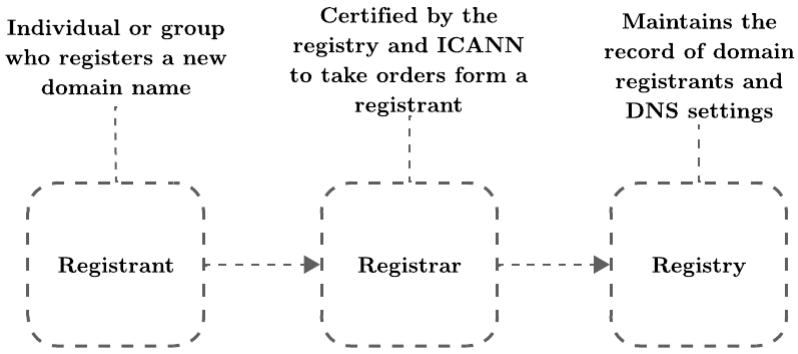


FIGURE 2.3: The domain name registration proccess and components.

Registrars act as intermediaries between people who register domains (registrants) and the registries, handling the transactions needed for domain registrations. For instance, when a registrar registers a domain under a TLD for an end user, it is required to inform the corresponding registry (such as VeriSign for .com domains). Following this, the registrar is charged a fee payable to the registry, which is typically included in the price issued to the end user.

The registration process allows the end user to choose the desired registration period, which usually ranges from 1 to 10 years, agreeing to terms and conditions, and providing accurate contact information for administrative, technical, and billing purposes (40). Domain registration is subject to periodic renewal to maintain ownership rights. The hierarchical structure of DNS, including the distinction between registrars and registries, ensures a distributed and organised approach to managing the vast array of domain names on the internet. This system enables users worldwide to access websites and services efficiently through the DNS.

2.1.5 DNS Security Extensions

The original design of the DNS did not include mechanisms for authentication, integrity, or confidentiality. As a result, DNS queries and responses are vulnerable to manipulation and interception. To address these weaknesses, several security extensions were developed and standardised.

DNSSEC: The DNSSEC protocol (41) adds cryptographic signatures to DNS records to support resolvers' efforts to validate the authenticity and integrity of responses. ANSs sign their zone data with private keys, and resolvers then validate those responses with associated public keys held in a chain of trust from the root. The objective of DNSSEC is to mitigate cache poisoning and data modification attacks by allowing resolvers to detect tampering with records. Unfortunately, the operational complexities of implementations, the increased size of responses (leading to possible fragmentation), and the lack of incentives for registries and registrars to deploy DNSSEC have contributed to incomplete adoption of the protocol.

DANE: DNS-Based Authentication of Named Entities (DANE) (42) builds on DNSSEC to bind TLS certificates to DNS names. This provides an alternative trust model to certificate authorities (CAs) and enables domain owners to specify which keys or CAs are authorised for their services. While promising, DANE adoption has been slow and is strongly dependent on widespread DNSSEC deployment.

Encrypted Transport: More recently, security protocols such as DoT and DoH (17) were introduced as a way of protecting the confidentiality of DNS traffic by encrypting queries and responses between clients and resolvers. These extensions effectively prevent passive eavesdropping by adding an extra level of difficulty for third parties trying to spy on or tamper with DNS lookups. However, their main limitation is that they do not resolve the matter of centralisation, since encrypted traffic is still directed towards a few specific public resolvers, raising new concerns about privacy and surveillance. Modern proposals, such as Oblivious DoH (ODoH) (43), aim to address this by separating the query origin from the resolution path, which reduces metadata exposure.

Overall, DNS security extensions have been a major improvement in mitigating long-standing vulnerabilities. However, their technical, economic, and governance challenges have delayed their deployment, and none of the existing mechanisms fully address the risks of resolver centralisation and censorship. These limitations motivate the exploration of decentralised approaches to DNS security, as examined in this research.

2.2 Peer-to-Peer (P2P) Architecture

2.2.1 Overview of P2P Networks

P2P systems are distributed architectures in which nodes, referred to as *peers*, communicate and cooperate directly without centralised coordination. Each peer acts both as a client and a server, contributing resources such as bandwidth, storage, or computational power to the network (44). Formally, a P2P network can be represented as a graph $G = (V, E)$, where V is the set of nodes, $E \subseteq V \times V$ represents direct communication links between peers, and $n = |V|$ is the total number of peers.

P2P systems are designed to achieve scalability, resilience, and fault tolerance through decentralisation. The network's resilience to node failures can be quantified by network connectivity $\kappa(G)$, which represents the minimum number of nodes whose removal disconnects the network. Fault tolerance improves as $\kappa(G)$ increases, reducing single points of failure and bottlenecks associated with centralised control.

P2P architectures are broadly categorised into *structured* and *unstructured* networks. In unstructured networks, nodes connect arbitrarily without a global organisation. Examples include Gnutella (45) and early BitTorrent systems (46). Each peer maintains k random neighbours, and query search is typically flooding-based, where a query propagates to all neighbours within h hops. The expected number of nodes reached is approximately:

$$N_h = \sum_{i=0}^{h} k^i = \frac{k^{h+1} - 1}{k - 1}.$$

While flexible and resilient to fluctuations, unstructured networks suffer from poor scalability: communication overhead grows exponentially with hop count h, and query success is probabilistic.

Structured P2P networks organise peers according to a deterministic topology to support efficient and predictable data lookup (explained in detail in Section 2.2.2). DHTs are a common structured approach. Each node $v \in V$ is assigned an identifier ID_v from a large identifier space $[0, 2^m - 1]$, typically generated via a cryptographic hash function $h : \text{key} \to [0, 2^m - 1]$. Each data item k is mapped to its responsible node according to:

$$v = \text{succ}(h(k)),$$

where $\text{succ}(x)$ returns the node whose identifier immediately follows x in the identifier space. Lookup operations in structured networks achieve logarithmic

complexity $O(\log n)$ in the number of nodes n. For example, in Chord, each node maintains a *finger table* of size $m = \log_2 n$, enabling query routing along $\lceil \log_2 n \rceil$ hops.

Structured P2P networks thus provide deterministic guarantees for data location and retrieval, whereas unstructured networks prioritise flexibility and redundancy over efficiency. The choice of network architecture directly impacts the performance, scalability, and resilience of distributed applications, such as decentralised DNS resolvers.

2.2.2 Structured P2P Networks

Structured P2P networks organise nodes and data deterministically to enable efficient data lookup (47). At the core of these networks are DHTs, which map keys to nodes using a consistent, distributed scheme. A DHT can be formally described as a tuple (N, K, F), where $N = n_1, n_2, \ldots, n_m$ is the set of nodes in the network, $K = k_1, k_2, \ldots, k_l$ is the set of keys (data identifiers), and $F : K \to N$ is the mapping function that assigns each key to its responsible node.

Each node $n_i \in N$ stores a subset of key-value pairs (k, v) such that:

$$n_i \supseteq \{(k, v) \mid F(k) = n_i\}$$

This ensures that any lookup for key k can be routed to its responsible node efficiently.

To assign keys to nodes in a scalable and fault-tolerant way, structured networks use consistent hashing. Let the hash function be $h : K \cup N \to [0, 2^m - 1]$, mapping both keys and nodes onto an m-bit identifier space arranged in a ring. Each key k is assigned to the first node n_i whose identifier satisfies the following condition:

$$h(n_i) \geq h(k) \pmod{2^m}$$

This minimises remapping of keys when nodes join or leave.

Structured P2P networks achieve efficient routing with $O(\log N)$ hops, where N is the total number of nodes. Let the routing table of node n_i contain r_i entries. Then, the maximum lookup path length L satisfies the following:

$$L \leq \lceil \log_2 N \rceil$$

To handle node failures, DHTs replicate keys across multiple nodes. Let r be the replication factor. Then, each key k is stored on r consecutive nodes in the identifier space:

$$\text{replicas}(k) = \{n_j \mid j = F(k), F(k)+1, \ldots, F(k)+r-1 \mod N\}$$

This ensures data availability even under node fluctuations.

2.2.3 Chord Distributed Hash Table (DHT)

Chord is a canonical structured P2P protocol that implements a DHT over a ring topology. It provides scalable, fault-tolerant key-value storage with deterministic lookup in $O(\log N)$ hops (48). Chord assigns each node n_i and key k an m-bit identifier using a hash function, as follows:

$$\text{ID}(x) = h(x) \in [0, 2^m - 1], \quad x \in N \cup K$$

The identifier space forms a modulo 2^m ring. Each key k is assigned to its successor node, defined as the first node whose ID equals or follows the key in the identifier space, as follows:

$$\text{successor}(k) = \min_{n_i \in N}\{\text{ID}(n_i) \geq \text{ID}(k)\} \mod 2^m$$

This mapping ensures that every key is uniquely and deterministically assigned.

Nodes are logically arranged in a circular identifier ring. Each node maintains information about a small subset of other nodes in the network to ensure efficient routing. This includes the *successor*, which is the next node on the ring, responsible for some portion of keys, and the *predecessor*, the previous node on the ring.

This ring structure enables consistent key assignment and smooth handling of node joins and departures.

To locate a key k, a node n_i forwards the request to the closest preceding node in its routing table whose ID does not exceed k. Let n_i have a routing table $FT_i = f_1, f_2, \ldots, f_m$, called the *finger table*, defined as follows:

$$f_j = \text{successor}(n_i + 2^{j-1} \mod 2^m), \quad j = 1, \ldots, m$$

The finger table allows exponential distance coverage along the ring. The lookup algorithm proceeds recursively or iteratively, halving the remaining distance to the target key at each hop. As a result, the maximum number of hops to locate any key satisfies the following:

$$\text{Hops}(k) \leq \log_2 N$$

DHT Tables: Finger Tables in Chord

In Chord, each node maintains a finger table to optimise key lookups and improve scalability. The finger table provides shortcut links around the ring, reducing the number of hops required for a lookup from $O(N)$ to $O(\log N)$.

For a node n_i with identifier $\text{ID}(n_i)$, the finger table FT_i consists of m entries (where m is the number of bits in the identifier space). Each entry f_j points to the successor of the following:

$$f_j = \text{successor}\big(\text{ID}(n_i) + 2^{j-1} \mod 2^m\big), \quad j = 1, \ldots, m$$

Entry j	**Start** $(n_i + 2^{j-1}) \bmod 2^m$	**Points to (Successor)**
1	$n_i + 1$	Node responsible for that ID
2	$n_i + 2$	…
⋮	⋮	⋮
m	$n_i + 2^{m-1}$	…

TABLE 2.2: Finger table structure for node n_i in Chord DHT.

Each finger entry effectively "jumps" around the ring, allowing large steps toward the target key.

The finger table reduces lookup hops by providing shortcuts: rather than traversing the ring node by node, a node forwards a lookup request to the closest preceding node in its finger table to the target key k.

Let n_i be the current node and k the key, defined as follows:

$$n_{\text{next}} = \max\{f_j \in FT_i \mid \text{ID}(f_j) < \text{ID}(k)\}$$

This strategy halves the remaining distance to the key at each step, giving $O(\log N)$ routing efficiency.

When a new node n_{new} joins:

1. It initialises its finger table using information from an existing node.
2. Updates the predecessor and successor links of affected nodes.
3. Transfers responsibility for keys in its range from its successor.

Formally, for keys k where:

$$\text{ID}(n_{\text{new}}) \leq \text{ID}(k) < \text{ID}(\text{successor}(n_{\text{new}}))$$

the key-value pairs are moved to n_{new}.

When a node leaves or becomes unreachable, its predecessor and successor update their links and redistribute keys so that the same coverage is maintained. Finger tables are also updated periodically to reflect the topology of the network to allow for efficient lookups. This structure ensures logarithmic routing with very minimal state per node and allows for scaling and fault tolerance in very large-scale networks.

2.2.4 Transition to Contribution

In a structured P2P network (represented by Chord), the topology is a fault-tolerant ring, which means key placement can be guaranteed, and lookups will happen in logarithmic time due to the finger tables. The fact that DHTs allow many nodes to resolve only key-value pairs at low routing overhead and maintain scalability gives structured P2P networks an advantage. These features make structured P2P a suitable candidate for a distributed DNS system. In a Chord network, a recursive DNS resolver could use a deterministic process to map a domain name to a node, resolving queries quickly and handling node joins and leaves.

In this work, we introduce Chord as the backbone for our decentralised DNS resolver, which applies DHT principles, consistent hashing, finger tables, and successor/predecessor maintenance as a replacement for centralised DNS infrastructure. This background establishes the theoretical framework and motivates the design choices made in the following chapters.

2.3 Decentralised Consensus

2.3.1 Consensus Overview

Decentralised systems require mechanisms to come to an agreement about shared data across untrusted participants. Consensus provides consistency and correctness in a distributed context. Traditional consensus protocols such as Paxos and Raft (49; 50) assume a fixed set of participants and, in general, are designed for permissioned settings. Although they offer strong consistency guarantees, they do not support the open and dynamic environments of public networks.

Blockchains have developed a new class of consensus protocols that operate in permissionless environments without central authorities. Nodes may join and leave the network at any time, and the system must cope with nodes that may behave dishonestly. Consensus mechanisms in decentralised systems must ensure agreement among honest nodes on the system state, validate correct and legitimate data only, guarantee progress despite network failures or delays, and prevent Sybil attacks, in which an adversary might create multiple identities to influence the system unfairly.

Different blockchain consensus mechanisms achieve Sybil resistance through distinct approaches. Prime examples include Proof-of-Work (PoW) (51), which relies on computational resources; Proof-of-Stake (PoS) (52), which associates influence with token holdings; and Proof-of-Space (PoSpace), which requires the allocation of physical storage capacity.

2.3.2 Merkle Trees

A Merkle Tree is a cryptographic data structure for efficient verification of data integrity. The Merkle Tree is a binary tree with leaf nodes defined asthe cryptographic hash of a data block. Non-leaf nodes are defined as the cryptographic hash of the concatenation of their child nodes (the concatenation of their children). The root node (Merkle root) provides a single value that can verify the integrity of all the data blocks in the tree (53). Figure 2.4 shows a simple Merkle Tree with four leaves. Each non-leaf node represents the hash of its children, resulting in the Merkle root at the top.

We define $D_1, D_2, \ldots, D_n$ as the data blocks to store in the Merkle tree. Each leaf node of the tree is computed as the hash of a data block:

$$L_i = H(D_i) \quad \text{for} \quad i = 1, 2, \ldots, n$$

where H is a cryptographic hash function (e.g., SHA-256).

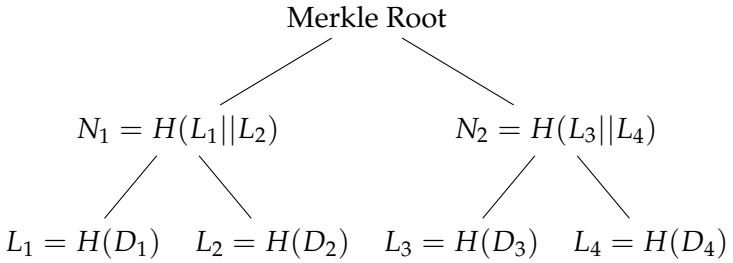


FIGURE 2.4: Example of a simple Merkle tree with four leaves.

Each non-leaf node in the Merkle tree is computed as the hash of the concatenation of its two child nodes:

$$N_j = H(N_{2j-1}||N_{2j})$$

where $||$ indicates concatenation, and N_{2j-1} and N_{2j} are the left and right child nodes, respectively. Here, j represents the index of a non-leaf node in the tree. Its values depend on the tree level: at each level above the leaves, j ranges from 1 to the number of nodes at that level.

The Merkle root is the top node of the tree. It is computed iteratively from the bottom of the tree to the top. To verify that a data block D_i is part of a Merkle tree, a Merkle proof is used, often referred to as *Merkle path*, which is a sequence of hashes from the corresponding leaf node L_i to the Merkle root. First, D_i is hashed to generate L_i and then iteratively combined with the sibling hashes along the Merkle path. If the resulting hash matches the known Merkle root, D_i's existence is confirmed without revealing the entire tree.

2.3.3 Proof-of-Space Consensus

PoSpace is a mechanism where participants show that they are committed to a certain amount of storage space. The security assumption is that a fixed amount of disk space is a resource that has economic cost, which makes it unattractive for adversaries to obtain large quantities cheaply. In a PoSpace-based system, nodes generate and store specific datasets (known as plots), which result from solving computational challenges. When plots are generated, they allow participants to produce small proofs of space with no requirement to recompute the initial expensive computation. PoSpace systems have much lower energy usage than PoW systems, while resisting Sybil attacks at the same level.

In this work, PoSpace is not used to elect leaders or generate blocks but, instead is used as a Sybil resistance mechanism at network admission. Nodes are required to

prove access to a set amount of storage space to join the decentralised DNS resolver network. Admission requires nodes to prove access to a specified amount of storage to join the decentralised DNS resolver network. This proof replaces reliance on traditional central authorities or economic barriers such as staking or enforcing network membership through verifiable physical resource commitment.

The PoSpace protocol is executed between two parties: a Prover P and a Verifier V. The protocol runs in two phases: initialisation and execution. The execution involves an interactive process where P and V exchange messages based on shared input, which consists of an identifier id, and local inputs unique to each party (inP for the Prover and inV for the Verifier). The id ensures that the Prover cannot reuse the same space for different statements. The protocol guarantees that the Prover has allocated a certain amount of storage (space commitment) and can prove this without revealing the data itself.

Initialisation phase: The initialisation phase is an interactive protocol that takes shared inputs, including an identifier id, a storage bound N, and potentially other parameters, defined as $\text{prm} = (\text{id}, N, \ldots)$. It involves an exchange between the Prover and the Verifier. During this phase, the Verifier may output a special symbol $\perp$ to indicate that the protocol is aborted, typically if the Prover fails to commit the required storage. The initialisation phase can be represented as:

$$(P \leftrightarrow V)(\text{prm}) \xrightarrow{\Pi_{\text{init}}} \{\text{Continue}, \perp\}$$

where Π_{init} indicates the initialisation phase protocol. The output is either "Continue," allowing the protocol to proceed, or $\perp$, signalling an abort.

Execution phase: The execution phase is an interactive process in which the Prover and the Verifier engage in a series of exchanges. In this phase, the Prover produces no outputs and the Verifier will either accept or reject the proof based on the interaction. This process can also be expressed as:

$$(P \leftrightarrow V) \xrightarrow{\Pi_{\text{exec}}} \{\text{Accept}, \text{Reject}\}$$

where Π_{exec} indicates the execution phase protocol. The Verifier's decision is based on the Prover's ability to demonstrate possession of the required storage without revealing the actual data.

Completeness: The PoSpace protocol is complete if, when the Prover honestly follows the protocol and possesses the required storage, the Verifier will always accept the proof. In other words, if the Prover has correctly committed the storage space during the initialisation phase and executes the protocol as specified, the Verifier should

accept the proof with high probability. If P has stored space consistent with the parameters $\text{prm} = (\text{id}, N, \ldots)$, then after executing the protocol, V will output accept.

Soundness: The PoSpace protocol is sound if no dishonest Prover can convince the Verifier to accept a proof unless the Prover has actually committed the required storage space. Even if the Prover attempts to cheat, the Verifier will detect the inconsistency and reject the proof. Thus, soundness ensures that the protocol is resistant to manipulation or fraud by ensuring that only those who have truly committed the necessary storage can generate valid proofs. If P does not have the required space, then the probability that V accepts the proof is negligible.

2.3.4 Transition to Contribution

The proposed decentralised DNS system integrates a practical PoSpace protocol to establish network membership security and trust among peers. In this protocol, nodes are required to commit verifiable physical storage resources in order to join and stay active in the network. Nodes' storage resources are cryptographically challenged and verified at regular intervals, ensuring that storage resources are still allocated and available for network service.

By linking the cost of participation to real storage resources, PoSpace introduces a direct cost on potential adversaries, and therefore making Sybil attacks (where an attacker generates a large number of fake identities) significantly more difficult. This ensures that network membership is secure and accountable while enabling nodes to productively and reliably contribute to distributed caching, query routing, and redundancy. Integrating PoSpace with the decentralised DNS resolver increases the system's fault tolerance, censorship resistance, and overall robustness, aligning with the goal of creating a scalable and secure structured P2P network.

Chapter 3

Related Work

From time to time, several studies (academic and commercial) have explored decentralised DNS approaches, highlighting both their potential benefits and limitations. Among these, blockchain-based DNS (BDNS) has received the most attention, with Namecoin being the first notable attempt to decentralise DNS and offer a blockchain-based alternative to traditional systems. Following this early work, numerous proposals have aimed to improve the security, availability, and censorship resistance of DNS by making use of blockchain's decentralised and tamper-evident properties.

In this chapter, we focus on the analysis of the current landscape of BDNS solutions and the progress made in this field. Our selection criteria include relevance, publication quality, impact, and recognition in both the academic and commercial communities. We consider works that directly address BDNS and are recognised for their innovative approaches or significant contributions. We also prioritise solutions that remain active in research or have achieved widespread recognition within the community. Apart from BDNS, we review non-blockchain proposals that explore decentralised DNS through alternative architectures, such as P2P networks and distributed hash tables. We present these approaches in detail, highlighting their key characteristics and objectives, identifying limitations, and outlining areas that require further research, including commercial investigation.

Following this, we review related work on PoSpace, focusing on its evolution as a resource-based security primitive. This second part of the related work chapter examines how PoSpace has progressed from early proposals centred on verifiable storage to more recent work integrating it into consensus protocols and Sybil resistance mechanisms. This review matches the discussion on decentralised DNS by providing the necessary context for understanding how PoSpace can serve as a secure admission control mechanism for the decentralised DNS resolver proposed in this research. Although these two areas have been studied separately, this research

connects them by proposing a decentralised DNS resolver network that uses PoSpace not for consensus but as a secure admission mechanism. This combination addresses a gap in existing solutions, which often lack practical, scalable methods for securing open participation against Sybil attacks.

3.1 Blockchain-based DNS Solutions

The application of blockchain technology with the DNS has been suggested as a way of overcoming existing limitations of the traditional DNS architecture, particularly relating to integrity, censorship resistance and transparency. BDNS solutions seek to apply the decentralised, tamper-resistant, and verifiable properties of distributed ledgers to a resolution infrastructure that consists of hierarchical and centralised trust. In these kind of systems, authoritative name servers are either replaced entirely or substituted with records on the blockchain, allowing common functionalities (domain ownership, registration, and resolution) to be governed by consensus and not any central trust.

This area of research has proposed several different approaches based on a variety of blockchain architectures, consensus algorithms, and design assumptions. Some proposals use blockchain as an additional security layer to the DNS system, while others suggest a complete replacement of the traditional DNS infrastructure with a fully P2P name resolution process. Each of the approaches include additional potential issues around scalability, query latency, and compatibility with the existing DNS standards (54). In the following section, we present a structured analysis of the existing BDNS solutions identified in the literature. To provide a comparative perspective, Table 3.1 summarises the key features and differentiating factors across these approaches. This discussion establishes the foundation for evaluating the relative position of the proposed architecture within the broader landscape of BDNS research.

3.1.1 Namecoin

Namecoin (55) (launched in 2011) is one of the pioneering BDNS projects. Its primary objective is to decentralise domain registration and create a secure and censorship-resistant system. Namecoin extends the traditional DNS to provide users with increased privacy and control over their domain names. By using a blockchain network, Namecoin seeks to eliminate central points of control, making it resistant to censorship and domain seizures.

Mechanism

Namecoin is based on the Bitcoin codebase (51) and functions as a unique cryptocurrency with a primary focus on providing a decentralised DNS. Operating independently, Namecoin manages the *.bit* TLD within its blockchain. The blockchain stores key-value pairs, where the keys represent domain names and the values contain data such as IP addresses and public keys. Users are able to interact with the system using wallet software such as Namecoin Core [1] or Electrum-NMC [2]. They can also register custom domain names, edit existing records, or transfer domain ownership using the wallet options. All of these actions generate transactions on the Namecoin blockchain and ensure that all domain registrations and records are resistant to censorship and tampering.

Challenges

Namecoin was the first BDNS to offer security and decentralisation. However, its primary limitation is the lack of support and adoption, leading to insufficient computing power. This makes Namecoin more vulnerable to well-known attacks like 51% attacks (56) compared to other analogous systems (57). Also, Namecoin's low fees encourage domain squatters, making it easy for people to buy names with very low cost. Unlike systems incorporating auctions or algorithmic pricing to align name costs with their market value, Namecoin lacks such mechanisms. This absence increases the risk of domain squatting, undermining the proper distribution and effective usage of domain names within the Namecoin ecosystem (55).

3.1.2 Blockstack

Blockstack (58) is a blockchain network that includes a decentralised DNS called the Blockstack Name System (BNS). Blockstack's main goal is to create a more user-centric internet that gives people more control over their digital identity and data. In particular, Blockstack's objective is to decentralise domain registration, by giving users ownership and control over their domain names. By using blockchain technology, Blockstack aims to create better security, privacy, and autonomy for users in a more open and fair internet environment.

[1] https://www.namecoin.org/download/
[2] https://www.namecoin.org/docs/electrum-nmc/

Mechanism

Blockstack DNS is built on Bitcoin's blockchain. Unlike traditional DNS systems, Blockstack DNS supports custom TLDs such as *.id*, *.podcast*, and *.helloworld*, all hosted on an independent blockchain. The blockchain is used to securely store records about domain registration, ownership, and decentralised identity data. Users interact with DNS through the Blockstack Browser [3], an easy-to-use application available on multiple platforms. The browser allows users to register domains, update records, and transfer ownership, which is done through transactions on the blockchain. DNS lookups of Blockstack domains are resolved through the browser, which can query the blockchain and retrieve decentralised identity and associated data for a specific domain.

Challenges

Scalability is a key issue for many blockchain solutions, and as Blockstack's user base and transaction volume grow, keeping efficiency and speed on its blockchain could become quite a challenge. Blockstack DNS runs on the Bitcoin blockchain, which has its own limitations regarding transaction throughput and block size. As more domains are registered and updated, the blockchain might get crowded, resulting in delays for DNS transactions and increased transaction fees. Additionally, reliance on a public blockchain like Bitcoin could create scalability concerns as the network expands, which might affect how well the Blockstack DNS system performs and responds.

3.1.3 Ethereum Name Service (ENS)

Ethereum Name Service (ENS) (59) is a decentralised DNS built on the Ethereum blockchain. Its primary objective is to simplify and improve the user experience in interacting with blockchain addresses. ENS allows users to register and manage domain names ending in *.eth* in a decentralised manner. The primary goal is to replace complex and hard to remember crypto addresses with human readable names, making the broader adoption of blockchain technology easier and faster. ENS aims to provide users with a secure, user friendly, and decentralised naming infrastructure on the Ethereum network.

Mechanism

ENS runs on the Ethereum blockchain and allows users to register domains in a decentralised way. It manages the *.eth* TLD, which is not compatible with traditional

[3] https://browser.blockstack.org

DNS, and important information, including domain registrations and ownership, remains stored in the Ethereum blockchain ledger. To interact with ENS, users can use Ethereum wallet software like MetaMask [4], which integrates with ENS features. The wallet supports tasks such as domain registration, DNS record updates, ownership transfers, and DNS lookups. The system offers a decentralised and secure way to manage domain names on Ethereum. It is worth noting that while some operations like domain lookups appear transactional and may suggest extra costs, they are actually read operations and do not require any transactions or fees.

Challenges

A major issue faced by ENS is the gas fees during congestion, which may affect the operation of the system, especially for users who are registering, updating, or transferring a domain name. During periods of high congestion on the Ethereum blockchain, gas fees can increase significantly while users are competing with each other to have their transactions validated by miners (60). Because of this, performing ENS transactions may become too expensive for users, especially for those participating with limited resources or smaller transactions. In addition to costs, gas fees add uncertainty for users during periods of congestion which may discourage them from planning and budgeting effectively for ENS transactions. This uncertainty in fees can have an effect on executing necessary actions on the ENS (such as renewing and updating domain names) that may disrupt the normal operating process of decentralised applications (dApps) or services.

3.1.4 Handshake

Handshake [5] is a decentralised, permissionless naming protocol with a core objective of revolutionising traditional DNS. Its primary goal is to create an alternative to existing CAs and naming systems by establishing a decentralised and censorship-resistant DNS infrastructure. Handshake seeks to democratise domain ownership, providing users with increased control over their digital identities and mitigating issues related to centralised control and censorship in the current DNS landscape.

Mechanism

Handshake naming protocol has its own dedicated blockchain, isolated from the traditional DNS. Introducing various custom TLDs such as *.hs* Handshake operates

[4] https://metamask.io/
[5] https://hsd-dev.org/files/handshake.txt

independently of traditional DNS, allowing users to create and manage domains on its blockchain. The blockchain stores ownership details, utilising a decentralised and secure system for tamper-resistant records. Interactions with Handshake occur through specialised wallet software like Namebase [6] enabling users to participate in blind auctions using HNS tokens for domain registration. DNS record updates, domain ownership transfers, and DNS lookups are executed through transactions on the Handshake blockchain, providing a decentralised and secure environment for managing and trading Handshake domain names.

Challenges

The main challenge for Handshake DNS is to get it to work with the legacy DNS infrastructure. While Handshake relies on a decentralised naming system, integrating its elements into the legacy DNS servers and resolvers (while maintaining proper performance and security) is complicated. One of these challenges is to achieve interoperability between the decentralised structure of Handshake and the hierarchical nature of the legacy DNS. In practical terms, protocols and standards must be developed to ensure that Handshake names can be resolved by legacy DNS servers and browsers without compromising performance or security.

3.1.5 Unstoppable Domains

Unstoppable Domains [7] is a project with the primary objective of using blockchain technology to provide users with censorship-resistant and truly decentralised domain names (61). The project aims to replace traditional domain extensions with blockchain-based extensions, allowing users to have full control and ownership of their domain names without the risk of censorship or domain seizures. Unstoppable Domains seeks to allow individuals to use their domain names to receive cryptocurrency payments, host decentralised websites, and manage their digital identities in a secure and privacy-focused manner.

Mechanism

Unstoppable Domains operates on various blockchain networks, including Ethereum and Zilliqa [8] and manages unique blockchain TLDs like *.crypto* and *.zil* that work independently from traditional DNS. Important information related to the domain names (such as ownership) is stored on the Ethereum blockchain making the domain

[6] https://www.namebase.io/
[7] https://unstoppabledomains.com/
[8] https://www.zilliqa.com/

immutable and resistant to censorship. User interaction is done through wallets (such as MetaMask) while DNS lookups for these domains are made through decentralised resolution protocols. The registration process includes a one-time fee and submitting a transaction on the Ethereum blockchain to tie the domain name to the user's Ethereum address.

Challenges

Similarly to ENS, the main challenge for Unstoppable Domains is the unpredictability of gas fees related to operation on the Ethereum blockchain. During times of network congestion or elevated gas prices due to significant activity on the network, fees can move rapidly higher. For example, findings in (62) saw that registering a new name through the Unstoppable Domains service led to approximately $80 in gas fees during a period of increased fees. In contrast, the actual cost of the name itself was $10. This challenge of unpredictable and potentially high gas fees during network congestion can deter users from utilising Unstoppable Domains and other Ethereum based services, particularly for smaller transactions or users with limited resources. This introduces uncertainty and unpredictability into the registration process, making it challenging for users to budget effectively and plan for domain acquisitions.

3.1.6 B-DNS

B-DNS (63) is another blockchain solution that aims to solve security vulnerabilities in the traditional DNS, such as cache poisoning and DDoS attacks. In order to overcome the limitations of existing BDNS solutions that rely on the heavy PoW protocol and inefficient query mechanisms, B-DNS introduces a PoS consensus protocol that relies on an index of domains. This work examines B-DNS and legacy DNS across security metrics and factors such as an attack's success ratio, cost, and attack surface. The experimental results show that B-DNS improves security by reducing successful attacks and increasing resistance to vulnerabilities compared to traditional DNS.

Mechanism

B-DNS is designed with a four-layer architecture. At the *Data Layer*, DNS records are stored as immutable transactions in the blockchain, utilising operation records for registration, update, and revocation functionalities. Inspired by Bitcoin's scripting system, these records enable dynamic domain ownership changes and content updates. The *Index Layer* provides fast searching by maintaining an index tree which maps domain names to IP addresses, with bloom filters helping in fast revocation checks. The *Consensus Layer* uses a PoS consensus protocol to ensure the consistency of

the DNS records through block generators that are chosen with stake-related probabilities. The *Network Layer* is responsible for communication between B-DNS name servers, recursive resolvers and end-users, so they can query DNS records directly from the blockchain while still being compatible with DNS servers and systems.

Challenges

Mitigating DDoS attacks in B-DNS presents a notable challenge despite the detection of fewer vulnerabilities compared to traditional DNS. In particular, the security analysis that was performed showed that in traditional DNS, 24 vulnerabilities that could lead to DDoS attacks were detected, in B-DNS only 12 vulnerabilities were detected. However, it is essential to recognise that this improvement does not necessarily indicate that B-DNS effectively prevents DDoS attacks. Instead, the B-DNS architecture may render it more resilient to such attacks, making it more challenging for attackers to exploit vulnerabilities and disrupt domain resolution processes. DDoS attacks could still disrupt B-DNS, so more research and development is needed to improve its defenses.

3.1.7 Blockzone

BlockZone (64) is a BDNS system aimed at addressing the centralisation issues of the traditional DNS architecture and management. BlockZone uses blockchain by treating DNS name servers as nodes in the network that can each maintain records for the entire network. The use of Practical Byzantine Fault Tolerance (PBFT) ensures consensus and data consistency while providing additional advantages, such as a fast consensus process and low network traffic. The result is that BlockZone has much higher parsing and authentication efficiency, and the improved consensus algorithm provides a much higher level of efficacy than PoW alternatives.

Mechanism

BlockZone operates using smart contracts on the Ethereum blockchain to manage DNS records, hierarchical references, and history updates. BlockZone uses four smart contracts for accuracy purposes: Consensus Contract (CC), Relationship Contract (RC), Ownership Contract (OC), and History Record Contract (HC). Adding a new node (name server) requires submitting a request to the blockchain network which the consensus contract will confirm and, if valid, approve. Data updates are made by synchronising resource records in the external InterPlanetary File System (IPFS) [9] and

[9] https://ipfs.tech//

sending the information to the service contract, which includes the updated domain name information and history records. Data is retrieved by end users when starting a query request and trusted servers query the blockchain's service contract to check the integrity of the received information. Finally, BlockZone's consensus algorithm (which is a variation of PBFT) combines communication and verification segments of the process, creating a more efficient system with minimal overhead.

Challenges

Although BlockZone shows significant improvement in parser throughput, efficient authentication, and distributing query requests, remains a considerable limitation. Firstly, the root servers at the centre of the system represent a single point of failure, especially when there are cache misses. The root server structure relies on a centralised system, contrary to BlockZone's design (non-centralised system), and it may impact system resilience and reliability. Secondly, even though BlockZone uses the PBFT consensus among server nodes to process transactions efficiently, the frequency of updates increases the length of the blockchain, causing storage overheads. The storage overhead of the blockchain also increases as the number of server nodes grows (for new domain storage). Therefore, BlockZone's system significantly improves authentication efficiency, but challenges associated with storage overhead and the possible centralised risk for a standard operating framework are still under consideration.

BDNS Name	Objectives	Mechanisms	Supported TLD	Limitations
Namecoin	Decentralised domain registration, censorship resistance	Bitcoin blockchain, blockchain-based registration	.bit	Insufficient computing power, integration, scalability, regulatory scrutiny
Handshake	Decentralised naming protocol, alternative to Certificate Authorities	Handshake blockchain, blockchain-based auction system	.hs	Usability, integration, competition with existing DNS, regulatory scrutiny
Blockstack	Decentralised internet, user control over data	Bitcoin blockchain	.id, .podcast, .helloworld	Usability, integration, scalability, regulatory considerations
ENS	Simplify interaction with Ethereum, human-readable smart contract addresses	Ethereum blockchain	.eth	Usability, integration, name resolution speed, regulatory considerations
Unstoppable Domains	Censorship-resistant domains, user-controlled	Ethereum and Zilliqa blockchains	.crypto, .zil	High gas fees, integration, industry acceptance, regulatory considerations
B-DNS	Prevent cache poisoning and DDoS attacks	Agnostic blockchain, 4-layer architecture	Agnostic	Vulnerable to DDoS attacks
Blockzone	Address centralisation issues of traditional DNS architecture and management	Ethereum blockchain	Agnostic	Single point of failure risk, storage overhead

TABLE 3.1: Objectives, mechanisms, supported TLDs and limitations of BDNS.

3.1.8 Other BDNS Proposals

The reviewed BDNS solutions reflect the most active and relevant research efforts in moving DNS to the blockchain. There are a few alternative solutions that have been proposed, but many of those are only in the preliminary stages of development, or they merely provide discussions that are high level, with little to no technical analysis. This maturity aspect limits the degree to which any meaningful analysis can be done. Performance data, security proofs, or deployment details are simply absent in most cases. However, identifying these emerging approaches helps to demonstrate the

range of interest in re-thinking DNS security and resilience through decentralised means. In this research, we include an overview of selected experimental or conceptual proposals, while recognising that their eventual impact will depend on further research, refinement, and empirical validation.

EmerDNS [10] operates on the Emercoin blockchain and uses blockchain transactions to store and manage records about domain names (65). It charges users for registering and updating domains with the Emercoin cryptocurrency (EMC). Each domain is cryptographically tied to the user's blockchain address so that ownership is linked to the user's blockchain identity. Unlike the traditional DNS, where registrars and registries are still in an administrative position of control, EmerDNS eliminates central points of control, making censorship or forced revocation of domains significantly more difficult. In addition, EmerDNS uses Emercoin's Name-Value Storage (NVS) (66), a generalised key-value storage system embedded in the blockchain, which allows arbitrary metadata to be attached to domain names (such as IP mappings, mail servers, or service descriptors). Despite its security and censorship-resistant benefits, EmerDNS suffers from limited adoption. It competes directly with the standard global DNS infrastructure, and domain names registered in EmerDNS are not universally resolvable without specialised software or resolvers recognising its namespace. This fragmentation reduces incentives for service providers and users to migrate from the traditional DNS ecosystem.

AuthLedger (67) offers a blockchain-based alternative to the traditional CA system, aiming to address popular issues related to CA compromise, fraudulent certificate issuance and excessive centralisation of trust. AuthLedger runs on Ethereum and achieves decentralised verification instead of hierarchical trust, meaning that all certificate issuance events are recorded and validated as part of Ethereum transactions. Unlike BDNS systems such as EmerDNS and Namecoin, AuthLedger does not manage or sell domain names, but only focuses on authentication, while it can be used as a decentralised public key infrastructure (PKI). The system allows for transparent and tamper-evident logging of authentication records, making it easier to audit the lifecycle of the certificate and to detect whether an issuer has misbehaved. However, being hosted on Ethereum comes with questions about scalability and cost, as frequent certificate operations can incur high gas fees.

DNSLedger (68) extends the vision of existing BDNS initiatives (e.g., Namecoin, Blockstack) and establishes a hierarchical multichain model for distributed domain management. Rather than storing all the records on a single blockchain, DNSLedger segments domain data across multiple interconnected chains, reducing bottlenecks. In this model, each chain can be mapped to a namespace, functional role, or security level, and inter-chain communication for domain resolution is developed. Compared to traditional models, this structure is more powerful in terms of scalability while

[10] https://emercoin.com/en/documentation/blockchain-services/emerdns/emerdns-introduction/

creating possibilities for applications within IoT ecosystems where lightweight devices will need efficient name resolution and identification. By decentralising domain management and creating an adaptable blockchain structure, DNSLedger improves resilience to tampering and cache poisoning attacks while not relying on centralised BDNS architecture. However, it depends on existing BDNS systems, limiting its independence in real-world applications and therefore its deployment remains largely conceptual, with few real-world implementations or adoption efforts to date.

BlockONS (69) is a permissioned blockchain DNS system built on Hyperledger to provide secure name resolution for IoT devices. It addresses the issues of scalability and unpredictability that are common in public blockchain systems, as services run in an open network. BlockONS achieves this by running within a closed, dedicated group of trusted organisations, unlike public blockchains, which involve multiple untrusted participants. The design of BlockONS mitigates specific security vulnerabilities, such as DNS cache poisoning and MITM attacks, which can be particularly harmful in IoT ecosystems as devices typically have limited security capabilities. More importantly, BlockONS's permissioned architecture ensures predictable levels of performance and governance, but at the same time sacrifices some of the openness and censorship resistance associated with public blockchain BDNS systems. BlockONS is therefore more likely to be considered for adoption within closed or industry-specific IoT deployments, rather than as a comprehensive alternative for the global DNS infrastructure.

ConsortiumDNS (70) introduces a three-layer architecture to address storage issues in early BDNS approaches, including Namecoin and Blockstack. In this design, the blockchain layer only stores domain transactions (registrations, changes, transfers), whereas the actual domain data is stored off-chain in a third-party system. An intermediate consensus layer acts as a bridge between the blockchain and the off-chain storage system, which coordinates transaction integrity and consistency. Separating these tasks enables a much more effective way of scaling the storage process, as the large-scale data associated with DNS does not need to be maintained in the blockchain itself. ConsortiumDNS was developed for more collaborative environments where multiple stakeholders, such as ISPs, registrars, and corporations, work together through a consortium architecture. This makes the system less decentralised than public BDNS systems, but potentially more efficient and cost-effective. However, off-chain storage introduces trust assumptions as the security of the overall system depends on the correct functioning and availability of the external storage infrastructure.

3.2 Alternative Non-Blockchain DNS Solutions

Alongside BDNS approaches, many systems have implemented decentralised DNS through different architectures. These approaches aim to overcome the restrictions of the traditional DNS infrastructure through P2P networks, DHTs, and other decentralised technologies. In contrast to BDNS systems, these approaches do not rely on a blockchain for consensus or record-keeping, but rather assume distributed control and authority from a decentralised network topology and architecture design. Their overall goal has been to retain the scale and efficiency of traditional DNS while removing the central points of failure present in the centralised DNS system. By relying on cooperative node participation, replication, and fault-tolerant routing methods, these DNS systems offer low-latency resolution while reducing the potential risk of censorship and tampering. While these systems can achieve higher resolution and lower overhead compared to blockchain-based solutions, they face challenges in terms of security guarantees, trust establishment, and widespread deployment, as their success depends on the willingness of participants to join and maintain the distributed infrastructure. We present the most notable non-blockchain DNS solutions below and summarise them in Table 3.2.

Danielis and Altmann (71) propose P-DONAS, a DHT-based P2P DNS system designed for ISP access nodes. With P-DONAS, DNS traffic load and operational costs are reduced by distributing resolution tasks to the access nodes, rather than putting all the weight on the central DNS resolver. This architecture can coexist with the traditional DNS system, allowing ISPs to adopt it in stages. Their findings validate their design for scalability and feasibility, both of which improve efficiency at high request rates, adding value to ISPs operating under growing internet demand.

Sancho and Pereira (72) present a censorship-resistant DNS that uses a P2P network to distribute zone files, which are user-verifiable and independently maintained. This design allows clients to bypass state-level censorship or filtering while retaining the flexibility to choose trusted nodes for verification, creating a selective trust model. By using distributed and cryptographically verifiable zone files, this system ensures that tampered or forged responses can be detected easily. The system is particularly relevant for scenarios facing government- or ISP-level restrictions on information access, since it serves as a practical anti-censorship mechanism.

Cox and Muthitacharoen (73) investigate DHash as an alternative to the Chord DHT DNS record storage. They have decoupled aspects of domain ownership and domain hosting by allowing the actual DNS records to be spread across several nodes, which separates domain administrative functions from domain hosting capabilities. This action improves fault tolerance, while it also mitigates previous administrative challenges. With a structured overlay, it can provide predictable lookup performance, even with failed nodes. The solution illustrates both the strong fault tolerance and

scalability potential of DHT-based DNS, while simultaneously highlighting the challenges of maintaining consistency and frequent updates.

Song and Koyanagi (74) propose a hybrid DNS model that benefits from the robustness of DHTs and the efficiency of traditional DNS hierarchies. They are motivated by the latency overhead common in P2P-based DNS applications. A hybrid architecture allows them to take advantage of the benefits of both paradigms: decentralised robustness from DHTs and query efficiency from the legacy DNS infrastructure. Their analysis demonstrates reduced latency compared to fully P2P systems, making it more suitable for real-world adoption where performance remains a critical concern.

OpenNIC[11] operates as a community-driven, decentralised DNS system that provides free and open access to top-level domains outside the control of ICANN. By adopting democratic governance principles, OpenNIC challenges the centralised authority of traditional DNS and promotes transparency, neutrality, and inclusiveness in namespace management. The system supports both alternative namespaces and interoperability with ICANN-managed domains, making it an example of a voluntary initiative that balances decentralisation with practical usability.

Unlike existing decentralised DNS solutions (both blockchain- and non-blockchain-based), which primarily attempt to restructure or replace the traditional DNS hierarchy, this research introduces a fundamentally different perspective by focusing on the recursive resolver. While most prior systems aim to substitute the root or authoritative name servers with distributed mechanisms, our architecture proposes replacing the recursive resolver with a decentralised, P2P network. The recursive resolver plays a pivotal role in DNS resolution, acting as the intermediary between clients and authoritative servers, and is often the single obstacle exploited in censorship and surveillance efforts. By decentralising this service, we directly address challenges that arise from traditional infrastructure, ensuring that the resolver is not affected by targeted blocking, traffic monitoring, or resolver poisoning.

In this model, the resolver functionality is spread among participants in the network and follows a DHT-based organisation to manage resolution queries at scale. In addition, the resolver design maintains the global consistency and verifiability of cached records and resolution decisions using consensus-based mechanisms to update the cached information. In this way, there is no single operator of the resolver, and the model mitigates the risks of misconfiguration, malicious tampering, or collusion as well. More importantly, this design does not interfere with or replace the existing DNS hierarchy of root and authoritative servers, which could reduce barriers to initial deployment and allow the resolver to be used in conjunction with existing internet infrastructure.

[11] https://www.hyperledger.org/

Name	Objective	Mechanism	Limitations
Sancho and Pereira	Censorship resistance	P2P distribution of cryptographically verifiable zone files with selective trust model	Overhead in maintaining distributed zone files; depends on trusted node selection
Cox and Muthitacharoen	Improve fault tolerance and scalability	DNS records stored in DHash (Chord DHT); separation of domain ownership and hosting	Update consistency issues; administrative complexity
Song and Koyanagi	Reduce latency in P2P DNS	Hybrid architecture combining DHT robustness with DNS hierarchy efficiency	Relies partly on legacy DNS; incomplete decentralisation
OpenNIC	Decentralised, community-governed DNS	Democratic governance; support for alternative namespaces and ICANN interoperability	Limited adoption; not widely recognised outside its community

TABLE 3.2: Objectives, mechanisms, and limitations of non-blockchain DNS solutions.

3.3 Proof-of-Space Solutions

Beyond decentralised DNS, a critical aspect of this research involves the use of resource-based approaches to secure participation and protect against Sybil attacks. PoW and PoS have been investigated in the blockchain literature, but both introduce several issues and limitations. PoW leads to high levels of energy consumption, and PoS concentrates power among already advantaged participants. However, PoSpace offers a different design principle that uses the storage capacity of devices, providing a more environmentally sustainable and more hardware-accessible mechanism for consensus and network admission. This research uses PoSpace to build on work exploring resource-efficient alternatives to PoW and PoS.

The concept of PoSpace has evolved significantly since its introduction, with research progressing from theoretical definitions of storage-based proofs to their application in full consensus protocols. Early work emphasised verifiable storage, where a prover demonstrates that they have allocated and reserved physical disk space, often through challenge–response mechanisms that require consistent access to precomputed data. Later proposals extended these foundations into practical systems for cryptocurrencies, such as SpaceMint (75), which demonstrated the feasibility of storage-based consensus in permissionless settings. Recent research has sought to address trade-offs between efficiency and security, including related issues such as grinding attacks, disk reuse, and proof generation costs. These improvements show the progress of PoSpace research from the conceptual construction of space-hard

functions to implementations that consider security, efficiency, and fairness in a distributed network.

The remainder of the literature review is dedicated to PoSpace mechanisms. Here, we focus on the design, implementation, and application of PoSpace in decentralised consensus and we sumarise our analysis in Table 3.3. We discuss both foundational work and more recent developments of PoSpace, indicating the evolution of PoSpace from theoretical proofs of verifiable storage to viable systems driving cryptocurrency and cryptocurrency networks. We examine multiple approaches to the allocation of space, challenge–response protocols, optimisations for efficiency, and security considerations such as resistance to Sybil attacks and precomputation strategies. By surveying these works, we also seek to introduce the design choices of our proposed PoSpace implementation and identify the gaps that motivate our contributions.

3.3.1 Foundational Proof-of-Space

Ateniese et al. (76) established the basis of PoSpace when they examined verifiable storage as a resource constraint. Their work introduced the principles and mechanisms for proving data retrievability in distributed storage systems which later PoSpace work builds on. Their work introduces formal definitions for proof generation and proof verification and demonstrates that storage effort can be systematically measured and audited, no matter the device's location, providing a model that protocols have adapted and optimised for decentralised storage networks.

Mechanism

This proposal specifically focuses on client–server scenarios while enabling cryptographic proofs that a server stores data without the need for resource-intensive calculations. The method provides data retrievability and auditability, with a lower reliance on classic PoW models. It also sets out the specifications for the interaction of a challenge–response protocol to generate and verify proofs of storage using probabilistic sampling of stored blocks as part of the verification process, thus allowing for efficient detection of loss of stored data or tampering. Furthermore, these techniques are used as a foundation for potential adaptations to decentralised systems as well as PoSpace-based protocols in blockchain settings.

Challenges

Although Ateniese et al.'s approach is a starting point, it does not consider decentralised consensus or distributed ledger applications. Its constructions are not

designed for low-latency and minimal-overhead setups, and thus are not practical for large-scale P2P applications. The mechanism is best suited for controlled client–server deployments and does not take into account adversarial strategies in a fully decentralised environment, while questions about scalability, parallelisation, and efficiency in blockchain adoption remain.

3.3.2 SpaceMint

Dziembowski et al. (75) extended the PoSpace concepts into a full cryptocurrency system called SpaceMint. They showed how storage-based proofs can replace computationally intensive PoW and provide a more energy-efficient alternative to blockchain consensus. They provided further detail on space proofs integrated into block validation, transaction validation, and leader election while noting practical considerations for security, fairness, and decentralisation in a PoSpace-based cryptocurrency.

Mechanism

SpaceMint separates the mining process (plotting) from block generation. Miners first perform the plotting and store cryptographic data structures that represent allocated storage space. In block generation, the protocol issues time-based challenges which require miners to provide proofs produced from their plots. The proofs show the possession of the allocated storage space but do not require continuous computation. Furthermore, the protocol uses verifiable delay functions (VDFs) and deterministic sampling of the stored blocks to guarantee fairness and unpredictability of leader selection. Integrating verifiable storage into the consensus mechanism provides security and prevents grinding attacks while using far less energy than a PoW scheme.

Challenges

SpaceMint requires a one-off, intensive plotting phase, during which miners create sizeable cryptographic data structures to commit to storage. This setup assumes access to abundant disk space and sufficient computational power, limiting participation by devices with constrained memory or CPU resources. The use of time-locked proofs by the protocol introduces latency between challenge issuance and proof verification, reducing responsiveness and flexibility for quick network joins or dynamic scaling. Finally, storage-bound attacks, such as grinding or selective withholding of plots, as well as the costs of creating the input/output (I/O) for the plotting and computation, are practical limitations for the deployment of SpaceMint on edge or mobile devices.

3.3.3 Simple Proofs of Sequential Work

Cohen and Pietrzak (77) present Simple Proofs of Sequential Work (PoSW), which offer cryptographic assurance that a specified amount of sequential computing has taken place. Unlike PoSpace, PoSW is not storage-bound, and the management of a computational delay is central to its design rather than utilising disk resources. Limiting the storage capacity of PoSW affects its applicability in a blockchain system that emphasise storage, but PoSW can still provide an additional mechanism for protocols requiring predictable timing and sequential verification.

Mechanism

The protocol uses sequential computation by requiring miners to perform a chain of dependent operations, that cannot be computed in parallel. Each intermediate step produces a computed value that contributes to the final verifiable proof that specifies the total number of steps taken, and updates with the output from the previous step. This process provides the assurance that the computation took a minimum, measurable amount of time. PoSW serves well as a potential replacement for time-lock puzzles, VDFs, and distributed protocols that require a delay that is provably enforced without relying on any external storage or pre-computed values.

Challenges

Due to its strict reliance on sequential computation, PoSW does not solve storage limited proofs efficiently because it cannot take advantage of precomputing or parallel resources. Moreover, the rapid setup of PoSW and the requirement for low-memory deployment make it difficult to implement on mobile devices. For blockchain-based PoSpace systems, these constraints reduce flexibility in fast network joins, adaptive difficulty adjustment, or integration with storage-efficient consensus mechanisms.

3.3.4 Proofs of Space-Time

Moran and Orlov (78) proposed Proofs of Space-Time (PoST), by extending the PoSpace model. However, PoST requires that the committed storage is maintained continuously over a defined period. In practice, participants must demonstrate that their plots remain allocated and accessible during that time, and they must produce proofs that can be independently verified at expected intervals. The time component increases resilience against attacks based on precomputation and recycling of plots. PoST is especially relevant for blockchain applications where the commitment of

resources must be sustained in order to provide secure consensus and proportional participation over time.

Mechanism

The protocol combines storage proofs with a temporal dimension in which participants are required to prove they maintain the allocated storage over time. The verification of storage proofs still works through periodic challenge–response interactions with the plots that participants have stored and ensures that previously stored data cannot be reused or faked. This provides the proof data with long-term verifiability and, by doing so, increases security against adversaries that may attempt to recycle storage or avoid the work of generating a proof, which is beneficial for long-running decentralised networks.

Challenges

The continuous, time-bound storage commitment increases setup, verification, and I/O overhead. Extended space-time requirements cause longer initialisation and synchronisation delays, limiting rapid onboarding of new nodes. These constraints reduce practicality in resource-constrained or dynamic environments, such as edge networks or mobile deployments, where storage- and time-efficiency are critical.

3.3.5 Chia Network

Chia Network's implementation (79) is the most popular and widely deployed PoSpace-based cryptocurrency. It relies on a disk-intensive plotting phase where users create massive cryptographic data structures, which can take many hours, and utilise temporary storage that can reach hundreds of gigabytes. These plots are used in the farming phase, where participants receive deterministic challenges and submit proofs from their plots to add blocks to the blockchain. The design reduces energy consumption compared to PoW, allows for decentralised participation based on storage capacity, requires a long-term commitment to storage, and maintains blockchain security.

Mechanism

The protocol separates the plotting (storage preparation) phase from the farming (block generation) phase. In the plotting phase, users precompute a large cryptographic data structure and write it to disk, proving a verifiable amount of

storage. In the farming phase, miners face a deterministic challenge based on the blockchain state and use their plots to construct quickly verifiable proofs. This design requires a long-term commitment to storage, allows blocks to be validated without centralised evidence generation, and removes the need for continuous, energy-intensive PoW computation.

Challenges

The plotting stage requires substantial disk space and computational resources, which is not only resource-intensive but also slow. Full plot initialisation can take several hours, and parallelising plot generation is difficult. These issues make it impractical to adopt the approach on devices with limited memory or storage. Clearly, PoSpace will require an architecture and protocol design that promotes faster initialisation, reduced I/O overhead, and verification approaches optimised for heterogeneous device environments.

3.3.6 AstroChain

AstroChain (80) is a permissionless blockchain that introduces PoSpace in combination with a reconfigurable random beacon protocol to improve security and performance. The random beacon protocol resolves some of the limits of traditional PoSpace mechanisms by providing verifiable, unpredictable randomness that can be included in the leader election protocol and mitigate the "nothing-at-stake" problem. AstroChain implements a modular consensus, separating proof and transaction chains to ensure progress, provide deterministic block finality, and allow for efficient parallel verification. Nodes commit storage plots and validate blocks when challenged randomly to achieve energy-efficient decentralised consensus while maintaining resilience to high levels of network churn.

Mechanism

AstroChain integrates a PoSpace participant agreement with a random beacon protocol and Byzantine consensus strategies. The miners are required to commit a given amount of storage in order to participate, and the random beacon provides verifiable, unpredictable randomness, which is used for leader election. The Byzantine agreement guarantees block finality with certainty even when some entities may be malicious. This modular design boosts energy efficiency, has favourable properties for scalability, and achieves security properties for permissionless environments with high churn values.

Challenges

Incorporating PoSpace with random beacons and Byzantine consensus makes the protocols more complex, increases operational overhead, and may have a large storage footprint, limiting the participation of resource-constrained devices. Complexity and overhead increase through the management of multiple chains, as well as through high network churn, creating difficulties in maintaining efficiency, security, and overall manageability in their operation. When a protocol includes timely randomness generation, fair leader selection, and fault-tolerant finality, this adds considerable computational and communication costs.

3.3.7 Etherspace

Etherspace (81) is a blockchain system that integrates PoSpace into Ethereum's blockchain protocol. Instead of using computational power to generate consensus, Etherspace uses disk space, which is a much more energy-efficient alternative to PoW. It builds on Ethereum's infrastructure, including its transaction processing and smart contracts, while utilising disk space as a source of participation for nodes. Etherspace ensures decentralisation and consensus security against Sybil attacks via verifiable storage proofs. It also ensures that block rewards are proportional to committed storage while facilitating sustainable and scalable operations in permissionless networks.

Mechanism

Participants engage in storage commitment by producing plots from pseudo-random functions, which convert their allocated storage into provable data structures. When block validation takes place, the protocol issues deterministic challenges, where miners respond with proofs that reference their plots. In this way, the protocol allows nodes to provide their claims based on provisioned storage instead of computing power, saving energy and maintaining security and fairness during consensus.

Challenges

Plot generation and verification are intensive in terms of resources and I/O, which can be frustrating for devices that have limited storage or memory. Combining the protocol with Ethereum makes the system more complex, and increases overhead and scalability constraints, especially when it comes time to onboard new nodes or provide high transaction throughput. The requirement to commit large storage plots

interferes with getting the network set up quickly, which limits how dynamic deployments can be in cases where there are resource constraints on the device.

3.3.8 Filecoin

Filecoin (82) is a decentralised storage network powered by blockchain technology that rewards users for providing and accessing storage in a trust-minimised marketplace. Filecoin was developed by Protocol Labs and officially launched in 2020. It is an extension of the IPFS; however, unlike IPFS, Filecoin has financial elements that reward storage operators for providing resources, whilst holding them accountable through cryptographic proofs. The Filecoin network is designed to be an efficient, global, trust-minimised, and censorship-resistant alternative to centralised storage solutions offered by cloud computing services..

Mechanism

Filecoin relies on two cryptographic proof systems: Proof-of-Replication (PoRep) (83) and PoSt. PoRep verifies that the storage provider has dedicated a unique piece of physical storage to the client's data and is not reusing that storage space for multiple clients. PoSt expands this concept by requiring the provider to prove, at regular intervals, that they continue to store the data, preventing the storage provider from discarding the data after proving it once. A storage miner generates either a PoRep or a PoSt proof, which is then verified on-chain, and miners compete to earn block rewards by allocating storage capacity and generating valid proofs. Clients pay the storage provider for storing and retrieving the data in Filecoin's native cryptocurrency (FIL).

Challenges

The complexity of PoRep and PoSt introduces significant computational overhead, making participation resource-intensive and limiting the number of potential storage providers. Data retrieval latency is another practical issue: unlike centralised cloud services that guarantee high availability and global distribution, Filecoin depends on voluntary providers whose responsiveness can vary. The economic sustainability of the model is also uncertain, since long-term participation relies on the value of the FIL token, which is subject to high volatility. Governance and coordination are further challenges, as upgrades to the protocols and security fixes must be implemented without centralisation pressure. Finally, usability barriers such as client onboarding, payment mechanisms, and data availability assurances need to be addressed before Filecoin can compete effectively with established cloud providers.

Name	Objective	Mechanism	Limitations
Foundational Proof-of-Space	Verifiable storage as a resource constraint	Client–server model; probabilistic sampling for storage proofs; efficient data retrievability	Not designed for decentralised consensus; limited scalability; unsuitable for large-scale P2P environments
SpaceMint	Energy-efficient blockchain consensus using storage	Separation of plotting and mining; time-based challenges; verifiable delay functions for fairness	Intensive plotting phase; requires abundant storage; latency from time-locked proofs; I/O overhead
Proof-of-Sequential-Work (PoSW)	Sequential proofs for predictable timing and verification	Sequential computation of dependent operations; verifiable delay without storage	Inefficient for storage-based systems; limited parallelism; poor suitability for mobile/low-memory devices
Proof-of-SpaceTime (PoST)	Continuous proof of storage over time	Extension of PoSpace with temporal commitment; periodic challenge–response	Higher I/O and setup overhead; slow onboarding; impractical for edge/mobile environments
Chia Network	Popular cryptocurrency based on PoSpace	Plotting (disk-intensive) + farming phases; deterministic challenges; long-term storage commitment	Plotting requires huge temporary storage and time; high I/O costs; difficult for resource-constrained devices
AstroChain	Secure, scalable PoSpace blockchain with randomness	PoSpace with random beacon for leader election; Byzantine agreement for block finality	Protocol complexity; storage-heavy; operational and communication overhead
Etherspace	Energy-efficient Ethereum variant using disk space	Storage plots from PRFs; deterministic challenges; proportional rewards to committed space	Plotting and verification resource-intensive; Ethereum integration adds complexity; scalability limits
Filecoin	Decentralised storage marketplace	Proof-of-Replication (PoRep) + Proof-of-Spacetime (PoSt); rewards via FIL token	High computational overhead; retrieval latency; token volatility; governance and usability challenges

TABLE 3.3: Objectives, mechanisms, and limitations of PoSpace and related protocols.

Unlike the previously mentioned works, this research introduces a practical PoSpace protocol designed to optimise efficiency and security by addressing specific weaknesses in PoSpace, namely amortisation and rationality attacks. Our PoSpace approach incorporates fast initialisation, much lower storage overhead, and secure verification methods to allow participation across a much wider selection of devices. Our analysis also provides insight into the scalable deployment of PoSpace, showing the high throughput the protocol can support and the efficiency it achieves, while retaining both security properties and energy efficiency.

3.4 Linking Related Work to The Research

In this chapter, we examined decentralised DNS solutions and PoSpace protocols. Non-blockchain DNS solutions are efficient and low-latency but do not offer security or censorship resistance. Blockchain DNS has greater trust and tamper resistance but worse scalability and performance compared to non-blockchain solutions. Similarly, PoSpace research has evaluated storage-based consensus mechanisms ranging from proofs of retrievability in theory to applications such as Chia and SpaceMint, as well as extensions of their original definitions, such as PoST and PoSW.

Our work derives from these insights, as we propose a practical and efficient PoSpace protocol within a decentralised DNS. With modifications to improve resistance to storage- and time-based attacks, optimise setup time, and enable wide user participation, our approach fills gaps between theoretical PoSpace constructions and practical implementation. This makes our protocol well suited for application in a full decentralised storage scheme for scalable, secure, and energy-efficient systems, directly guiding the design of our DNS-based blockchain constructs.

Chapter 4

Threat Model

This chapter defines the adversarial assumptions and threat landscape for the proposed system. The goal is to formally define adversarial capabilities, identify potential attacks, and quantify their impact on the system. Since the design integrates a decentralised recursive resolver and a PoSpace-based membership mechanism, each component requires a separate threat model. We first introduce global adversary assumptions, followed by the specific threats to the resolver and the PoSpace scheme. The chapter concludes with a summary table that maps adversarial capabilities to threats and mitigation strategies.

4.1 Adversary Assumptions

We assume a polynomially bounded adversary $\mathcal{A}$ that can control a fraction f of the nodes in the network. The adversary has computational resources exceeding those of individual honest participants, but cannot break standard cryptographic assumptions such as the collision resistance of hash functions or the reliability of digital signatures. The adversary can observe, inject, delay, and drop network traffic. Collusion between adversarial nodes is allowed, enabling coordinated behaviours such as query manipulation or resource pooling.

We define N as the total number of participating nodes and H as the set of honest nodes. The fraction of adversarial nodes is $f = 1 - |H|/N$. We assume $f < 1/3$ for protocols requiring Byzantine fault tolerance, although for Sybil resistance in PoSpace the effective limit is determined by adversarial storage capacity (see Section 4.3).

We do not assume a global passive adversary that can monitor all internet traffic simultaneously. Attacks against external infrastructure (e.g., DNS root or authoritative servers), physical coercion of node operators, or cryptographic breaks are out of scope.

4.2 Threat Model for the Decentralised Resolver

The decentralised resolver is exposed to a broad range of threats that can impact its reliability and trustworthiness. These include integrity attacks, where adversaries attempt to inject or manipulate DNS records, availability threats such as DoS or eclipse attacks that disrupt query resolution; privacy risks arising from the observation and association of query metadata; and censorship-oriented behaviours, where adversarial nodes selectively block or alter requests to introduce access restrictions. Each of these categories point to critical security and operational challenges that the system must address to provide a resilient alternative to the centralised recursive resolver.

4.2.1 Integrity Threats

Adversaries may attempt cache poisoning, forged responses, or replay attacks to compromise the correctness of query resolution. In a cache poisoning attack, the adversary $\mathcal{A}$ injects a forged record r' for a query q such that $r' \neq r$, where r is the correct authoritative response. Once accepted by an honest node $u \in H$, the malicious record r' may be cached and subsequently served to multiple clients until the TTL expires, expanding the impact of a single successful injection. This affects both the accuracy of name resolution and the trustworthiness of the network.

Forged responses can also be constructed by exploiting predictable transaction identifiers, port numbers, or routing behaviour to outpace legitimate responses from authoritative servers. If the adversarial response r' arrives before the authentic r, honest resolvers may accept the malicious answer. Replay attacks represent another integrity risk: here, $\mathcal{A}$ resends outdated but once valid responses r_{t-1} in epoch t to mislead clients into using stale or incorrect mappings. Although the response is theoretically valid, its actual context has expired, creating opportunities for adversaries to direct traffic away from legitimate services.

The combined effect of these attacks is the corruption of distributed caches and the loss of client trust in resolution. Since a single poisoned cache entry can influence a large number of subsequent queries, integrity threats represent one of the most critical risks to decentralised recursive resolution and motivate the use of multi-path verification, DNSSEC compliance, and integrity-aware replication policies in the proposed design.

4.2.2 Availability Threats

DoS and eclipse attacks represent the primary threats to availability in a decentralised resolver. In a DoS attack, an adversary $\mathcal{A}$ floods a target node $u \in H$ with queries or malformed messages, exhausting its processing, bandwidth, or memory resources and preventing it from servicing legitimate requests. Since decentralised resolvers lack the large-scale infrastructure and traffic engineering used by centralised providers, individual nodes may be more vulnerable to saturation without enforced admission controls.

Eclipse attacks compromise availability by isolating an honest node from the rest of the network. Here, adversarial nodes attempt to occupy all overlay connections of u, such that u's routing table consists solely of malicious peers. Once eclipsed, u is unable to access honest paths and may receive only adversarial responses. The probability of an eclipse increases with the adversarial fraction f and decreases with the node degree d. For example, if each node maintains $d = O(\log N)$ neighbours in an overlay of size N, and $\mathcal{A}$ controls fN nodes, the probability that all d neighbours of u are adversarial is approximately f^d. This exponential relationship shows that even minor increases in node degree can effectively reduce eclipse probability, though at the cost of increased maintenance overhead.

When many nodes join and leave very quickly, the network becomes unstable (routing tables often break), limiting availability. Adversaries can take advantage of this to perform join-leave rotations, continually replacing honest neighbours with malicious ones, thereby increasing the probability of eclipsing targets. Adversaries can also partition the network by disrupting links between clusters of honest nodes in a way that partitions the network into isolated components, thereby reducing overall availability.

4.2.3 Privacy Threats

Even if queries are encrypted with mechanisms like DoH or DoT, the metadata of timing, frequency, and destination patterns are still exposed. Adversaries exploit this metadata to model users' sensitive behaviours, such as inferences about a user's browsing patterns, apps being used, or even social interactions. For example, if the user regularly looks up domains associated with a messaging service, an adversary may assume the user is communicating with someone. If the user makes periodic queries to domain names associated with healthcare or financial sites, an adversary may be able to assume the user has private health interests or medical conditions.

Colluding adversarial nodes are particularly effective at deanonymising users. If $\mathcal{A}$ controls k out of h intermediate hops in a query path of length h, the probability that $\mathcal{A}$ can observe both the source and destination of a query is given by

$$\Pr[\text{linkability}] = \frac{\binom{h}{k}}{\binom{N}{k}},$$

where N is the total number of participating nodes. As the adversarial fraction f grows, the likelihood of linkability increases significantly, weakening the privacy guarantees of the system. Even without full end-to-end visibility, partial observations can be analysed across time to build probabilistic user profiles.

Traffic analysis can also enhance adversarial capabilities. By examining inter-arrival times, query burst patterns, or correlations with external events, adversaries can perform website fingerprinting or detect the use of evasion tools. In decentralised networks, this threat is exacerbated if queries are routed through a small number of nodes or if caching patterns expose popular domains. While encryption raises the bar against passive interception, it does not eliminate metadata leakage, making privacy one of the most difficult properties to guarantee in recursive resolution.

4.2.4 Censorship Threats

Adversarial nodes could selectively drop, delay, or alter queries for targeted domains and enable censorship policies inside the resolver. An adversary could block access to politically sensitive content, censor entire categories of services (such as social media platforms) or redirect users to alternative resources that include manipulated information. Censorship threats are different from integrity attacks, which seek to create large-scale confusion, as their main focus is to suppress or control information flows.

In decentralised environments, censorship can be realised through coordinated collusion. When query validation requires responses from multiple nodes, adversaries controlling a sufficient fraction of those nodes may refuse correct answers or return false negatives. Specifically, if a consensus validation process requires t out of h nodes to agree, and $t = \lceil h/2 \rceil$, then the probability of successful censorship by adversaries is approximately

$$\Pr[\text{censorship success}] \approx f^t,$$

where f is the adversarial fraction of the network. This probability illustrates the critical role of agreement thresholds: low thresholds reduce latency but increase

susceptibility to censorship, whereas higher thresholds improve resilience at the cost of efficiency.

Censorship can also be performed in an indirect form. Adversarial nodes can create systematic delays for queries on specific domains to the point where access becomes functionally unusable without technically blocking access. For instance, adversarial nodes may selectively cause damage by manipulating data routing tables or cache entries to forward queries into dead ends and block access. An adversarial node may apply dynamic censorship techniques and only drop queries under certain conditions (e.g., during political activity or at times of high traffic), making detection more difficult.

4.3 Threat Model for the Proof-of-Space Protocol

The PoSpace membership mechanism is designed to bind network participation to physical storage resources. We consider a set of attacks that a malicious Prover might perform to reduce storage costs or manipulate the verification process (84; 85). These attacks exploit weaknesses in system assumptions about storage commitments and Prover behaviour.

4.3.1 Fake Storage Commitment (Amortisation Attack)

An amortisation attack (78) occurs when a malicious Prover P_i collaborates with other Provers P_j, where $i \neq j$, to reduce their individual storage costs.

Instead of each Prover allocating the required storage independently during the initialisation phase, they share a single committed storage space across multiple identities. This allows them to reuse the same stored data while pretending to have committed distinct storage spaces.

Formally, during the initialisation phase:

$$(P_i \leftrightarrow V)(\mathrm{prm}_i) \xrightarrow{\Pi_{\mathrm{init}}} \text{Continue}, \quad \forall i$$

where each Prover P_i uses the same underlying storage but presents distinct identifiers id_i to the Verifier V. Since the protocol relies on ids to differentiate commitments, the malicious Provers benefit from the fact that the storage itself is not directly verified, only the commitment to it.

As a result, multiple Provers falsely appear to satisfy the storage requirements, effectively dividing the storage cost among themselves. This undermines the

protocol's guarantees by allowing them to amortise the storage cost across multiple identities.

4.3.2 On Demand Computation (Rationality Attack)

A rationality attack occurs when a malicious Prover avoids committing the required storage during the initialisation phase and instead regenerates the necessary data on demand during the execution phase (86). In an honest execution, the Prover should store a dataset corresponding to their storage commitment. However, in this attack, the Prover skips storing the data and uses computational resources to recompute the required information interactively whenever challenged by the Verifier.

Formally, during the execution phase:

$$(P \leftrightarrow V) \xrightarrow{\Pi_{\text{exec}}} \text{Accept}$$

even though P has not truly allocated storage. The Prover relies on fast recomputation to respond correctly within the protocol's time constraints. This allows the Prover to pass verification without actually dedicating storage space, contradicting the core principle of PoSpace.

The attack takes advantage of the fact that the protocol may not provide strong enough latency checks to distinguish between data retrieval from storage and on the fly recomputation. As a result, a rational adversary might choose to trade off storage costs for computational effort, breaking the initial storage conditions.

4.3.3 Sybil Attack via Multiple Identities

In the context of PoSpace, a Sybil attack (87) via multiple identities occurs when a malicious Prover generates multiple identities $P_1, P_2, \ldots, P_m$ to falsely increase their influence or rewards in the PoSpace protocol without proportionally increasing their storage commitment.

In a typical PoSpace scheme, each Prover participates in the initialisation phase with a unique identifier id to commit to a storage bound N:

$$(P_i \leftrightarrow V)(\text{prm}_i) \xrightarrow{\Pi_{\text{init}}} \text{Continue}, \quad \forall i$$

In this attack, a single Prover P creates and registers multiple identities $\text{id}_1, \text{id}_2, \ldots$ while using the same underlying storage or only slightly increasing it. As a result,

each identity appears to commit independent storage during initialisation, but the Prover manipulates the protocol to gain more rewards, challenges, or influence without actually allocating that storage.

Since the protocol relies on identifiers to differentiate commitments, the adversarial attack allows the Prover to maximise their presence in the system while using minimal additional resources to appear as many distinct participants. This impacts the PoSpace protocol by giving a single adversary excessive control or rewards, and thus breaks the fairness assumption regarding honest storage commitments.

4.4 Out-of-Scope Threats

In this research, we do not consider threats from nation state control, physical seizure of storage devices, or legal orders requiring operators to censor queries. These sorts of complications are primarily political, or legal in nature, and fall outside the scope of purely technical solutions. Likewise, attacks that exploit weaknesses in the underlying cryptographic primitives such as breaking a collision resistant hash function, or forging a digital signature, are excluded as these types of assumptions ensure the security of the wider internet ecosystem. Attacks against DNS root servers, TLD registries or authoritative name servers are also excluded, as this work focuses only on the recursive resolution layer of the DNS. While these external components are also necessary to fully protect the DNS infrastructure, addressing attacks against them requires different models of trust and governance that are unrelated to the purpose of this research.

Tables 4.1 and 4.2 provide an overview of the main threats considered in this work, the corresponding adversarial capabilities required to mount them, and the mitigation strategies embedded in the proposed design. The table serves as a concise reference, highlighting how the threat model connects directly to the mechanisms introduced for security, privacy, and resilience in the decentralised resolver and PoSpace protocol.

Threat Category	Attack	Description / Impact
Integrity	Cache poisoning	Inject forged DNS records into caches. Clients receive incorrect mappings.
	Forged responses	Exploit predictable identifiers/ports to race authentic responses. Malicious answers accepted.
	Replay attacks	Resend expired but once valid responses. Clients misled with stale records.
Availability	DoS	Flood node with queries/malformed traffic. Exhaust resources.
	Eclipse attack	Adversary controls all neighbours of a node. Isolates it from honest peers.
	High churn	Join-leave rotations destabilise routing tables. Increase eclipse probability.
	Partitioning	Disrupt links between clusters of honest nodes. Splits network, lowers availability.
Privacy	Metadata leakage	Observe timing/frequency/destination patterns; user profiling possible.
	Colluding nodes	Multiple adversarial hops enable source–destination linkability.
	Traffic analysis	Timing and correlation analysis. Enables fingerprinting and behaviour inference.
Censorship	Selective blocking	Drop/modify queries for targeted domains. Deny access.
	Consensus manipulation	Control majority of validators. Force false negatives.
	Indirect/adaptive	Introduce delays or routing manipulation. Stealth censorship.

TABLE 4.1: Threats in the Decentralised Resolver.

Threat Category	Attack	Description / Impact
Integrity/Fairness	Amortisation attack	Multiple Provers share the same storage. Reduce real cost, undermines guarantees.
	Rationality attack	On-demand recomputation instead of storing data. Bypasses storage requirement.
	Sybil attack	One Prover creates many identities. Gains unfair influence/rewards without proportional storage.

TABLE 4.2: Threats in the Proof-of-Space protocol.

Chapter 5

Construction of the Decentralised Resolver

The proposed decentralised DNS resolver aims to eliminate the dependence on centralised recursive resolvers by distributing the resolution process across independent nodes. Each node is responsible for storing and serving DNS records (requested by other nodes) using a DHT, a process that enables fault-tolerant resolution of records. Queries are routed through the network (according to a structured protocol), where nodes efficiently locate the appropriate authoritative records. Each set of DNS records is cryptographically signed to ensure integrity and authenticity, preventing tampering or spoofing. Similar to traditional resolvers, local caching allows nodes to improve network performance while reducing bandwidth usage. This architecture can support scalable and resilient (e.g., censorship-resistant) DNS resolution while being extensible with current domain naming standards.

5.1 Formal System Specification

The Decentralised DNS Resolver is formally defined as a tuple

$$\mathcal{S} = \langle \mathcal{N}, \mathcal{R}, \mathcal{P}, \mathcal{K}, \mathcal{H} \rangle,$$

where:

- $\mathcal{N}$ is the set of participating nodes (peers).
- $\mathcal{R}$ is the global dataset of DNS resource records.
- $\mathcal{P} = \{C_1, C_2, \ldots, C_n\}$ is a partition of $\mathcal{R}$ into n discrete chunks.
- $\mathcal{K}$ is the m-bit identifier space $[0, 2^m - 1]$.

- $\mathcal{H} : (\mathcal{D} \cup \mathcal{N}) \rightarrow \mathcal{K}$ is the consistent hashing function mapping domain names d and nodes n into $\mathcal{K}$.

Definition 1 (Responsibility): A node n_j is responsible for a chunk C_i if

$$H(C_i) \in (ID(n_{j-1}), ID(n_j)] \pmod{2^m}.$$

5.2 High-Level Overview

Figure 5.1 outlines the components of the decentralised resolver and the flow of interactions between them. Each node $p \in P$ is placed on a Chord ring and stores a subset $C_i \subset F$ called a *chunk*, where F is the initial global dataset of DNS records. The peers responsible for handling the same chunk C_i form a permissioned blockchain (88), which is used solely for updating the chunk, as we explain in Section 5.5. The service is accessible only to nodes that have joined the network as peers. When a DNS request is initiated, the node uses the Chord DHT to locate the node responsible for the relevant chunk and extracts the full domain record. This resolution process operates in a decentralised manner, with the DHT providing an up-to-date list of peers and their corresponding chunks.

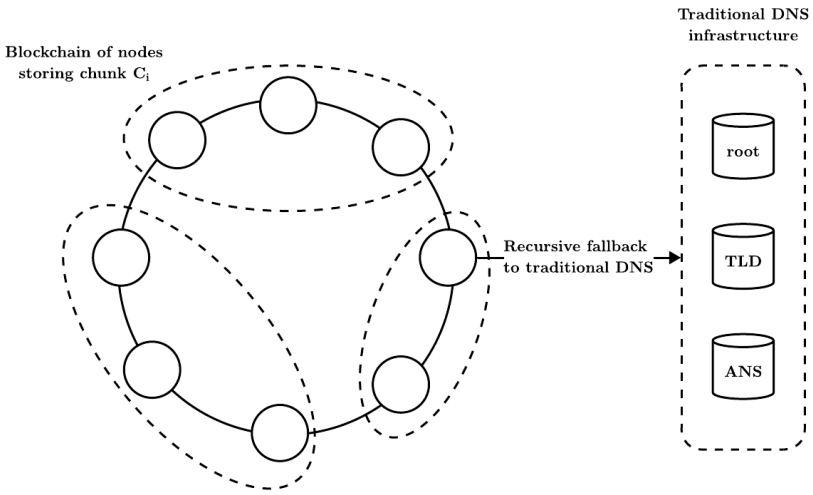


FIGURE 5.1: The structure and components of the decentralised resolver.

DNS records are time-sensitive, meaning that entries already cached expire according to their TTL values, with new records updated or added frequently. As a result, many queries may fail if the necessary data is unavailable locally or in the relevant peer set. In this situation, when a node cannot resolve a query from local or peer-stored data, it initiates the resolution process from the root servers, traversing the DNS hierarchy as needed (root $\rightarrow$ TLD $\rightarrow$ authoritative servers). This approach removes any dependency on an external recursive resolver, keeps the structure decentralised, and provides completeness to the resolution process.

5.3 Storage Model and Node Initialisation

The global DNS dataset consists of a large volume of domain–IP mappings that continuously evolve. To handle this scale and enable decentralised management, the dataset is partitioned into N chunks $C_1, C_2, \ldots, C_N$. Each chunk C_i contains a fixed-size (range-based) subset of records, determined by a consistent hashing function $h : \mathcal{D} \rightarrow [0, M)$, where $\mathcal{D}$ is the set of domain names and M defines the hash space:

$$C_i = \{d \in \mathcal{D} \mid h(d) \in [\frac{(i-1)M}{N}, \frac{iM}{N})\}$$

This ensures a uniform distribution of domains across nodes, reducing hotspots. For example, if $N = 10$ and $M = 2^{32}$, a domain with hash value $h(\texttt{example.com}) = 1{,}234{,}567{,}890$ would be assigned to chunk C_4.

Partitioning improves parallelism: multiple nodes can process queries or updates for different chunks simultaneously, lowering latency and increasing throughput. We

define Q as the total number of queries per second and k_i as the number of nodes serving chunk C_i. Then the expected per-node load is:

$$L_i = \frac{Q_i}{k_i}, \quad Q_i \approx Q \cdot \frac{|C_i|}{\sum_{j=1}^{N} |C_j|}$$

As the network grows, new peers can be dynamically assigned partitions using a consistent hashing ring, ensuring balanced load without central management. Formally, when a new node joins at position p in the hash space, it assumes responsibility for all keys d where $h(d) \in [p_{\text{prev}}, p)$, and the previous owner retains only those with $h(d) \geq p$.

This architecture supports modular record management, as specific administrative domains (e.g., .com, .org) can be mapped to dedicated partitions, which reduces the likelihood of lookup conflicts while also simplifying the zone delegation process. The strategy of policy-aware partitioning exploits semantic groupings of domains, such as clustering domains by registrant or geographic region to provide more locality and thus query efficiencies.

5.3.1 Key Assignment via Cryptographic Hashing

Each chunk C_i is assigned a unique key k_i using a cryptographic hash function H, such as the Secure Hash Algorithm 1 (SHA-1), commonly used in DHT and Chord architectures. The key is computed as:

$$k_i = H(C_i)$$

where C_i is represented as a deterministic serialisation of its records (e.g., concatenated domain–IP pairs in lexicographic order). The hash output places C_i at a point in the circular identifier space $[0, 2^m)$, where m is the bit length of the hash (e.g., $m = 160$ for SHA-1).

Cryptographic hashing provides two core benefits that are fundamental to the design of secure and efficient distributed systems:

Uniform distribution: Each chunk is probabilistically assigned a location in the identifier space, reducing storage and query hotspots. For instance, if N chunks are uniformly distributed over 2^m identifiers, the expected distance between neighbouring keys is approximately $2^m / N$.

Collision resistance: It is computationally infeasible for a malicious node to forge a chunk key, preventing targeted manipulation of data placement.

Key-based assignment enables deterministic, stateless routing in the network. When a node n_j has identifier $id_j \in [0, 2^m)$ in the Chord ring, the chunk C_i is stored at the successor node n_s such that:

$$id_s = \min\{id_j \mid id_j \geq k_i \bmod 2^m\}.$$

This allows any node to locate C_i using only its local finger table, without global knowledge of the dataset. For example, if $k_i = 0x3FA2$ and nodes have IDs $0x1000$, $0x4000$, $0x8000$, then C_i is assigned to node $0x4000$ as its immediate successor. Routing efficiency scales logarithmically with network size: in a network of N nodes, the expected number of hops to locate a chunk is $O(\log N)$, guaranteeing efficient lookups even as the system grows.

5.3.2 Chunk Placement Using the Chord DHT

Each chunk C_i with key k_i is assigned to a responsible peer using Chord's consistent hashing mechanism. We define $n_1, n_2, \ldots, n_P$ as the set of peers in the network with identifiers $id_j \in [0, 2^m)$, where m is the hash bit length. The successor node of k_i is defined as:

$$\text{successor}(k_i) = \min\{id_j \mid id_j \geq k_i \bmod 2^m\}.$$

This node stores C_i and answers queries for the corresponding domain records. If k_i exceeds all node IDs, the successor wraps around to the node with the smallest identifier, ensuring continuity of the circular space.

Each node maintains a finger table containing m entries, where the r-th entry points to the node at distance 2^{r-1} from the current node, modulo 2^m. Formally, for node n_j:

$$\text{finger}[r] = \text{successor}(id_j + 2^{r-1} \bmod 2^m), \quad r = 1, \ldots, m.$$

This enables routing to any key in $O(\log P)$ hops, where P is the number of active peers, even in large-scale networks. For example, in a network with $P = 32$ nodes and $m = 8$, a lookup may require at most $\log_2 32 = 5$ messages.

To improve load balancing, *virtual nodes* can be used. Each physical peer is assigned v virtual identifiers, effectively splitting its storage responsibility among multiple positions in the ring. This reduces variance in key distribution and mitigates hot spots.

Chord also supports *proximity-aware routing*. When multiple nodes qualify as the successor for a key, the node with the lowest observed network latency is chosen. This

optimises query response times without violating the consistency guarantees of the DHT (89).

This placement model ensures fully decentralised operation, as nodes independently determine responsibility for any chunk, fetch or update it as needed, and maintain the ring without relying on a central coordinator. Key reassignments occur only when peers join or leave, affecting $O(\log P)$ keys, which limits disruption in dynamic environments.

5.3.3 Node Join Procedure and Topology Update

The node join procedure serves as a critical security boundary where the PoSpace protocol is enforced to prevent Sybil attacks. Unlike standard P2P protocols that allow arbitrary entry, this framework requires a verifiable resource commitment before topology updates are committed to the identifier ring. This formalises the admission process, ensuring that every new peer contributes a measurable physical resource to the network's resilience. The operational sequence for a joining node is detailed in Algorithm 1.

Algorithm 1 Node Membership (Join)

Input: New node n_{new}, Bootstrap node n_{boot}
Output: Updated routing state or REJECT

1. **Identity Assignment:** Compute $ID_{\text{new}} = \mathcal{H}(n_{\text{new}})$.
2. **Network Entry:** Locate successor node
$$n_{\text{succ}} = n_{\text{boot}}.\text{find_successor}(ID_{\text{new}}).$$
3. **PoSpace Admission:**
   - n_{new} executes INITIALISATION_PHASE (Algorithm 6.1) to generate Merkle root R_{new}.
   - n_{succ} verifies the cryptographic proof and digital signature.
   - If verification fails, return REJECT.
4. **Data Transfer:**
   n_{new} fetches all chunks C_i such that $ID(C_i) \in (ID_{\text{pred}}, ID_{\text{new}}]$ from n_{succ}.
5. **Topology Repair:**
   - Update predecessor and successor pointers to integrate n_{new} into the ring.
   - Trigger STABILIZE() to update finger tables.
6. **Replication:**
   - Participate in replication to maintain availability across s successor nodes.

5.3.4 Replication for Fault Tolerance

Each chunk C_i is replicated at its s immediate successors in the Chord ring to ensure resilience against node failures. Formally, if the primary node responsible for C_i has identifier id_i, then replicas are stored at nodes with identifiers:

$$id_{i+j} = succ^j(id_i), \quad j = 1, 2, \ldots, s,$$

where $\text{succ}^j(id_i)$ denotes the j-th successor of id_i in the ring. This scheme tolerates up to $s - 1$ consecutive node failures without data loss.

Replication improves both fault tolerance and query availability. We assume that a node $n_{primary}$ is offline. Then any replica n_{i+j} can respond to queries for C_i. During node joins or departures, the new or neighbouring nodes retrieve chunks from one of the replicas instead of the original node, minimising network disruption:

$$C_i^{new} \leftarrow C_i^{replica}, \quad \text{if } n_{primary} \text{ unavailable.}$$

The replication factor s is configurable. Its optimal value can be estimated using the expected churn rate c and node reliability r:

$$P_{data_loss} = (1 - r)^s \leq \epsilon,$$

where ϵ is the maximum acceptable probability of data loss. Increasing s improves reliability but raises storage and update costs. Updates to replicas can be synchronous for strong consistency or asynchronous for reduced bandwidth and latency, depending on system requirements.

Extensions to the replication strategy include erasure coding, which reduces storage overhead by storing coded fragments instead of full copies, and proximity-aware replication, where replicas are placed on geographically diverse nodes to improve resilience against regional outages. However, successor-based replication provides a simple, decentralised, and effective mechanism suitable for dynamic P2P DNS networks.

5.4 Processing DNS Queries

To resolve the ambiguity inherent in applying legacy DHT lookups to a record-based DNS system, we formalise the resolution process as a two-stage mapping. First, the requested domain D is hashed to identify its corresponding chunk C_i. Second, the Chord routing protocol is used to locate the node n_{succ} currently responsible for that identifier space. This procedure is formalised in Algorithm 2.

5.4.1 Distributed Lookup and Query Validation

If the node does not store chunk C_i, it initiates a lookup using the Chord DHT. We assume n_0 to be the querying node. The query is forwarded along the finger table toward the successor of k_i, reaching the node n_s responsible for C_i in $O(\log P)$ hops, where P is the total number of peers.

Once it receives the responses $R = \{r_1, r_2, \ldots, r_m\}$ from the queried peers (primary and replicas), the node validates each response:

- Confirm structural correctness: required fields include `name`, `type`, `TTL`, and `value`.

Algorithm 2 Distributed DNS Resolution (Search)

Input: Domain name D, Requesting Node n_{req}
Output: Resource Record r or Error

1. **Key Generation:** Compute $k \leftarrow \mathcal{H}(D)$.
2. **Target Identification:** Identify target chunk C_i such that $k \in \text{Range}(C_i)$.
3. **DHT Lookup:**
   - Find node $n_{\text{succ}} = \text{successor}(k)$ using n_{req}'s finger table.
   - Request chunk C_i from n_{succ} and its s replicas.
4. **Validation:**
   - Collect responses $R = \{r_1, r_2, \ldots, r_m\}$.
   - If $\text{Majority}(R)$ exists, return v^*.
5. **Fallback:**
   - Else, execute $\text{TRADITIONAL_DNS_FALLBACK}(D)$.
   - Store result in C_i and update n_{succ}.

- Discard responses with invalid format, missing fields, or inconsistent TTL values.
- Apply a majority vote to reconcile conflicting responses:

$$v^* = \arg\max_v |\{r_k \in R \mid r_k = v\}|$$

$$\text{Accept } v^* \text{ if } |\{r_k \in R \mid r_k = v^*\}| > \frac{m}{2}, \text{ else reject.}$$

This mechanism tolerates up to $\lfloor \frac{m-1}{2} \rfloor$ faulty or malicious responses. For example, if $m = 5$ replicas respond, at least 3 must agree on the same value for acceptance.

Optional extensions include weighting responses based on node reliability w_k or applying cryptographic proofs (e.g., signatures) to prevent spoofing:

$$v^* = \arg\max_v \sum_{r_k = v} w_k$$

This improves trust without requiring centralised validation, ensuring that the distributed query returns accurate and consistent DNS records.

5.4.2 Fallback Resolution via Traditional DNS

If a requested domain D is not found in any chunk C_i ($D \notin \bigcup_i C_i$), the node initiates a fallback resolution using the conventional DNS hierarchy as follows:

1. The node sends iterative or recursive queries to root servers, then to TLD servers, and finally to the authoritative server for D, following the standard delegation chain.
2. Each response is validated for correctness, including the presence of `name`, `type`, `TTL`, and `value`, as in Section 5.4.
3. Upon successful resolution, the node stores the retrieved record in the appropriate chunk C_i according to the mapping $\mathcal{M}(D) = H(D)$, updating replica sets if necessary (see Section 5.5).
4. If the record does not exist in the global DNS infrastructure, the node returns an `NXDOMAIN` response to the client.

This mechanism guarantees availability for uncached or new domains, while ensuring they will eventually be added to the distributed network. For example, if `example.xyz` were not found in the DHT, the node would request `example.xyz` from the authoritative `.xyz` server, cache the responses on its node, and then have those responses cached on each successor node, avoiding any repeated fallback requests.

5.4.3 Mechanics of the Chunk-Level Permissioned Ledger

To ensure the integrity of DNS updates without the overhead of a global chain, the system utilises a *Micro-Ledger* architecture. Each chunk C_i is governed by a local consortium consisting of the primary node n_{primary} and its s successors.

Consortium Membership and Churn Management Membership in a chunk's consortium is dynamic and tied directly to the Chord ring topology. When a node n_{new} joins the ring at position k, it automatically becomes a validator for the chunks C_i falling within its successor range. Access is granted implicitly via the PoSpace admission protocol; a node cannot participate in the consensus process without first providing a valid, signed Merkle root to the existing consortium members.

Consensus and Validation Role The primary node responsible for a chunk acts as the Proposer, while the s replicas act as Validators. For a DNS update T_{update} to be committed:

- The Proposer broadcasts the update to the s replicas.
- Each replica verifies the signature against the domain owner's public key and checks the PoSpace liveness of the Proposer.
- A commitment occurs only upon reaching a majority agreement ($> s/2$).

Storage and Synchronisation The ledger is stored redundantly across the s nodes. When a node leaves the network, the Chord STABILIZE() protocol triggers ledger synchronisation. The new s-th successor fetches the recent transaction history from the remaining honest peers to preserve the chain of custody for that chunk.

Incentives and Sybil Resistance To discourage Sybil attacks, PoSpace challenges are issued at fixed intervals. Nodes that fail a challenge are immediately evicted from the consortium, and their stored records are re-replicated to healthy nodes. As an incentive to address sustainability challenges, nodes maintaining a high *Liveness Score* (proven via consistent PoSpace responses) are granted priority bandwidth for their own outbound recursive queries.

The formal process of state commitment within this ledger is detailed in Algorithm 3.

Algorithm 3 Chunk-Level Consensus (Commit)

Require: Update Transaction T, Consortium Peers $P = \{n_1, \ldots, n_s\}$
Ensure: Boolean (Commit/Reject)
Proposal: Proposer n_p signs and broadcasts T to all validators $n \in P$.
Verification:
for all $n_i \in P$ **do**
    Validate DNS record format, signature, and TTL
    Verify PoSpace liveness of n_p via most recent challenge timestamp
    if valid **then**
        Broadcast `PRE-COMMIT` to P
    end if
end for
Agreement:
if count(`PRE-COMMIT`) $> s/2$ **then**
    Commit T to local chunk-ledger
    Update corresponding DHT replicas
    return `True`
end if
return `False`

5.5 Updating DNS Records

DNS record updates happen in two scenarios. The first is expiration-based updates. When a record reaches its TTL, it is considered expired, and the responsible peers query the traditional DNS hierarchy to fetch the current value and initiate the update process. The second scenario is the addition of new records. Records retrieved via the fallback mechanism, as explained in Section 5.4.2, are introduced into the network and assigned to the appropriate chunk C_i using the mapping $\mathcal{M}(D) = H(D)$.

To maintain consistency and availability, updates are organised between peers responsible for the same chunk. Expired or modified records are replaced, while new records are propagated to successor nodes according to the replication factor s. To tolerate malicious nodes and ensure integrity, updates are recorded on a permissioned blockchain operating at the chunk level. Peers managing the same chunk form a local consortium that validates and commits updates. This approach eliminates transaction fees and reduces computational overhead compared to a permissionless blockchain. While record validation and blockchain consensus occur, nodes continue serving DNS requests through the Chord DHT, ensuring low-latency lookups and uninterrupted availability. Each chunk thus benefits from both decentralised storage and tamper-resistant update management.

5.5.1 Updates Based on TTL

When a DNS record expires according to its TTL, the nodes responsible for storing it initiate a refresh procedure. Each node first queries the traditional DNS hierarchy to obtain the latest version of the record. We define the current record as R_{old} and the newly fetched record as R_{new}. To propagate this update securely and consistently, the node creates a transaction $T_{\text{update}}(R_{\text{old}}, R_{\text{new}})$ and submits it to the permissioned blockchain that governs the chunk C_i containing the record.

The blockchain applies strict validation rules, checking record format, TTL correctness, and compliance with network policies. All peers storing C_i participate in the consensus process $\mathcal{C}^{\text{cons}}$, which ensures agreement before committing the update. The consensus outcome is formally defined as:

$$\mathcal{C}^{\text{cons}}(T_{\text{update}}) = \begin{cases} \text{valid}, & \text{if consensus is reached} \\ \text{invalid}, & \text{if consensus fails} \end{cases}$$

Once the validation is successful, each node updates its local copy of the record:

$$\text{Update}(\text{IP}, R_{\text{new}}(D)) \quad \forall\, \text{IP} \in C_i$$

The update process also integrates with the Chord DHT replication mechanism. Successor nodes holding replicas of C_i are notified of the validated update, ensuring that all replicas reflect the latest state. This maintains fault tolerance and read availability, allowing any replica to respond accurately to queries even if the primary node is offline.

To improve the efficiency of the network, updates can also be grouped or applied asynchronously over the replicas, so that bandwidth and consistency can be balanced. In case of conflicting updates (such as multiple nodes fetching different versions of the same record), the blockchain consensus mechanism adjusts them, therefore ensuring only one authoritative version is accepted. This combination of processes including TTL-driven refresh, Chord replication, and blockchain consensus, maintains the integrity, availability and consistency of DNS records across the distributed network, regardless of high demand or malicious peers.

5.5.2 Updates Based on Traditional DNS Fallback

When a requested DNS record is absent from the network ($D \notin C_i$), the nodes responsible for the chunk that should contain the record initiate a fallback retrieval from the traditional DNS hierarchy. The requesting node first uses the Chord DHT to identify which chunk C_i the record belongs to, as described in Section 5.4. If the requesting node itself stores the relevant chunk, it coordinates the insertion of the new record among all peers managing C_i.

Each node retrieves the record from traditional DNS and constructs a transaction $T_{\text{new}}(R_{\text{new}}(D))$ representing the addition of the new record. This transaction is submitted to the permissioned blockchain governing the chunk. All peers participate in the consensus process $\mathcal{C}^{\text{cons}}$, which validates the content according to network rules, ensuring the record is correctly formatted, non-conflicting, and authorised:

$$\mathcal{C}^{\text{cons}}(T_{\text{new}}) = \begin{cases} \text{valid}, & \text{if consensus is reached} \\ \text{invalid}, & \text{if consensus fails} \end{cases}$$

After validation, the record $R_{\text{new}}(D)$ is stored locally by all nodes responsible for chunk C_i:

$$\text{Add}(\text{IP}, R_{\text{new}}(D)) \quad \forall\, \text{IP} \in C_i$$

The update also triggers replication to successor nodes holding replicas of C_i, ensuring that the record remains available even if some nodes go offline. Nodes may propagate

updates asynchronously to optimise network bandwidth while preserving consistency.

If the record belongs to a chunk managed by other nodes, the requesting node forwards the retrieval request via the DHT. The responsible nodes then perform the traditional DNS fallback, submit the transaction to their blockchain, and insert the new record following the same process. This procedure guarantees that all nodes storing the chunk agree on the new record, maintaining integrity and consistency across the network. By combining DHT based identification, fallback retrieval, blockchain validation, and replication, the system guarantees that even previously uncached or novel DNS records are securely integrated and reliably available in the distributed network.

5.6 Evaluation of The Decentralised Resolver

We evaluate the strengths of the proposed architecture relative to legacy DNS, pointing out its benefits, especially given the primary advantages in terms of decentralisation, security, and transparency. The system overcomes many of the principal disadvantages of traditional DNS architecture to provide greater resilience to failure, tampering, and censorship, and we demonstrate how the proposed architecture achieves this. We elaborate on these improvements below.

5.6.1 Centralisation and Single Points of Failure

Traditional DNS depends on centralised resolvers and authoritative servers. This creates critical single points of failure; outages, misconfigurations, or direct attacks against resolvers and servers can lead to loss of domain resolution for very large populations of users (as discussed in Chapter 2). The proposed model mitigates these vulnerabilities by distributing the DNS records across a distributed P2P network. Each chunk of records is replicated across multiple nodes, allowing for redundancy and load balancing.

We assume C_i to be a chunk of DNS records, replicated across $r = |C_i|$ nodes, and $F \subset C_i$ the set of failed or unreachable nodes. The probability that a query fails due to node unavailability is:

$$P(\text{QueryFail}) = \begin{cases} 0, & |F| < r \\ 1, & |F| = r \end{cases}$$

This shows that the system remains available as long as at least one replica per chunk is online. With $r \geq 2$, the network can tolerate at least $r - 1$ simultaneous failures per chunk without service disruption.

DHT based routing ensures that queries are dynamically redirected to available replicas. Formally, for any key k mapping to chunk C_i:

$$\text{Resolve}(k) = \text{any node } n \in C_i \setminus F.$$

Chunk level consensus further guarantees that updates are consistent across all replicas. If T_{update} is an update transaction and $\mathcal{C}_i^{\text{cons}}$ the chunk consensus protocol, then:

$$\forall n \in C_i, \quad n.\text{accept}(T_{\text{update}}) \iff \mathcal{C}_i^{\text{cons}}(T_{\text{update}}) = \text{valid}.$$

This prevents a failed or malicious node from corrupting DNS records. By combining DHT based routing, chunk replication, and consensus-validated updates, the architecture guarantees fault tolerant resolution and mitigates risks associated with centralised points of failure.

5.6.2 Censorship and Tampering

Traditional DNS queries are vulnerable to censorship, interception, and manipulation, as shown by real-life examples, including the cases examined in Section 1. Compromised resolvers or adversaries can manipulate responses, direct them to alternative destinations, or prevent access to a specific domain. The proposed architecture addresses this threat through the use of a P2P network and validation techniques compared to a traditional DNS query. Under this model, DNS records are distributed across multiple peers in the network, and for every relevant DNS query, responses from all nodes are collected to provide the best possible resolution for the corresponding chunk.

We previously defined C_i as a chunk of DNS records replicated across $r = |C_i|$ nodes. We also define $M \subset C_i$ as the set of malicious nodes attempting tampering. A query result is accepted only if more than $\frac{r}{2}$ nodes agree. The probability that tampering by malicious nodes succeeds is:

$$P(\text{TamperSuccess}) = \begin{cases} 0, & |M| \leq \frac{r}{2} \\ 1, & |M| > \frac{r}{2} \end{cases}$$

This shows that as long as the majority of nodes are honest, isolated manipulation cannot influence the result. The result is valid only if:

$$\text{Valid}(Q) \iff \left|\{n \in Q : n.\text{response} = v\}\right| > \tfrac{r}{2}$$

where Q is the set of responses collected for a query.

Randomised node assignment and replication make it statistically unlikely for an attacker to control a majority of nodes in any chunk. Any detected inconsistency triggers retries, consensus-based conflict resolution $\mathcal{C}_i^{\text{cons}}$, or fallback to traditional DNS:

$$\text{Resolve}(k) = \begin{cases} v, & \text{if Valid}(Q) \\ \text{Retry or fallback}, & \text{otherwise} \end{cases}$$

Combined with cryptographic verification of signatures and distributed validation, this mechanism ensures that the integrity of responses is not compromised and clients receive accurate DNS records even under adversarial conditions.

5.6.3 Security Considerations

Traditional DNS is vulnerable to attacks such as cache poisoning, forged responses, and replay attacks, largely due to its reliance on centralised resolvers with limited validation. The proposed architecture improves security by decentralising trust and ensuring consensus for every DNS record update. All updates, whether triggered by record expiration or retrieved via fallback to traditional DNS, must undergo validation through a chunk-level blockchain consensus protocol $\mathcal{C}_i^{\text{cons}}$ among the peers responsible for that chunk.

An update transaction T is accepted only if a sufficient number of peers $k \geq \frac{|C_i|}{2} + 1$ validate it according to predefined rules, including record format, TTL validity, and digital signatures. If consensus fails, the update is discarded, preventing a single malicious or compromised node from corrupting data. We define R as a DNS record stored in chunk C_i and $\mathcal{M} \subset C_i$ as the set of malicious nodes. The probability P that a malicious update is accepted is bounded by:

$$P(T_{\text{malicious accepted}}) = \begin{cases} 0, & |\mathcal{M}| < \frac{|C_i|}{2} \\ \epsilon, & |\mathcal{M}| \geq \frac{|C_i|}{2} \text{ (negligible if digital signatures are required)} \end{cases}$$

This shows that as long as a majority of nodes are honest, malicious updates are rejected with high probability. The system also prevents injection, replay, and double-spending attacks because each transaction is uniquely signed, timestamped, and recorded on the blockchain.

The combination of distributed storage, cryptographic verification, and mandatory consensus removes single points of failure and ensures that no record can be altered without agreement from the majority of responsible peers. Formally, the integrity of a record R is guaranteed if:

$$\forall R \in C_i, \quad \text{Integrity}(R) \iff \mathcal{C}_i^{\text{cons}}(T_{\text{update}}) = \text{valid}$$

Even in the presence of malicious nodes, this approach maintains the integrity, authenticity, and availability of DNS records across the network, providing provable resilience against common DNS attacks.

5.6.4 Lack of Transparency and Control

Traditional DNS services are not transparent; clients do not know the path taken to resolve the DNS query or which DNS servers were involved in the resolution process. Traditional DNS also suffers from response deception, with possible incorrect or stale responses caused by misconfigured or malicious DNS servers. Clients also cannot verify DNS server liveness, which limits the ability to detect unresponsive or malicious resolvers.

In the proposed model, each node n_i independently handles resolution, selects peers for a chunk C_j, and verifies returned data using a consensus protocol $\mathcal{C}_j^{\text{cons}}$. We define R_k as the set of responses from all responsible peers for record k. A response v is accepted if and only if:

$$\left|\{n_i \in C_j : n_i.\text{response}(k) = v\}\right| > \frac{|C_j|}{2}$$

This ensures that the result reflects the majority agreement, providing a formally verifiable guarantee that the response cannot be controlled by a minority of compromised nodes.

The system guarantees correct resolution if the fraction f of malicious nodes in a chunk:

$$f < \frac{1}{2} \implies \text{ValidResponse}(k)$$

Clients can identify which nodes produced the response, since each node performs independent queries to multiple peers and validates their responses. Node liveness can be determined by measuring response times, which allows timely identification of unresponsive or possibly hijacked nodes.

The architecture achieves transparency, control, and formally provable integrity of DNS responses via the interaction of distributed query formulation, peer consensus-based validation, and independent verification.

5.7 Chapter Summary

We explored the limitations of traditional DNS and clarified how our alternative architecture can overcome them. By storing DNS records in a P2P network, there are no single points of failure, further reducing exposure to DoS attacks. Consensus is required when updating records at chunk level, allowing for safe and effective responses, without a single malicious or compromised node altering or corrupting the data. Query results must reach majority agreement among responsible peers in order to be accepted, therefore providing strong defenses against censorship, tampering, and coordinated attacks. Each node in the P2P network independently validates responses from its peers, as well as their liveness, which helps ensure transparency, data integrity, and accountability. Using dynamic routing through the Chord DHT, and replicating chunks, the service remains available with high utility even in the event of network partitions or failures among some nodes.

Table 5.1 summarises the security and reliability improvements of the proposed system compared to traditional DNS. The decentralised design reduces single points of failure, mitigates tampering via majority voting, and increases censorship resistance through distributed consensus. Transparency is improved because each node independently verifies responses, and availability under network partition is enhanced through DHT based routing and replication.

Property	**Traditional DNS**	**Proposed System**
Single Point of Failure	High	Low
Vulnerability to Tampering	High	Low (majority voting)
Censorship Resistance	Low	High (distributed consensus)
Transparency	Low	High (independent verification)
Availability under Partition	Moderate	High (replication + DHT)

TABLE 5.1: Comparison of Traditional DNS vs. Proposed Decentralised DNS

Formally, the correctness of query resolution is guaranteed as long as the fraction f of malicious nodes in any chunk satisfies $f < \frac{1}{2}$. This bound ensures that majority-based validation provides provable integrity, authenticity, and availability guarantees, even

under adversarial conditions. The combination of distributed storage, cryptographic verification, replication, and consensus makes the proposed system a robust, fault-tolerant alternative to traditional DNS.

Chapter 6

Practical Proof of Space Construction

The proposed PoSpace protocol is designed to address the inefficiencies and attack vectors found in current implementations. Our goal is to reduce processing time and resource usage while ensuring security protection. As shown in the sequential diagram in Figure 6.1, the protocol relies on two components, following the definition of PoSpace as explained in Section 2.3.3: the Prover P and the Verifier V. The Prover generates a storage proof (proof of space) by committing a fixed amount of disk storage and writing structured data into a Merkle tree. This process involves generating deterministic data segments and storing their hashes in a hierarchical structure that supports efficient proof generation. The Verifier issues a challenge consisting of specific criteria. To respond, the Prover retrieves the corresponding data segments and forms a simple proof (Merkle path). The Verifier then checks the proof's validity using the Merkle root and challenge values, without needing access to the full dataset. This interaction confirms that the Prover continues to store the original data and prevents shortcuts or regeneration attacks.

The overall setup time associated with PoSpace is typically high, because of the initialisation phase, which executes complex cryptographic computations. Although these operations, such as large-scale sorting and encoding, are not computationally intensive individually, their collective effect results in high CPU, memory, and disk I/O usage over time (90). The design complexity is intentional, since it helps prevent regeneration attacks while providing fairness among participants. In this chapter, we describe the proof structure, the challenge-response mechanism, and the optimisations applied to reduce setup time while maintaining the security properties.

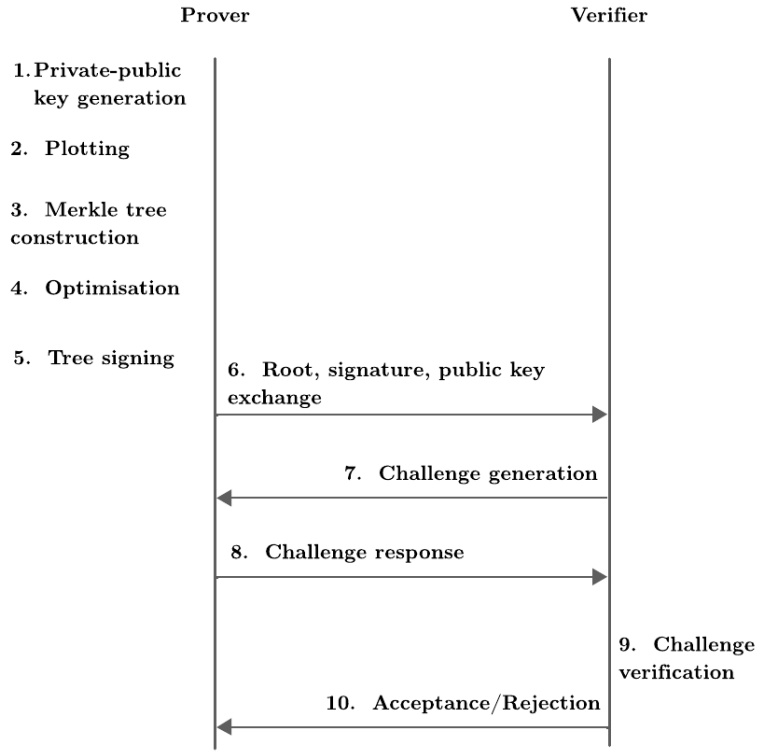


FIGURE 6.1: The PoSpace challenge–response interaction between the Prover and the Verifier.

6.1 Initialisation Phase

In the proposed PoSpace protocol, the initialisation phase is designed to be straightforward yet effective, providing sequential hashing to commit storage space. The goal of the data generation (plotting) is to generate a file F of a fixed size (e.g., 100 GiB) by sequentially generating and storing cryptographic hashes as shown in Algorithm 4. This file serves as a verifiable commitment to the allocated storage space.

6.1.1 Key Generation

The plotting phase is demonstrated in Algorithm 4 and begins with the Prover generating a public–private key pair (pk, k) (91) using Elliptic Curve Cryptography (ECC) (92), which serves as an input to the plotting process. Given the vast space of possible keys (2^{256} for a 256-bit key), we consider the probability of generating a key that already exists in the system negligible. This ensures that each Prover's commitment is distinct and prevents conflicts or overlaps in the plotted space, maintaining the integrity of the PoSpace protocol.

6.1.2 Data Generation (Plotting)

We define the initial block as $h(0)$, and it is computed using the cryptographic hash function H:

$$h(0) = H(\gamma \parallel k)$$

where H is a SHA-256 cryptographic hash function with a fixed output size of 256 bits (32 bytes), and $\parallel$ indicates concatenation of an initial constant variable γ and the value of k.

6.1.3 Plotting Through Iterative Hashing

During this phase, the Prover generates a sequence of hashes iteratively. Each hash h_i depends on the previous hash h_{i-1} and the private key k. This process is defined as:

$$h_i = H(h_{i-1} \parallel k), \quad \text{for } i = 1, 2, \ldots, N-1,$$

where H is the same hash function as before, and N is the total number of hashes determined by the Prover's allocated storage space. The sequence of hashes $\{h_0, h_1, \ldots, h_{N-1}\}$ is stored in F, with each hash occupying 256 bits (32 bytes).

The size of F is directly determined by the number of hashes N, calculated as:

$$N = \frac{S}{\text{Hash size}} = \frac{S}{32 \text{ bytes}},$$

where S is the total storage size allocated by the Prover. Thus, for a file size of $S = 100$ GiB:

$$N = \frac{100 \times 2^{30}}{32} = 3,355,443,200 \text{ hashes}.$$

The file F, consisting of N hashes, is stored on disk and represents the Prover's storage commitment. This cryptographic structure ensures the Prover has dedicated the declared space, as any attempt to regenerate or forge F would require recomputing the entire sequence of hashes.

6.1.4 Merkle Tree Construction

After generating F, the next step in the initialisation process is to construct the Merkle tree that represents F. Since each node in the Merkle tree stores a cryptographic hash of size 32 bytes, the total size of the Merkle tree is:

$$\text{Merkle tree size} = (2N - 1) \times 32\,\text{bytes}.$$

In comparison, the size of the original file F is:

$$\text{File size} = N \times 32\,\text{bytes}.$$

This shows that the Merkle tree is approximately twice the size of the file F.

6.1.5 Optimization

If the Prover were to retain both the original file F and the Merkle tree generated from F, the total storage requirement would be the sum of the size of F and the size of the Merkle tree. Let us assume that the initial storage requirement is 200 GiB. If the file F is 100 GiB, the total storage needed would be 100 GiB for the file plus an additional 200 GiB for the Merkle tree (as it is typically about twice the size of F), resulting in a total of 300 GiB. Since the initial storage requirement is only 200 GiB, this exceeds the available space. To optimise storage, the Prover deletes the original file F after constructing the Merkle tree and retains only the tree. Since the Merkle tree contains all the hashes from F as its leaf nodes and includes additional internal nodes to enable proof verification, it serves as a complete and verifiable representation of the data. This allows the Prover to meet the space commitment while still being able to respond to any challenges posed by the Verifier.

6.1.6 Tree Signing

The final step is tree signing. After the Prover has produced the Merkle tree, signs the Merkle root using their private key as proof of ownership of the Merkle tree (93). The Prover then sends the root, signature, and corresponding public key to the Verifier. The Verifier verifies the signature of the Merkle root with the public key, which confirms that the Merkle tree belongs only to this specific Prover and not to someone else attempting to redistribute their tree.

The Verifier stores the root and the public key in a file that is distributed across the entire network through a DHT. The DHT acts as a decentralised storage mechanism

where Merkle root–public key pairs are retained on different nodes within the network, ensuring fault tolerance and high availability (94). The Verifier and other participants can therefore access Merkle roots efficiently using the public key, even in a distributed and dynamic environment (95). The public key is an important resource for verifying the validity of the Merkle root and ensuring that it was produced by the Prover.

Algorithm 4 Initialisation Phase

```
1: procedure INITIALISATION
2:     (pk, k) ← GenerateKeyPair()
3:     h(0) ← H(γ ‖ k)
4:     for i = 1 to N − 1 do
5:         h(i) ← H(h(i − 1) ‖ k)
6:     end for
7:     F ← StoreHashes(h)
8:     merkle_tree ← CreateMerkleTree(F)
9:     Delete(F)
10:    σ ← Sign(k, merkle_tree.root)
11:    Send(merkle_tree.root, σ, pk)
12:    if VerifySignature(σ, merkle_tree.root, pk) then
13:        StoreInDHT(merkle_tree.root, pk)
14:    else
15:        RejectCommitment()
16:    end if
17: end procedure
```

6.2 Verification Phase

In the verification phase, the Verifier ensures that the Prover has correctly stored the data corresponding to their unique file F by issuing a challenge. The solution to this challenge is provided by the Prover through a proof derived from the Merkle tree, which efficiently demonstrates the correctness of the stored data. The challenge process involves generating a Merkle path for a randomly selected element of the Prover's Merkle tree, as shown in Algorithm 5.

6.2.1 Challenge Generation

The Verifier randomly selects a leaf node index i from the Prover's Merkle tree. Each leaf corresponds to the SHA-256 hash of a specific data block in the Prover's unique file F. This random selection ensures that the Prover cannot predict in advance which part of the Merkle tree will be referenced. Once the index is chosen, the Verifier issues a request to the Prover for the Merkle path associated with the leaf at position i. The

Merkle path is the sequence of sibling hashes stored along the branches of the tree that connect the selected leaf to the Merkle root. By providing this path, the Prover enables the Verifier to confirm the position and linkage of the chosen leaf within the tree.

6.2.2 Challenge Verification

Once the Prover calculates the Merkle path for the selected leaf index, it sends this path to the Verifier. To verify the Merkle path, the Verifier iteratively reconstructs the Merkle root by applying the hash function to the current hash and its corresponding sibling hash in the Merkle path, as shown in Figure 6.2. Specifically, for each step j (where $j \in \{1, 2, \ldots, N-1\}$, with N being the number of hashes in the tree), the Verifier computes:

$$\text{Hash}_j = H(\text{Hash}_{j-1} \parallel \text{SiblingHash}_j)$$

where H indicates the cryptographic hash function (SHA-256 in this case), and $\parallel$ denotes concatenation.

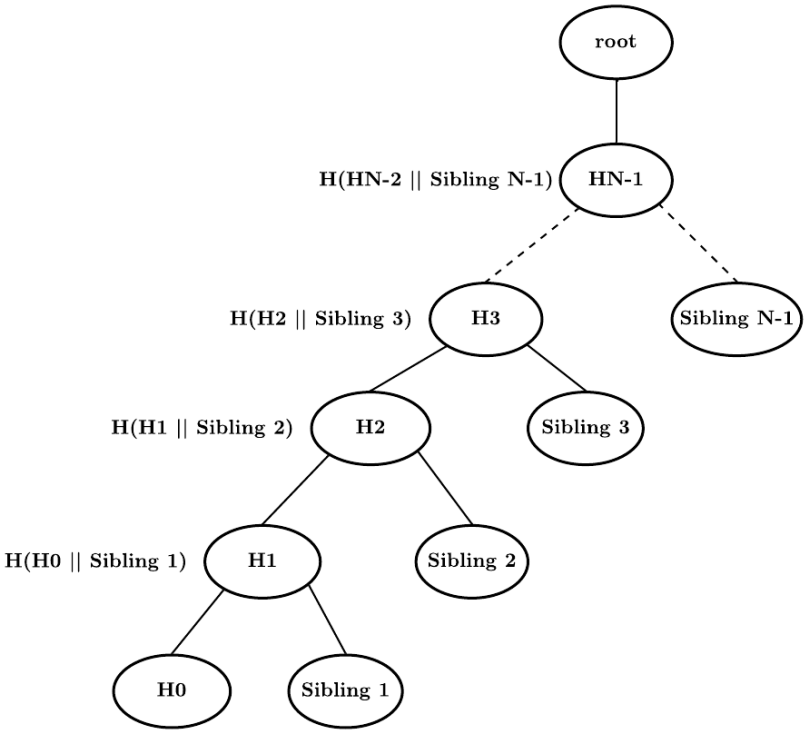


FIGURE 6.2: Reconstruction of the Merkle root based on the Prover's challenge response.

Formally, the Prover provides the leaf hash $\text{Hash}_0 = H(\text{block}_i)$ together with the sequence of sibling hashes $\{\text{Sibling Hash}_1, \text{Sibling Hash}_2, \ldots, \text{Sibling Hash}_{N-1}\}$. The Verifier then performs the following computations:

$$\begin{aligned}
\text{Hash}_1 &= H(\text{Hash}_0 \parallel \text{Sibling Hash}_1) \\
\text{Hash}_2 &= H(\text{Hash}_1 \parallel \text{Sibling Hash}_2) \\
\text{Hash}_3 &= H(\text{Hash}_2 \parallel \text{Sibling Hash}_3) \\
&\vdots \\
\text{Hash}_{N-1} &= H(\text{Hash}_{N-2} \parallel \text{Sibling Hash}_{N-1})
\end{aligned}$$

The final value Hash_{N-1} is the Merkle root. This value must match the Merkle root that the Prover originally committed to. The process is repeated for each level of the Merkle tree until the root is reconstructed (96). If the computed root matches the Prover's signed Merkle root, the proof is considered valid; otherwise, it is rejected.

Algorithm 5 Verification Phase

```
1:  procedure GENERATECHALLENGE
2:      i ← RandomLeafIndex(N)
3:      L ← F[i]
4:      merkle_path ← RequestMerklePath(i)
5:      Send(merkle_path)
6:  end procedure
7:  procedure VERIFYCHALLENGE
8:      L ← ReceivedLeaf
9:      merkle_path ← ReceivedMerklePath
10:     root ← L
11:     for j = 1 to N − 1 do
12:         root ← H(root ‖ Sibling Hash_j)
13:     end for
14:     if root == Prover's Merkle Root then
15:         AcceptProof()
16:     else
17:         RejectProof()
18:     end if
19: end procedure
```

6.2.3 Periodic Storage Verification

After the Prover has successfully responded to the Verifier's challenges and accepted to join the network, they eventually become eligible to participate in the consensus process. Periodic validation ensures that the Prover still holds the committed storage.

This verification occurs regularly at fixed intervals, with new PoSpace proofs required to demonstrate that the data have not been altered or discarded. The periodic nature of this verification ensures that the Prover acts rationally, meaning that they have not deleted or replaced the data and still comply with the storage rules and requirements.

If the Prover fails to respond to a request during a periodic verification check, penalties are applied, such as exclusion from the network. Periodic verification enables honest participants to remain in the network long term while ensuring that dishonest participants do not engage in fraudulent activities such as data regeneration or deletion.

6.3 Security Evaluation

In Chapter 4, we defined the threat model of the PoSpace scheme and identified a series of potential attacks a malicious Prover might attempt, including fake storage commitments, on demand computation, and Sybil attacks via multiple identities. In this section, we evaluate the security of the proposed PoSpace protocol against these threats. By listing the failure conditions for each attack, we prove that the protocol is resilient to these malicious behaviours. Each of the following subsections presents a proof that the proposed PoSpace protocol's design effectively mitigates the risks introduced by these attacks, ensuring its robustness and reliability.

6.3.1 Fake Storage Commitment (Amortisation Attack)

For two different Provers P_i and P_j, their datasets are distinct due to the uniqueness of their private keys. Specifically, for each Prover, the initial hash values and resulting hashes are different, while hash collisions occur with negligible probability:

$$k_i \neq k_j \quad \Rightarrow \quad h_0^{(i)} \neq h_0^{(j)}$$

$$\forall j, \quad h_j^{(i)} \neq h_j^{(j)}$$

Since the dataset F_i is used to construct the Merkle tree T_i, and the Merkle root R_i is derived from the dataset, it follows that:

$$T_i \neq T_j \quad \Rightarrow \quad R_i \neq R_j$$

Thus, even if a malicious Prover tries to share a dataset, the Merkle root is still uniquely associated with the original Prover's private key, and they cannot successfully perform amortization attacks. Because of the cryptographic hash functions used in the scheme, if a dataset is shared, then the correct Merkle root will be distinct for different Provers. This mechanism guarantees soundness because only a Prover with the correct data will generate a valid proof for their particular private key and Merkle root.

6.3.2 On Demand Computation (Rationality Attack)

We define the time required to retrieve a Merkle path from storage as t_{store}, and the time required to recompute the Merkle tree and generate the required path as $t_{\text{recompute}}$. For a Prover to succeed in this attack, the following condition must be satisfied:

$$t_{\text{recompute}} \leq t_{\text{store}}$$

The time complexity for Merkle tree construction is:

$$t_{\text{recompute}} = O(N \log N)$$

where N is the number of leaves in the Merkle tree. The time to retrieve a Merkle path from storage is:

$$t_{\text{store}} = O(\log N)$$

Since $O(N \log N) \gg O(\log N)$ for large N, we can conclude that:

$$t_{\text{recompute}} \gg t_{\text{store}}$$

Storing a Merkle tree is always faster than recomputing a Merkle tree on demand. Moreover, even if the adversary generates the file and recomputes the Merkle tree, the cost of doing so is still greater than that of simply retrieving the Merkle path from storage. This makes the rationality attack impractical.

Theorem 6.1 (Soundness against Rationality Attacks)**.** *The proposed PoSpace protocol is sound if a Prover $\mathcal{P}$ can pass the VERIFY_CHALLENGE procedure with non-negligible probability only by possessing the stored dataset F. For any storage bound N where $N \gg 1$,*

the protocol ensures that the time for on-demand recomputation $t_{recompute}$ is strictly greater than the time for path retrieval t_{store} plus a network-defined timeout threshold δ.

Proof. 1. Let t_{store} be the complexity of retrieving a Merkle path, which is $O(\log N)$.

2. Let $t_{\text{recompute}}$ be the complexity of regenerating the hash sequence and reconstructing the Merkle tree, which is $O(N \log N)$.

3. For $N = 3{,}355{,}443{,}200$ (representing approximately 100 GiB of commitment), the ratio
$$\frac{t_{\text{recompute}}}{t_{\text{store}}} \approx N.$$

4. Enforce a response timeout δ such that
$$t_{\text{store}} < \delta < t_{\text{recompute}}.$$
Any adversary $\mathcal{A}$ attempting on-demand recomputation will fail to return a valid proof within this time bound.

5. Therefore, the probability that an honest Verifier $\mathcal{V}$ accepts a recomputed proof is negligible.

□

The Prover has no chance of recomputing the Merkle tree fast enough to meet the Verifier's response time and thus will ultimately fail; therefore, the PoSpace protocol is sound because only honest Provers can generate a valid proof having stored the file data. In addition, the computational cost of Merkle tree recomputation guarantees that malicious Provers will not be able to produce valid proofs without having stored the requisite data.

6.3.3 Sybil Attack via Multiple Identities

Each Prover is required to sign the Merkle root R_i using a private key k_i before submitting it to the network:

$$\sigma_i = \text{Sign}(R_i, k_i)$$

where σ_i is the digital signature and Sign is a cryptographic signing function. The Verifier checks the signature:

$$\text{Verify}(\sigma_i, R_i, PK_i) = 1$$

where PK_i is the public key corresponding to k_i. Since cryptographic signatures are unique to each Prover, registering multiple identities requires generating distinct key pairs $(k_1, PK_1), (k_2, PK_2), \ldots$, each tied to a valid Merkle root R_i. If a Prover attempts to reuse storage for multiple identities, they must satisfy the condition:

$$R_1 = R_2 = \cdots = R_m$$

which conflicts with the uniqueness rules defined by key-dependent hashing. Each Prover is required to have their own private key that is linked to the Merkle root. This leaves no possibility of a Sybil attack, since each identity is linked to unique data and signatures. As the system uses digital signatures that are tied to unique public keys, it prevents malicious Provers from creating numerous identities while keeping the storage requirements at a very low level.

6.4 Performance Evaluation

To assess the performance of the proposed PoSpace protocol, we compared it against the Chia PoSpace protocol, as it is a widely known and well-established protocol in this field, and serves as a suitable point of reference for assessing improvements in scalability and efficiency. Chia has over 115,000 active nodes, as of its April report, and is a recognised decentralised network that serves as a global example of large scale implementation of PoSpace, making it an appropriate comparison for evaluating the performance of our proposed solution (97).

The analysis related to the performance evaluation was carried out on a computer running Windows 10 Enterprise. We conducted three trial evaluations of the experiments for both protocols to maintain a fair and consistent evaluation between trials for each protocol. The hardware was configured with a 3.2 GHz 4-core/8-thread CPU, a 500 GiB SSD with read and write rates of 1200 MB/s, a 200 GiB HDD with read and write rates of 200 MB/s, and 16 GiB of memory. We implemented our protocol in Python 3 because of its wide range of libraries for cryptographic operations, ease of development, and flexibility to focus on the core protocol logic and have reliable performance on multiple different hardware configurations (98).

We summarised our findings in Table 6.1, which presents the average time for each phase/operation across the three trials for both protocols.

Our calculations do not include the time for the synchronisation of the blockchain, as it does not occur during the initialisation step of the PoSpace protocol. The synchronisation process involves downloading the complete or partial chain history and verifying that every block reaches consensus with the current state of the

Algorithm	Phase/Operation	Average Time
Chia PoS	Forward propagation	7h 22m
	Backpropagation	4h 18m
	Compression	2h 7m
	Final writing	57m
	Total time	**14h 44m**
Proposed PoS	CPU Time (Hashing)	50m 41s
	I/O Time	12m 6s
	Memory Allocation	6m 12s
	Loop Overhead	20m 15s
	Idle/OS Scheduling	9m 36s
	Total time	**1h 38m**

TABLE 6.1: Chia and proposed PoSpace initialisation performance.

network. This process is applicable to most blockchain environments and is handled by the core client software prior to any consensus operation (99). Thus, we consider synchronisation a separate process and do not include it when estimating the PoSpace initialisation cost.

6.4.1 Experimental Setup: Chia Network Implementation

Most experiments were performed using the official Chia installer, which is a straightforward and reliable way to install the software when no changes to the functionality's internal operation were needed.

We installed Chia in the normal way: we downloaded the installer `https://www.chia.net/download/`, executed it on our test machine, and completed the setup process. The installer handled all dependencies and other setup procedures and deployed the software to eventually connect it with the Chia blockchain.

After installing the software, we tested its usability (verified that it was functional) by starting the Chia application. For this step, we chose the installer to reduce setup time and limit the chances of setup errors, making it suitable for experiments where custom builds were not required.

Running the Blockchain Node

Once installed, the next step was to operate the blockchain node and connect it to the network. We used the command-line interface rather than the graphical interface, as it provided finer control and supported automation for experimental tasks.

The node was started using:

```
chia start node
```

This command launched the client and started peer discovery as well as the synchronisation of the blockchain. The node began downloading and verifying the history of the blockchain, which is a prerequisite for participating in the network. This first-time synchronisation was very slow but necessary to ensure the node reached a consistent state. However, (as noted in Section 6.4), the time for synchronising the blockchain is not included in our results as it does not occur during the initialisation phase of the PoSpace.

Node status was monitored with:

```
chia show -s
```

This command reported synchronisation progress, peer connections, and overall operational status. Monitoring was especially useful during the initial synchronisation phase, where network and disk performance significantly affected progress.

When maintenance or configuration changes were required, the node was stopped using:

```
chia stop node
```

This ensured a clean shutdown without data corruption.

Creating Plots and Starting to Farm

To participate in Chia's PoSpace consensus, storage capacity must be allocated through the creation of *plots*. Plots are large files containing cryptographic structures that serve as proofs of committed disk space.

A standard plot was created using:

```
chia plots create -k 32 -n 1
```

The parameter `-k 32` sets the plot size, while `-n 1` specifies the number of plots to produce in a single invocation. The `k=32` size is the default and recommended by the Chia protocol, and produces a plot that is approximately 101.4 GiB. For the purpose of our experiments, we limited the process to a single plot (`n=1`) to better estimate overall resource expenses and to keep nominal overhead time associated with plotting minimal. As discussed in section 6.4.2, the plotting is relatively demanding and time-consuming; typically takes a few hours depending on other hardware specifications such as CPU speed, memory bandwidth, and storage performance.

Farming was then initiated with:

```
chia start farmer
```

This process continuously monitored the blockchain for any challenges and checked local plots for valid proofs. Farming operated in the background and required few resources.

Running both the node and the farmer created a complete environment with the ability to fully participate in the Chia network. In this environment, we observed PoSpace in action and collected experimental data on system performance and network behaviour.

6.4.2 Baseline: Chia PoSpace Performance Evaluation

We monitored the Chia PoSpace plotting process using Chia's official CLI commands (100), which provide detailed performance statistics for every phase of the plotting process (101). These allowed us to measure elapsed time for stages associated with Forward Propagation, Backpropagation, Compression, and Final Write (to file). Each of the plotting phases represents distinct computational work, thus providing an indication of the relative workload and time distribution. We conducted three trials with consistent hardware and configuration conditions to evaluate the accuracy and variability of the plots. The results collected, including stage by stage performance performances, and total plotting time, are summarised in Table 6.2, providing a detailed view of the plotting workload properties.

Phase	**Trial 1**	**Trial 2**	**Trial 3**
Forward propagation	7h 15m	7h 28m	7h 22m
Backpropagation	4h 25m	4h 16m	4h 12m
Compression	2h 8m	2h 6m	2h 9m
Final writing	44m	1h 6m	1h 1m
Total time	**14h 32m**	**14h 56m**	**14h 44m**

TABLE 6.2: Chia PoSpace initialisation phase performance.

As shown in Figure 6.3, the data show minimal changes. The Forward Propagation phase remains stable during these trials; Trial 1 performed slightly faster than Trial 2. The Backpropagation phase saw a consistent decrease in time, with Trial 3 being the fastest, indicating possible improvement over time. The Compression phase is nearly constant, though Trial 2 is slightly faster. The Final Writing phase was the most unpredictable part of the testing. Trial 1 took 44 minutes, while Trials 2 and 3 ran for around an hour, indicating possible differences in disk performance or system load.

In general, there is limited time variation across the total time differences, with Trial 2 being the longest, likely associated with the additional time spent in both Forward

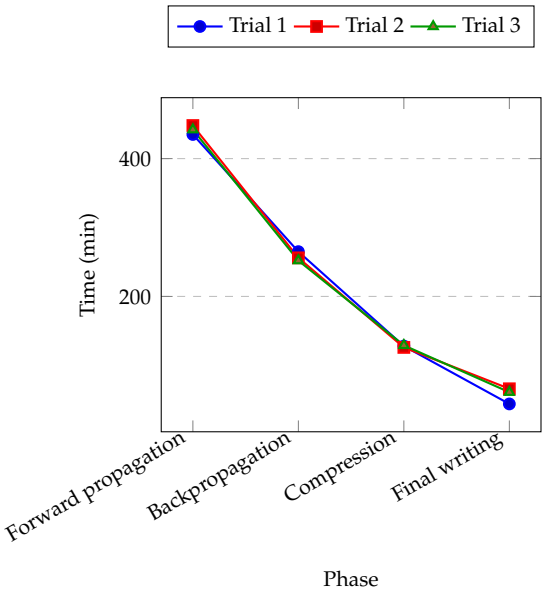


FIGURE 6.3: Chia PoSpace initialisation phase timings across trials.

Propagation and Final Writing. The variability in time difference suggests that even the process is relatively stable, some phases may vary. More specifically, writing and propagation times may differ based on system conditions and trial specific circumstances. For example, writing time may be affected by disk speed or system load, while propagation time can be impacted due to network connectivity or short-term hardware fluctuations.

6.4.3 Proposed PoSpace Protocol Performance Evaluation

To assess our protocol's performance, we ran experiments to obtain all key metrics contributing to the overall time of the process: computing time, memory usage, disk I/O, and resource consumption. The protocol specifically attempts to manage the key phases of PoSpace initialisation efficiently by reducing overhead while improving scalability. The main parts of the experiment are as follows:

CPU time (hashing): This measures the time the CPU spends generating the SHA-256 hashes. The CPU computes each hash by concatenating the key with the previous hash and performing the SHA-256 hashing operation.

I/O time (writing): This measures the time required for the system to write each generated hash to disk. As the hashes are computed, they are sequentially written to the file, which can be slower than the CPU's processing time.

Memory allocation: The experiment uses memory to store the key, hashes, and data during each iteration. This metric captures the time spent by the system in managing these memory resources.

Loop overhead: This represents the time spent on control operations, such as checking the loop condition (whether the total number of bytes written is less than the target file size) and incrementing the counter for each hash written.

Idle/OS scheduling: This reflects the time the CPU is inactive or waiting for the operating system to schedule tasks, such as waiting for I/O operations to complete or for other resources to become available.

The results, including the breakdown of time spent in different stages, are presented in Table 6.3, providing a clear overview of the efficiency of the initialisation process.

Operation	**Trial 1**	**Trial 2**	**Trial 3**
CPU time (hashing)	49m 47s	51m 48s	50m 27s
I/O time (writing)	11m 37s	12m 54s	11m 47s
Memory allocation	5m 35s	6m 32s	6m 29s
Loop overhead	19m 35s	21m 6s	20m 4s
Idle/OS scheduling	8m 40s	10m 6s	10m 1s
Total time	**1h 35m**	**1h 42m**	**1h 38m**

TABLE 6.3: Proposed PoSpace initialisation phase performance.

By collecting these metrics, we can determine the contribution of each factor in the time taken by the initialisation phase. Elapsed time for the entire operation is calculated by subtracting the start time from the end time of each phase. This lets us separate the performance constraints, and also prepares us to identify potential ways of optimising.

Figure 6.4 shows minor performance differences across the three trials. CPU time is consistent and stable, while I/O and memory usage are increased slightly from Trial 1 to Trial 2. Idle/OS time also increased (Trial 2 and Trial 3), showing possible scheduling delays. Overall, the total execution time did not vary much, with Trial 2 taking the most time to execute and Trial 1 being the fastest of all.

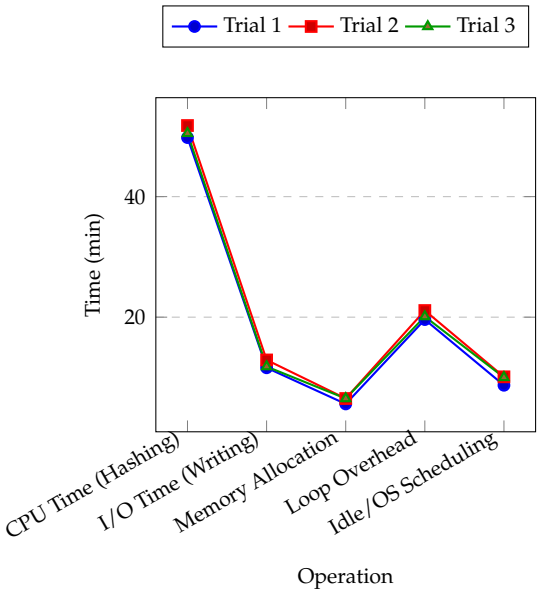


FIGURE 6.4: Proposed PoSpace initialisation resource breakdown across trials.

Overall, the results show that the system's performance is consistent, with small fluctuations likely due to minor system factors rather than substantive changes in the process itself.

6.4.4 Comparative Security Analysis: Chia vs. Proposed PoSpace

While the proposed PoSpace protocol achieves a 90% reduction in initialisation time, it is necessary to formally compare its security properties against the industry-standard Chia Network implementation. The primary trade-off involves the recomputation threshold. The high initialisation cost in the Chia Network serves as a deterrent against rationality attacks, where a node recomputes data on demand instead of storing it. In contrast, our protocol prioritises accessibility for resource-constrained devices. To maintain security, we enforce strict response-time validation such that $t_{\text{store}} \ll t_{\text{recompute}}$, ensuring that efficiency gains do not compromise Sybil resistance.

Property	**Chia Network (Baseline)**	**Proposed PoSpace**
Initialisation Cost	Extremely high (14.73 hours avg.)	Low (1.64 hours avg.)
Sybil Resistance	Bound by physical disk capacity	Bound by key-dependent iterative hashing
Regeneration Risk	Negligible due to plotting complexity	Managed via strict temporal thresholds (δ)
Target Devices	Enterprise-grade storage/CPU	Heterogeneous (Edge/Mobile)

TABLE 6.4: Security and operational trade-offs between Chia and the proposed framework.

6.5 System-Wide Performance and Malicious Scenario Analysis

To address the requirement for an evaluation of the framework beyond the initialisation phase, this section presents a theoretical simulation of the live system's performance. This analysis integrates the experimental PoSpace results from Section 6.4 with established P2P performance metrics under clearly specified assumptions.

6.5.1 Latency and Scale-up Simulation

We model the total resolution latency L as a function of the number of nodes N in the Chord ring and the average inter-node network latency τ. The average number of hops required to locate a DNS record chunk is defined as:

$$\frac{1}{2}\log_2 N.$$

Assumption: We assume a global network of $N = 1{,}000$ peers with an average inter-node latency $\tau = 50\,\text{ms}$.

Projection: Under these conditions, the average lookup latency is:

$$L \approx \left(\frac{1}{2}\log_2 1000\right) \times 50\,\text{ms} \approx 5 \times 50\,\text{ms} = 250\,\text{ms}.$$

This indicates that the system maintains sub-second resolution time even as the network scales, satisfying low-latency requirements.

6.5.2 Throughput and Resource Efficiency

The system's throughput is bounded by the efficiency of the PoSpace verification process. Based on the experimental results in Table 6.1, the proposed PoSpace protocol completes initialisation in approximately 1.64 hours.

Node Admission: A new node can become a fully functional validator in under two hours, significantly outperforming the Chia baseline of 14.7 hours.

Query Capacity: Because the resolution load is distributed uniformly across the ring using cryptographic hashing, the network's total query capacity increases linearly with each additional peer, preventing the single points of failure found in centralised recursive resolvers.

6.5.3 Malicious Scenario Analysis (Threat Model Validation)

We evaluate the framework's robustness against the threats identified in Chapter 4, focusing on Sybil Inflation and Consensus Tampering.

Integrity Guarantee: Each DNS record update requires majority agreement within a consortium of s successors. In a consortium of $s = 5$, the system remains secure if the number of malicious nodes satisfies:

$$|M| \leq 2.$$

Sybil Resistance: To mount a successful attack on a single chunk, an adversary must commit approximately 300 GiB of verifiable storage to control a majority of a 5-node consortium. This ties system security directly to the physical resource costs of the PoSpace protocol.

Eclipse Resistance: By maintaining a node degree of $d = \log_2 N$, the probability of an honest node being eclipsed by an adversary controlling a fraction f of the network is:

$$f^d.$$

For $f = 0.2$ and $N = 1000$ (so $d = 10$), this gives:

$$0.2^{10} \approx 1.02 \times 10^{-7},$$

providing strong statistical security.

6.6 Chapter Summary

Comparing the results of our protocol and Chia PoSpace indicates significant performance improvements, particularly in terms of total time and phase-specific efficiencies as shown in Figure 6.4. This difference is mainly due to the more efficient use of computational resources. For instance, our protocol's hash generation (Forward propagation phase for Chia) is much faster because of its optimised hashing protocol and minimal I/O overhead, which contrasts with Chia's longer propagation times, caused by its more resource intensive operations.

Final Write in our protocol is considerably quicker, with times being consistent and much shorter than Chia's. This improvement is due to more efficient disk I/O operations and less intensive memory management, which reduce the need for extensive system scheduling and idle time during the writing phase. In practice, this means nodes can go through the plotting process faster and consume less energy

while putting less strain on the storage used to make their commitments. These improvements make it more reasonable to use with multiple devices with different specifications, including equipment with limited memory or slower disks where the write demand of the Chia protocol is typically a bottleneck.

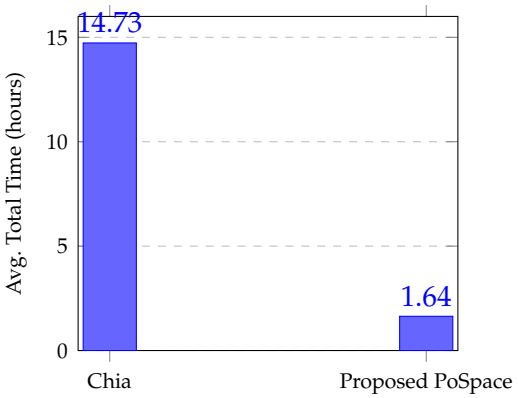


FIGURE 6.5: Average total initialisation time across trials (in hours)

In summary, the proposed protocol offers a more efficient solution compared to Chia, particularly due to its optimised hashing, lower I/O time, and efficient memory usage, which contribute to significantly reduced initialisation times. These improvements point to our more scalable architecture, designed to minimise resource contention and maximise throughput.

Chapter 7

Future Work

The idea of adopting a decentralised DNS resolver for achieving security optimisation represents a significant contribution to the field. The solution we propose aims to address current DNS security challenges and vulnerabilities by providing a secure and decentralised resolution mechanism capable of overcoming cyber attacks. The scope of the study includes identifying existing DNS security issues, evaluating the applicability of blockchain technology for DNS resolution, and designing a decentralised DNS resolver that improves transparency, resilience, and user control. In addition to the resolver design, this work introduces a custom PoSpace algorithm to support decentralised storage commitments in a scalable and efficient manner. This algorithm lets participants prove they are storing DNS records without needing trust or central servers.

Future work in this research focuses on two directions: improving the performance, security, and compatibility of the decentralised DNS resolver; and addressing risks in the PoSpace initialisation process, including fast regeneration, time-based proof validation, and fairness in resource usage.

7.1 Decentralised DNS Resolver

7.1.1 Performance Evaluation

While the decentralised DNS resolver meets its theoretical design goals, its performance in real-world conditions remains untested. Future work should evaluate key metrics such as system latency, resolution accuracy, query success rate, and throughput under realistic workloads. This includes deploying the resolver in controlled network environments, benchmarking against conventional DNS resolvers, and measuring behaviour at scale with large numbers of concurrent queries.

Simulations should also assess the resolver's capacity to tolerate dynamic conditions, including congestion on nodes, delays due to querying load, and incomplete network partitions. Testing against these adversarial conditions (e.g., targeted DoS or Sybil attacks) would show the level of its robustness in insecure environments. These evaluations should quantify the operational limits and identify bottlenecks that affect stability and responsiveness.

In addition, long running experiments are needed to assess the impact of resource consumption over time, including bandwidth usage, memory overhead, and response consistency. Such studies will determine whether the system can ensure reliable operation over extended periods and influence the design of potential optimisations for large-scale deployment.

7.1.2 Security and Robustness

Investigation into security in adversarial contexts is a significant area of future work. The resolver should be assessed under a variety of attack vectors, from DoS, to populations Sybil based identity inflation and targeted manipulation of responses to user queries. A crucial point is the inconsistent name resolution caused by malicious or compromised nodes, as this can undermine confidence in the system.

Evaluating the system under adversarial attack models will improve understanding of its level of resilience and will highlight weaknesses or the limits of non-redundant fault tolerance features of the system. For example, large scale Sybil experiments will measure the minimum cost required for an adversary to bias resolution results, while DoS experiments will provide insights into throughput and latency collapse under sustained attack.

Testing should also investigate the effectiveness of mitigation attempts including stronger consensus rules, adaptive rate limiting, node reputation systems, and redundancy of query resolution paths. Longer term studies should evaluate the implications of collusion between adversaries, as well as the ability of the system to recover from a partial compromise without requiring a wide network reset. This will identify feasible security measures and important protocol upgrades to enable effective deployment of the network in adversarial contexts.

7.1.3 Compatibility with Existing DNS Infrastructure

Future work should also assess interoperability with the current DNS ecosystem. This includes evaluating the behaviour of the decentralised resolver alongside traditional recursive resolvers, authoritative servers, domains with DNSSEC enabled, and caching strategies that are currently in use. These would provide controlled

experiments to assess whether a decentralised resolution approach introduces inconsistencies, increases response times in a mixed deployment, or conflicts with existing validation procedures.

A key direction is identifying points where the decentralised model diverges from legacy DNS assumptions, such as trust in hierarchical authorities, cache invalidation rules, and negative response handling. Understanding these variations will inform the design of supporting mechanisms or middleware that allow decentralised resolvers to integrate smoothly into the overall infrastructure without affecting security standards or breaking compatibility with existing clients.

Another important area is evaluating interoperability with middleboxes (e.g., firewalls, NATs, and DNS filters) that may include unknown assumptions about DNS behaviour. Long-term studies should also assess how decentralised resolution interacts with widely deployed applications and services, ensuring that integration does not disrupt user experience or impact system reliability.

7.2 Proof of Space Initialisation

7.2.1 Risk of Fast Regeneration

The proposed PoSpace protocol reduces initialisation time to improve efficiency. However, this optimisation introduces a critical risk: a Prover with sufficient computational resources may choose to repeatedly regenerate the dataset on demand rather than persistently storing it. This behaviour violates the security assumption of PoSpace, namely that long-term storage is economically preferable to recomputation.

In contrast, systems such as Chia intentionally implement long and costly setup times, making recomputation impractical and incentivising persistent storage. By allowing rapid initialisation, our protocol may inadvertently create an environment where non-rational or adversarial Provers bypass storage requirements without facing significant penalties. Similar concerns have been raised by Ren and Devadas (102), who demonstrate that unchecked high-speed computation can violate PoSpace standards by redirecting the balance away from storage costs.

Future work should systematically quantify the recomputation threshold by modelling the trade-off between hardware cost, energy consumption, and storage requirements. Experimental evaluation with different hardware configurations (e.g., GPU and FPGA acceleration) can help identify realistic attack vectors. Countermeasures may include introducing memory-hard functions, periodic audits that bind proofs to time intervals, or hybrid approaches that combine storage verification with computational challenges to raise the cost of regeneration attacks.

7.2.2 Time-Based Proof Validation

To address the issue of dataset regeneration, it would be useful for future work to explore dynamic time-based validation techniques. The idea is that the Prover's response time to given challenges could indicate whether the proof was retrieved from persistent storage or recalculated on demand by the Prover. This approach introduces a time dimension, unrelated to cost, that may discourage an adversary's complete reliance on computational power without penalties.

An example implementation could establish adaptive response-time thresholds based on the distribution of network latency and processing power during normal operations. Responses that are clearly faster or slower than expected could be considered as suspicious. For example, if a proof is returned too fast, the protocol could assume that this is an on-demand recomputation, in contrast to retrieval from persistent storage. Alternatively, if a response takes too long, this could be interpreted as a sign of strategic inactivity.

This concept builds upon difficulty-adjustment techniques used in PoW systems (103), and is further extended in PoST (78), which link storage to time to prevent shortcut strategies. Future work should define these timing constraints formally, examine their security guarantees under specific adversary models, and evaluate their practicality in diverse design approaches where systemic latency and processing capacities vary considerably.

7.2.3 Fairness and Resource Asymmetry

Response-time regulation also helps address broader fairness issues in PoSpace protocols. As Dembo et al. (84) emphasise, distributed systems often show notable differences in computational and storage resources across participants. Without safety measures, high performance adversaries with specialised hardware could regenerate data faster than honest nodes can retrieve it from storage, leading to centralisation and reduced inclusivity.

Setting a bound on proof acceptance through response times is a way to address this concern. The protocol requires proofs to match retrieval times expected from persistent storage and therefore prevents raw computing power from providing an unfair advantage. This shows that storage is the key resource being protected, rather than computation.

Other mechanisms, such as probabilistic auditing, proof-weighting schemes that enable resource diversity, and bandwidth or energy models with PoSpace, can also reduce systemic assumptions. Such changes would support the goal of decentralising

the system and maintaining reasonable equity in participation among nodes with different capabilities.

Chapter 8

Conclusions

In this research, we introduced a decentralised DNS resolution framework that seeks to mitigate risks and limitations identified in traditional DNS, including, but not limited to, centralisation, single points of failure, and lack of transparency in the resolution process. We proposed a distributed network of nodes linked by a consensus-based mechanism to update data and provide improved availability, security, and resilience to attacks. The architecture reduces the risk of cache poisoning and forged responses by requiring consensus to authorise updates and verification of responses amongst multiple peers. It also supports full visibility and control for nodes over the domain resolution process, verifying data integrity and providing a basis for detecting data tampering. Ultimately, the dynamic, verifiable mechanism of replication by design maximises fault tolerance, improves resilience to both network disruptions and censorship attacks, and results in a secure and censorship-resistant DNS framework that is needed for a better-performing and more open internet infrastructure.

In addition, this research introduces a practical PoSpace protocol designed to address key limitations found in existing implementations. Unlike current alternatives, the proposed PoSpace minimises CPU usage and delays, achieving the goal of replacing PoW with a lightweight, efficient mechanism. We defined a threat model specific to PoSpace systems and demonstrated our protocol's resistance to the identified attacks. The performance analysis indicates that our approach outperforms Chia's PoSpace in terms of speed and scalability.

The proposed decentralised DNS resolver and the improved PoSpace algorithm outline the foundation for a more secure, efficient, and resilient internet infrastructure. By identifying core vulnerabilities in both name resolution and decentralised storage, this research enables the practical deployment of scalable, trustless systems. Continued development and real-world testing will be key to achieving full adoption and maximising the security benefits of these designs.

Appendix A

Installing and Running the Proposed PoSpace

This appendix explains how to install and run the PoSpace prototype used in the experiments.

A.1 Requirements

- Python 3.8 or higher
- Git
- pip (Python package manager)

A.2 Installation

```
# Clone the repository
git clone https://github.com/georgegiam/rpos.git
cd rpos

# (Optional) create a virtual environment
python3 -m venv venv
source venv/bin/activate    # On Linux/macOS
venv\Scripts\activate       # On Windows

# Install dependencies
pip install -r requirements.txt
```

LISTING A.1: Cloning and installing dependencies

A.3 Running the PoSpace

The main script can be executed directly after installation.

```
python main.py
```

LISTING A.2: Running the proof-of-space

A.4 Verifying Installation

To confirm that the environment is set up correctly, run the script with the help option:

```
python main.py --help
```

Glossary

TCP: Transmission Control Protocol

IP: Internet Protocol

DNS: Domain Name System

ISP: Internet Service Provider

VoD: Video on Demand

RFC Request for Comments

DDoS: Distributed Denial of Service

CDN: Content Delivery Networks

IOT: Internet of Things

UDP: User Datagram Protocol

HTTP: Hypertext Transfer Protocol

HTTPS: Hypertext Transfer Protocol Secure

DoH: DNS over HTTPS

DoT: DNS over TLS

DANE: DNS-Based Authentication of Named Entities

CC: Command Control

DNSSEC: Domain Name System Security Extensions

MITM: Man-in-the-Middle

ARP: Address Resolution Protocol

DHT: Distributed Hash Table

P2P: Peer-to-Peer

PBFT: Practical Byzantine Fault Tolerance (

A: Address Record

AAAA: Quad-A Record

MX: Mail Exchanger Record

NS: Name Server Record

CNAME: Canonical Name Record

TLD: Top-level Domains

gTLD generic TLDs

ICANN: Internet Corporation for Assigned Names and Numbers

TTL: Time To Live

IXFR: Incremental Zone Transfer

AXFR: Asynchronous Full Transfer Zone

ccTLD: Country Code TLDs

GNSO: Generic Names Supporting Organization

ccNSO: Country Code Names Supporting Organization

GAC: Governmental Advisory Committee

SSAC: Security and Stability Advisory Committee

IANA: Internet Assigned Numbers Authority

CA: Certificate Authority

BDNS: Blockchain-based DNS

ENS: Ethereum Name Service

dApps: Decentralised Applications

IPFS: InterPlanetary File System

PKI: Public key Infrastructure

IoT: Internet of Things

DHT: Distributed Hash Table

DLT: Distributed Ledger Technology

VDF: Verifiable Delay Functions

CPU: Central Processing Unit

I/O: Input/Output

PoSpace: Proof of Space

PoSW: Proofs of SequentialWork

PoST: Proofs of Space-Time

SHA: Secure Hashing Algorithm

ECC: Elliptic Curve Cryptography

SSD: Solid State Drive

HDD: Hard Disk Drive

CLI: Command Line Interface

OS: Operating System

NAT: Network Address Translation

FPGA: Field-Programmable Gate Array

CPU: Central Processing Unit

References

[1] NS1. Analysis of 7.5 trillion dns queries reveals public resolvers dominate the internet, 2023. Accessed: 2025-04-17.

[2] Xfinity. Xfinity by comcast, 2023. Accessed: 2025-04-17.

[3] T-Mobile. T-mobile usa, 2023. Accessed: 2025-04-17.

[4] Paul Mockapetris. Domain names-concepts and facilities. Technical report, 1987.

[5] Paul V Mockapetris. Rfc1035: Domain names-implementation and specification, 1987.

[6] Chengxi Xu, Yunyi Zhang, Fan Shi, Hong Shan, Bingyang Guo, Yuwei Li, and Pengfei Xue. Measuring the centrality of dns infrastructure in the wild. *Applied Sciences*, 13(9):5739, 2023.

[7] Rashna Kumar, Sana Asif, Elise Lee, and Fabi'an E Bustamante. Third-party service dependencies and centralization around the world. *arXiv preprint arXiv:2111.12253*, 2021.

[8] Trinh Viet Doan, Justus Fries, and Vaibhav Bajpai. Evaluating public dns services in the wake of increasing centralization of dns. In *2021 IFIP Networking Conference (IFIP Networking)*, pages 1–9. IEEE, 2021.

[9] Hadrien Hours, Ernst Biersack, Patrick Loiseau, Alessandro Finamore, and Marco Mellia. A study of the impact of dns resolvers on cdn performance using a causal approach. *Computer Networks*, 109:200–210, 2016.

[10] Synthia Wang, Kyle MacMillan, Brennan Schaffner, Nick Feamster, and Marshini Chetty. Measuring the consolidation of dns and web hosting providers. *arXiv preprint arXiv:2110.15345*, 2021.

[11] Samantha Bates, John Bowers, Shane Greenstein, Jordi Weinstock, Yunhan Xu, and Jonathan Zittrain. Evidence of decreasing internet entropy: the lack of redundancy in dns resolution by major websites and services. Technical report, National Bureau of Economic Research, 2018.

[12] Wikipedia contributors. Tier 1 network — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Tier_1_network, 2025. [Online; accessed 9-September-2025].

[13] Lin Jin, Shuai Hao, Haining Wang, and Chase Cotton. Understanding the impact of encrypted dns on internet censorship. In *Proceedings of the Web Conference 2021*, pages 484–495, 2021.

[14] Angelique Medina. Inside the fastly outage: Analysis and lessons learned. ThousandEyes Blog, June 2021. Online; accessed 9-September-2025.

[15] Tom Strickx and Jeremy Hartman. Cloudflare outage on june 21, 2022. Cloudflare Blog, June 2022. Online; accessed 9-September-2025.

[16] Glenn Greenwald and Ewen MacAskill. Nsa prism program taps in to user data of apple, google and others. *The Guardian*, 2013.

[17] Timm Böttger, Felix Cuadrado, Gianni Antichi, Eder Leão Fernandes, Gareth Tyson, Ignacio Castro, and Steve Uhlig. An empirical study of the cost of dns-over-https. In *Proceedings of the Internet Measurement Conference*, pages 15–21, 2019.

[18] Zi Hu, Liang Zhu, John Heidemann, Allison Mankin, Duane Wessels, and Paul Hoffman. Specification for dns over transport layer security (tls). Technical report, 2016.

[19] Rashna Kumar and Fabián E Bustamante. Reclaiming privacy and performance over centralized dns. *arXiv preprint arXiv:2302.13274*, 2023.

[20] Cybersecurity and Infrastructure Security Agency (CISA). Mitigate dns infrastructure hijacking. https://www.cisa.gov/news-events/cybersecurity-advisories/aa19-024a, 2019. Accessed: 2025-09-10.

[21] NHS Digital. Cc-2885: Dnspionage campaign – dns hijacking activity. https://digital.nhs.uk/cyber-alerts/2019/cc-2885, 2019. Accessed: 2025-09-10.

[22] Taejoong Chung. Understanding the slow deployment of dnssec. In *Proceedings of the Internet Measurement Conference (IMC)*, 2017. Highlights low DNSSEC adoption due to deployment complexity, registrar coordination, and compatibility issues; only 1% of .com/.net/.org domains signed.

[23] Sheng Zeng and Xue Wu. The great firewall of china: A study of dns filtering. *International Journal of Computer Applications*, 96(18):28–35, 2014.

[24] Rima S Tanash, Zhouhan Chen, Tanmay Thakur, Dan S Wallach, and Devika Subramanian. Known unknowns: An analysis of twitter censorship in turkey. In *Proceedings of the 14th ACM Workshop on Privacy in the Electronic Society*, pages 11–20, 2015.

[25] Robert Jewitt and Majid Yar. Consuming the illegal: Situating piracy in everyday experience, 2013.

[26] Azadeh Akbari and Rashid Gabdulhakov. Platform surveillance and resistance in iran and russia: The case of telegram. *Surveillance & Society*, 17(1/2):223–231, 2019.

[27] Devashish Gosain, Anshika Agarwal, Sahil Shekhawat, H. B. Acharya, and S. Chakravarty. Mending wall: On the implementation of censorship in india. In *arXiv preprint arXiv:1806.06518*, 2018. Shows that 1

[28] ICANN Security and Stability Advisory Committee (SSAC). Sac127: Dns blocking revisited. ICANN SSAC Advisory, 2025. Discusses use of DNS blocking for illegal content and warns of collateral damage, over-blocking, lack of transparency.

[29] Luca Belli. Private ordering and the rise of terms of service as cyber-regulation. *International Journal of Law and Information Technology*, 2016. Argues intermediaries enforce rules via contract, shaping network architecture and public access.

[30] United Nations Human Rights Office. Internet shutdowns: trends, causes, legal implications and impacts on a range of human rights. https://datajournalism.com/read/longreads/internet-shutdowns-data-reporting, 2022. UN analysis reported by DataJournalism.com; highlights dramatic impacts of shutdowns on journalists, activists, and civil society.

[31] John D Day and Hubert Zimmermann. The osi reference model. *Proceedings of the IEEE*, 71(12):1334–1340, 1983.

[32] Roy Arends, Rob Austein, Matt Larson, Dan Massey, and Scott Rose. Dns security introduction and requirements. Technical report, 2005.

[33] Nevil Brownlee, Kimberly C Claffy, and Evi Nemeth. Dns measurements at a root server. In *GLOBECOM'01. IEEE Global Telecommunications Conference (Cat. No. 01CH37270)*, volume 3, pages 1672–1676. IEEE, 2001.

[34] W Toorop, S Dickinson, S Sahib, P Aras, and A Mankin. Rfc 9103: Dns zone transfer over tls, 2021.

[35] Hans Klein. Icann and internet governance: Leveraging technical coordination to realize global public policy. *The Information Society*, 18(3):193–207, 2002.

[36] Stefan Bechtold. Governance in namespaces. *Loy. LAL Rev.*, 36:1239, 2002.

[37] ICANN. Domain name registration process. https://www.icann.org/resources/pages/domain-name-registration-process-2023-11-02-en, 2023. Accessed: 24 September 2025.

[38] Wikipedia contributors. Domain name registry. https://en.wikipedia.org/wiki/Domain_name_registry, 2025. Accessed: 24 September 2025.

[39] Cloudflare. What is a domain name registrar? https://www.cloudflare.com/learning/dns/glossary/what-is-a-domain-name-registrar/, 2025. Accessed: 24 September 2025.

[40] Scott E Coull, Andrew M White, Ting-Fang Yen, Fabian Monrose, and Michael K Reiter. Understanding domain registration abuses. *Computers & security*, 31(7):806–815, 2012.

[41] Suranjith Ariyapperuma and Chris J Mitchell. Security vulnerabilities in dns and dnssec. In *The Second International Conference on Availability, Reliability and Security (ARES'07)*, pages 335–342. IEEE, 2007.

[42] Paul Hoffman and Jakob Schlyter. The dns-based authentication of named entities (dane) transport layer security (tls) protocol: Tlsa. Technical report, 2012.

[43] Sudheesh Singanamalla, Suphanat Chunhapanya, Marek Vavruša, Tanya Verma, Peter Wu, Marwan Fayed, Kurtis Heimerl, Nick Sullivan, and Christopher Wood. Oblivious dns over https (odoh): A practical privacy enhancement to dns. *arXiv preprint arXiv:2011.10121*, 2020.

[44] John Buford, Heather Yu, and Eng Keong Lua. *P2P networking and applications*. Morgan Kaufmann, 2009.

[45] Matei Ripeanu. Peer-to-peer architecture case study: Gnutella network. In *Proceedings first international conference on peer-to-peer computing*, pages 99–100. IEEE, 2001.

[46] Arnaud Legout, Guillaume Urvoy-Keller, and Pietro Michiardi. Understanding bittorrent: An experimental perspective. 2005.

[47] Sameh El-Ansary, Luc Onana Alima, Per Brand, and Seif Haridi. Efficient broadcast in structured p2p networks. In *International workshop on Peer-to-Peer systems*, pages 304–314. Springer, 2003.

[48] Ion Stoica, Robert Morris, David Liben-Nowell, David R Karger, M Frans Kaashoek, Frank Dabek, and Hari Balakrishnan. Chord: a scalable peer-to-peer lookup protocol for internet applications. *IEEE/ACM Transactions on networking*, 11(1):17–32, 2003.

[49] Álvaro García-Pérez, Alexey Gotsman, Yuri Meshman, and Ilya Sergey. Paxos consensus, deconstructed and abstracted. In *European Symposium on Programming*, pages 912–939. Springer International Publishing Cham, 2018.

[50] Junjie Hu and Ke Liu. Raft consensus mechanism and the applications. In *Journal of physics: conference series*, volume 1544, page 012079. IOP Publishing, 2020.

[51] Satoshi Nakamoto. Bitcoin whitepaper. *URL: https://bitcoin. org/bitcoin. pdf-(: 17.07. 2019)*, 9:15, 2008.

[52] Fahad Saleh. Blockchain without waste: Proof-of-stake. *The Review of financial studies*, 34(3):1156–1190, 2021.

[53] Andreas M Antonopoulos. *Mastering Bitcoin: unlocking digital cryptocurrencies*. " O'Reilly Media, Inc.", 2014.

[54] George Giamouridis, BooJoong Kang, and Leonardo Aniello. Blockchain-based dns: Current solutions and challenges to adoption. 2024.

[55] Harry A Kalodner, Miles Carlsten, Paul M Ellenbogen, Joseph Bonneau, and Arvind Narayanan. An empirical study of namecoin and lessons for decentralized namespace design. In *WEIS*, volume 1, pages 1–23, 2015.

[56] Fredy Andres Aponte-Novoa, Ana Lucila Sandoval Orozco, Ricardo Villanueva-Polanco, and Pedro Wightman. The 51% attack on blockchains: A mining behavior study. *IEEE access*, 9:140549–140564, 2021.

[57] Fran Casino, Nikolaos Lykousas, Vasilios Katos, and Constantinos Patsakis. Unearthing malicious campaigns and actors from the blockchain dns ecosystem. *Computer Communications*, 179:217–230, 2021.

[58] Muneeb Ali, Jude Nelson, Ryan Shea, and Michael J Freedman. Blockstack: A global naming and storage system secured by blockchains. In *2016 {USENIX} annual technical conference ({USENIX}{ATC} 16)*, pages 181–194, 2016.

[59] Pengcheng Xia, Haoyu Wang, Zhou Yu, Xinyu Liu, Xiapu Luo, and Guoai Xu. Ethereum name service: the good, the bad, and the ugly. *arXiv preprint arXiv:2104.05185*, 2021.

[60] Mirko Bez, Giacomo Fornari, and Tullio Vardanega. The scalability challenge of ethereum: An initial quantitative analysis. In *2019 ieee international conference on service-oriented system engineering (sose)*, pages 167–176. IEEE, 2019.

[61] Wajiha Rehman, Hijab e Zainab, Jaweria Imran, and Narmeen Zakaria Bawany. Nfts: Applications and challenges. In *2021 22nd International Arab Conference on Information Technology (ACIT)*, pages 1–7. IEEE, 2021.

[62] Audrey Randall, Wes Hardaker, Geoffrey M Voelker, Stefan Savage, and Aaron Schulman. The challenges of blockchain-based naming systems for malware defenders. In *2022 APWG Symposium on Electronic Crime Research (eCrime)*, pages 1–14. IEEE, 2022.

[63] Zecheng Li, Shang Gao, Zhe Peng, Songtao Guo, Yuanyuan Yang, and Bin Xiao. B-dns: A secure and efficient dns based on the blockchain technology. *IEEE Transactions on Network Science and Engineering*, 8(2):1674–1686, 2021.

[64] Wentong Wang, Ning Hu, and Xin Liu. Blockzone: A blockchain-based dns storage and retrieval scheme. In *International Conference on Artificial Intelligence and Security*, pages 155–166. Springer, 2019.

[65] Alexander Yakubov, Wazen Shbair, Anders Wallbom, David Sanda, et al. A blockchain-based pki management framework. In *The First IEEE/IFIP International Workshop on Managing and Managed by Blockchain (Man2Block) colocated with IEEE/IFIP NOMS 2018, Tapei, Tawain 23-27 April 2018*, 2018.

[66] Ankush Singla and Elisa Bertino. Blockchain-based pki solutions for iot. In *2018 IEEE 4th International Conference on Collaboration and Internet Computing (CIC)*, pages 9–15. IEEE, 2018.

[67] Zhi Guan, Abba Garba, Anran Li, Zhong Chen, and Nesrine Kaaniche. Authledger: A novel blockchain-based domain name authentication scheme. In *ICISSP*, pages 345–352, 2019.

[68] Xinan Duan, Zhiwei Yan, Guanggang Geng, and Baoping Yan. Dnsledger: Decentralized and distributed name resolution for ubiquitous iot. In *2018 IEEE International Conference on Consumer Electronics (ICCE)*, pages 1–3. IEEE, 2018.

[69] Wondeuk Yoon, Indal Choi, and Daeyoung Kim. Blockons: Blockchain based object name service. In *2019 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pages 219–226. IEEE, 2019.

[70] Xiangui Wang, Kedan Li, Hui Li, Yinghui Li, and Zhiwei Liang. Consortiumdns: A distributed domain name service based on consortium chain. In *2017 IEEE 19th International Conference on High Performance Computing and Communications; IEEE 15th International Conference on Smart City; IEEE 3rd International Conference on Data Science and Systems (HPCC/SmartCity/DSS)*, pages 617–620. IEEE, 2017.

[71] Peter Danielis, Vlado Altmann, Jan Skodzik, Tim Wegner, Achim Koerner, and Dirk Timmermann. P-donas: a p2p-based domain name system in access networks. *ACM Transactions on Internet Technology (TOIT)*, 15(3):1–21, 2015.

[72] Ricardo Sancho and Ricardo Lopes Pereira. Hybrid peer-to-peer dns. In *2014 International Conference on Computing, Networking and Communications (ICNC)*, pages 977–981. IEEE, 2014.

[73] Russ Cox, Athicha Muthitacharoen, and Robert T Morris. Serving dns using a peer-to-peer lookup service. In *International Workshop on Peer-To-Peer Systems*, pages 155–165. Springer, 2002.

[74] Yiting Song and Keiichi Koyanagi. Study on a hybrid p2p based dns. In *2011 IEEE International Conference on Computer Science and Automation Engineering*, volume 4, pages 152–155. IEEE, 2011.

[75] Stefan Dziembowski, Sebastian Faust, Vladimir Kolmogorov, and Krzysztof Pietrzak. Proofs of space. In *Annual Cryptology Conference*, pages 585–605. Springer, 2015.

[76] Giuseppe Ateniese, Randal Burns, Reza Curtmola, Joseph Herring, Lea Kissner, Zachary Peterson, and Dawn Song. Provable data possession at untrusted stores. In *Proceedings of the 14th ACM conference on Computer and communications security*, pages 598–609, 2007.

[77] Bram Cohen and Krzysztof Pietrzak. Simple proofs of sequential work. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 451–467. Springer, 2018.

[78] Tal Moran and Ilan Orlov. Simple proofs of space-time and rational proofs of storage. In *Advances in Cryptology–CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part I 39*, pages 381–409. Springer, 2019.

[79] Chia Network. Chia consensus whitepaper. `https://www.chia.net`, 2021. Accessed: 2025-04-07.

[80] Joao Vasco Estrela Martinho. *Efficient Proof-of-Space approaches for permissionless blockchains*. PhD thesis, Instituto Superior Técnico Lisbon, Portugal, 2022.

[81] Diogo Peres Castilho. Etherspace: Practical proof-of-space for blockchains. 2019.

[82] Davi Pedro Bauer. Filecoin. In *Getting Started with Ethereum: A Step-by-Step Guide to Becoming a Blockchain Developer*, pages 97–101. Springer, 2022.

[83] Ben Fisch, Joseph Bonneau, Nicola Greco, and Juan Benet. Scaling proof-of-replication for filecoin mining. *Report. Protocol Labs Research*, 2018.

[84] Amir Dembo, Sreeram Kannan, Ertem Nusret Tas, David Tse, Pramod Viswanath, Xuechao Wang, and Ofer Zeitouni. Everything is a race and nakamoto always wins. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 859–878, 2020.

[85] Shuyang Tang, Jilai Zheng, Yao Deng, Ziyu Wang, Zhiqiang Liu, Dawu Gu, Zhen Liu, and Yu Long. Towards a multi-chain future of proof-of-space. In *Security and Privacy in Communication Networks: 15th EAI International Conference, SecureComm 2019, Orlando, FL, USA, October 23-25, 2019, Proceedings, Part I 15*, pages 23–38. Springer, 2019.

[86] Seth James Nielson, Scott A Crosby, and Dan S Wallach. A taxonomy of rational attacks. In *International workshop on peer-to-peer systems*, pages 36–46. Springer, 2005.

[87] John R Douceur. The sybil attack. In *International workshop on peer-to-peer systems*, pages 251–260. Springer, 2002.

[88] Christine V Helliar, Louise Crawford, Laura Rocca, Claudio Teodori, and Monica Veneziani. Permissionless and permissioned blockchain diffusion. *International Journal of Information Management*, 54:102136, 2020.

[89] Ali J Fahs and Guillaume Pierre. Proximity-aware traffic routing in distributed fog computing platforms. In *2019 19th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*, pages 478–487. IEEE, 2019.

[90] Chia Network Inc. Chia blockchain - plotting implementation. https://github.com/Chia-Network/chia-blockchain, 2025. Accessed: 2025-06-18.

[91] Joel Weise. Public key infrastructure overview. *Sun BluePrints OnLine, August*, pages 1–27, 2001.

[92] Joppe W Bos, J Alex Halderman, Nadia Heninger, Jonathan Moore, Michael Naehrig, and Eric Wustrow. Elliptic curve cryptography in practice. In *Financial Cryptography and Data Security: 18th International Conference, FC 2014, Christ Church, Barbados, March 3-7, 2014, Revised Selected Papers 18*, pages 157–175. Springer, 2014.

[93] Georg Becker. Merkle signature schemes, merkle trees and their cryptanalysis. *Ruhr-University Bochum, Tech. Rep*, 12:19, 2008.

[94] Guido Urdaneta, Guillaume Pierre, and Maarten Van Steen. A survey of dht security techniques. *ACM Computing Surveys (CSUR)*, 43(2):1–49, 2011.

[95] Dan Boneh and Victor Shoup. A graduate course in applied cryptography. *Draft 0.5*, 2020.

[96] Boris Ederov. Merkle tree traversal techniques. *Bachelor Thesis, Darmstadt University of Technology Department of Computer Science Cryptography and Computer Algebra*, 2007.

[97] Chia Network. Chia nodes, 2023. Accessed: 2023-04-08.

[98] Lutz Prechelt. An empirical comparison of seven programming languages. *Computer*, 33(10):23–29, 2000.

[99] Pietro Danzi, Anders Ellersgaard Kalor, Cedomir Stefanovic, and Petar Popovski. Analysis of the communication traffic for blockchain synchronization of iot devices. In *2018 IEEE international conference on communications (ICC)*, pages 1–7. IEEE, 2018.

[100] Chia Docs. Cli commands reference, 2025.

[101] Chia. Chia proof of space construction, 2020.

[102] Ling Ren and Srinivas Devadas. Proof of space from stacked expanders. In *Theory of Cryptography Conference*, pages 262–285. Springer, 2016.

[103] Daniel Kraft. Difficulty control for blockchain-based consensus systems. *Peer-to-peer Networking and Applications*, 9:397–413, 2016.